The Network Layer

Chapter 5 Overview

Based on Computer Networks, Fifth Edition by Andrew Tanenbaum and David Wetherall

Disclaimer

Compilation of Sources: These slides compile content from various academic resources, standard networking textbooks (including Andrew S. Tanenbaum's Computer Networks), and instructional materials.

AI-Assisted Content Generation: Certain diagrams, visual aids, and conceptual illustrations in these slides were generated, refined, or structured with the help of AI tools (such as Gemini, ChatGPT, and Claude) for educational clarity.

Academic Purpose & Permissions: These slides are prepared strictly for educational, non-commercial teaching purposes. Academic use, sharing, and distribution for classroom instruction are permitted, provided proper attribution to the original author and source materials is maintained.

Network Layer Design Issues

- Store-and-forward packet switching
- Services provided to transport layer
- Implementation of connectionless service
- Implementation of connection-oriented service
- Comparison of virtual-circuit and datagram networks

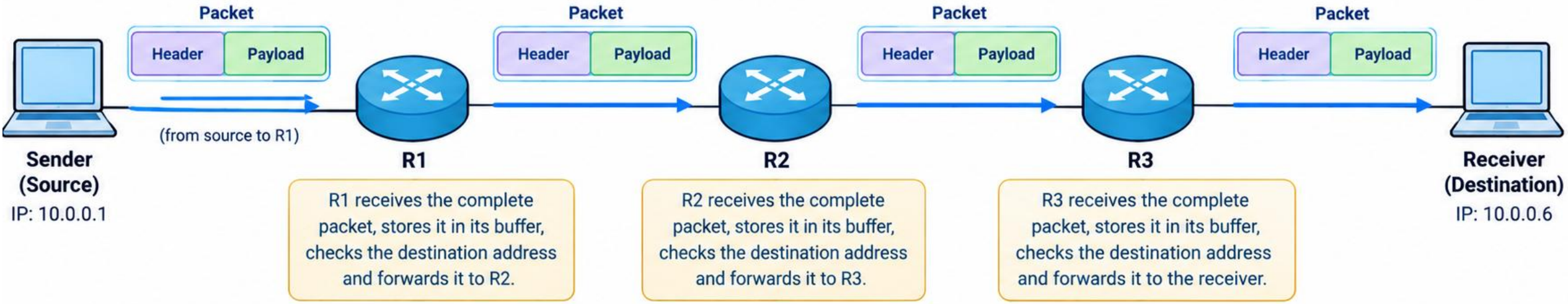
Store-and-Forward Packet Switching

The environment of the network layer protocols requires data to traverse multiple intermediate devices.

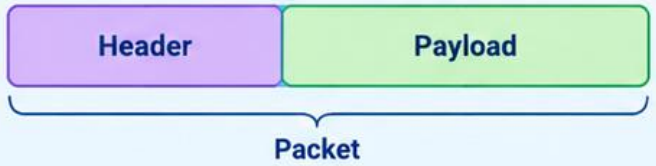
- A host with a packet to send, transmits it to the nearest router, either on its own LAN or a point-to-point link to the carrier.
- The packet is stored at the router until it has fully arrived and its checksum has been verified.
- Once fully received and validated, it is forwarded to the next router along the path until it reaches the destination.

Store-and-Forward Packet Switching

Each router receives the complete packet (header + payload),
stores it in its buffer, then checks the destination and forwards it to the next hop.



Packet Format (example)

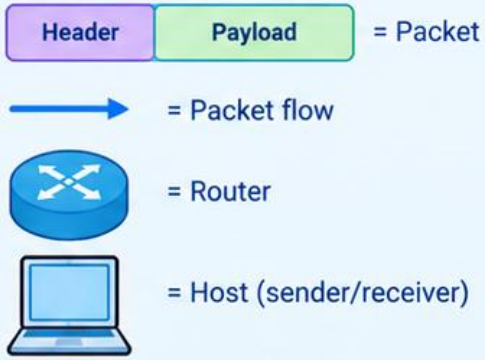


- **Header:** contains control information (e.g., source and destination addresses, sequence number, etc.)
- **Payload:** actual user data

Steps in Store and Forward

1. R1 receives the entire packet from the sender and stores it in its buffer.
2. R1 checks the destination address in the packet header and determines the next hop (R2).
3. R1 forwards the packet to R2 over the outgoing link.
4. R2 receives the complete packet, stores it, performs the same process and forwards it to R3.
5. R3 does the same and finally forwards the packet to the receiver.

Legend



V.U.

Services Provided to the Transport Layer

The network layer must provide a clear and reliable interface to the transport layer above it.



Independence

Services should be independent of the underlying router technology.



Shielding

The transport layer is shielded from the number, type, and topology of the routers present in the subnet.



Uniform Addressing

Network addresses available to the transport layer use a uniform numbering plan, even across LANs and WANs.

Implementation of Connectionless Service

Datagram Networks

If a connectionless service is offered, packets are injected into the subnet individually and routed independently of each other.

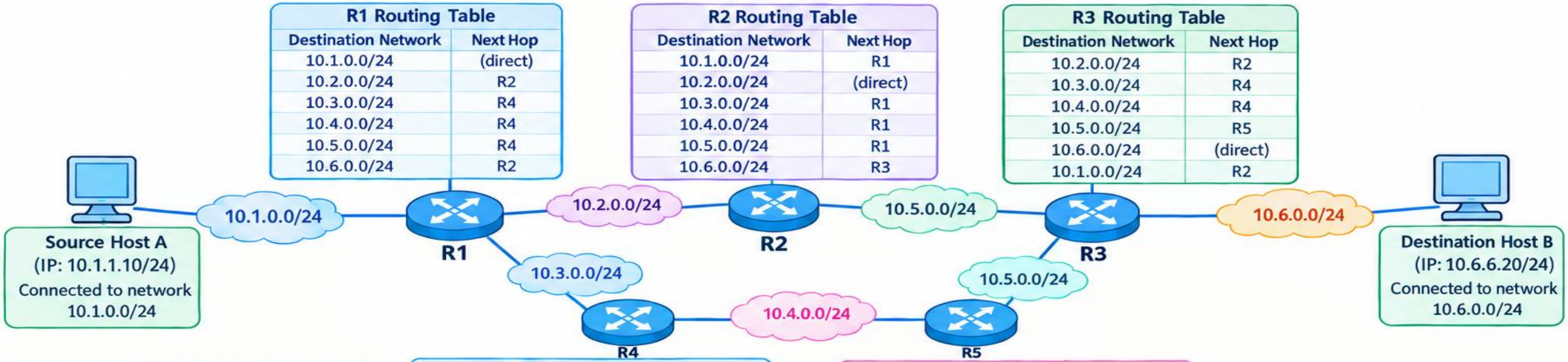
No advance setup is needed. In this context, the packets are frequently called **datagrams** (in analogy to telegrams).

The network that manages them is called a **datagram network**.

- Packets contain the full destination address.
- Routers do not hold state about connections.
- Routes can change dynamically per packet.

Routing Within a Datagram Network (Connectionless)

Each datagram is routed independently. Routers use their routing tables to forward each datagram without establishing a connection.



R1 Routing Table

Destination Network	Next Hop
10.1.0.0/24	(direct)
10.2.0.0/24	R2
10.3.0.0/24	R4
10.4.0.0/24	R4
10.5.0.0/24	R4
10.6.0.0/24	R2

R2 Routing Table

Destination Network	Next Hop
10.1.0.0/24	R1
10.2.0.0/24	(direct)
10.3.0.0/24	R1
10.4.0.0/24	R1
10.5.0.0/24	R1
10.6.0.0/24	R3

R3 Routing Table

Destination Network	Next Hop
10.2.0.0/24	R2
10.3.0.0/24	R4
10.4.0.0/24	R4
10.5.0.0/24	R5
10.6.0.0/24	(direct)
10.1.0.0/24	R2

R4 Routing Table

Destination Network	Next Hop
10.3.0.0/24	(direct)
10.4.0.0/24	R5
10.1.0.0/24	R1
10.2.0.0/24	R1
10.5.0.0/24	R5
10.6.0.0/24	R5

R5 Routing Table

Destination Network	Next Hop
10.4.0.0/24	(direct)
10.5.0.0/24	R3
10.6.0.0/24	R3
10.1.0.0/24	R4
10.2.0.0/24	R4
10.3.0.0/24	R4

Legend

- Network (with subnet)
- Path taken by datagram (example: 10.1.1.10 → 10.6.6.20)
- Alternative path (if available)
- Router
- Host

How Routing Works (Connectionless)

1. Host A creates a datagram with destination IP 10.6.6.20 and sends it to its default router (R1).
2. R1 looks up its routing table and forwards the datagram to R2 (next hop), because 10.6.0.0/24 is reachable via R2.
3. R2 looks up its routing table and forwards the datagram to R1 (next hop), because 10.3.0.0/24 is reachable via R1.
4. R1 forwards the datagram to R4 (next hop) for 10.3.0.0/24.
5. R4 forwards the datagram to R5 (next hop) for 10.4.0.0/24.
6. R5 forwards the datagram to R3 (next hop) for 10.5.0.0/24.
7. R3 delivers the datagram to the destination network 10.6.0.0/24 (directly connected).

Key Points

- Next hop must be a directly connected neighbor router (not a non-neighbor).
- **Each** router has a routing table with entries for all reachable networks (including the destination network if directly connected).
- Packets are forwarded hop-by-hop using the next hop from the table.
- No connection is established; each datagram is routed independently.

Implementation of Connection-Oriented Service

Virtual-Circuit Networks

If connection-oriented service is used, a path from the source router to the destination router must be established before any data packets can be sent.

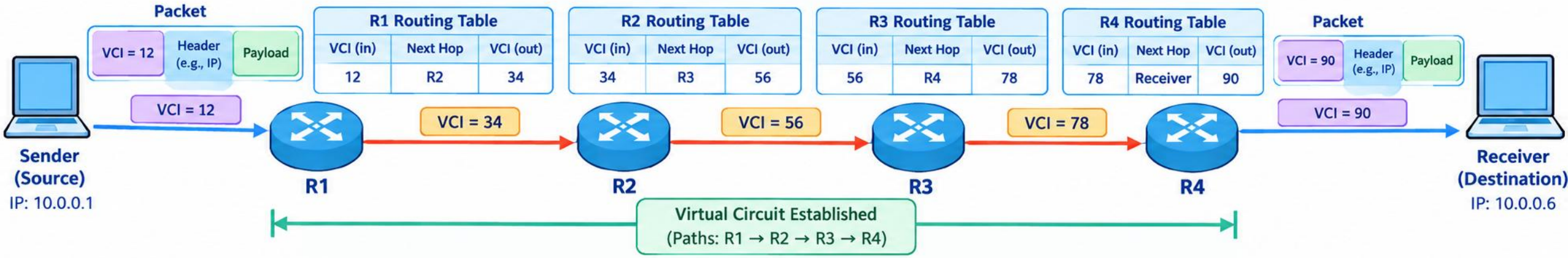
This connection is called a **VC (Virtual Circuit)**.

The network is called a **virtual-circuit network**.

- Requires an initial setup phase to establish the circuit.
- Routers must maintain state for every active connection.
- Routing decisions are made once during setup; packets follow the predefined path.

Routing Within a Virtual-Circuit Network

A virtual circuit is established before data transfer. Each packet carries a virtual circuit identifier (VCI) and routes forward the packet based on their routing tables for that virtual circuit.



Packet Format (example)

VCI = 12 (Virtual Circuit ID)	Header (e.g., IP)	Payload

- **VCI:** identifies the virtual circuit
- **Header:** contains control information (e.g., source/destination IP, sequence number, etc.)
- **Payload:** actual user data

Steps in Routing (Virtual Circuit)

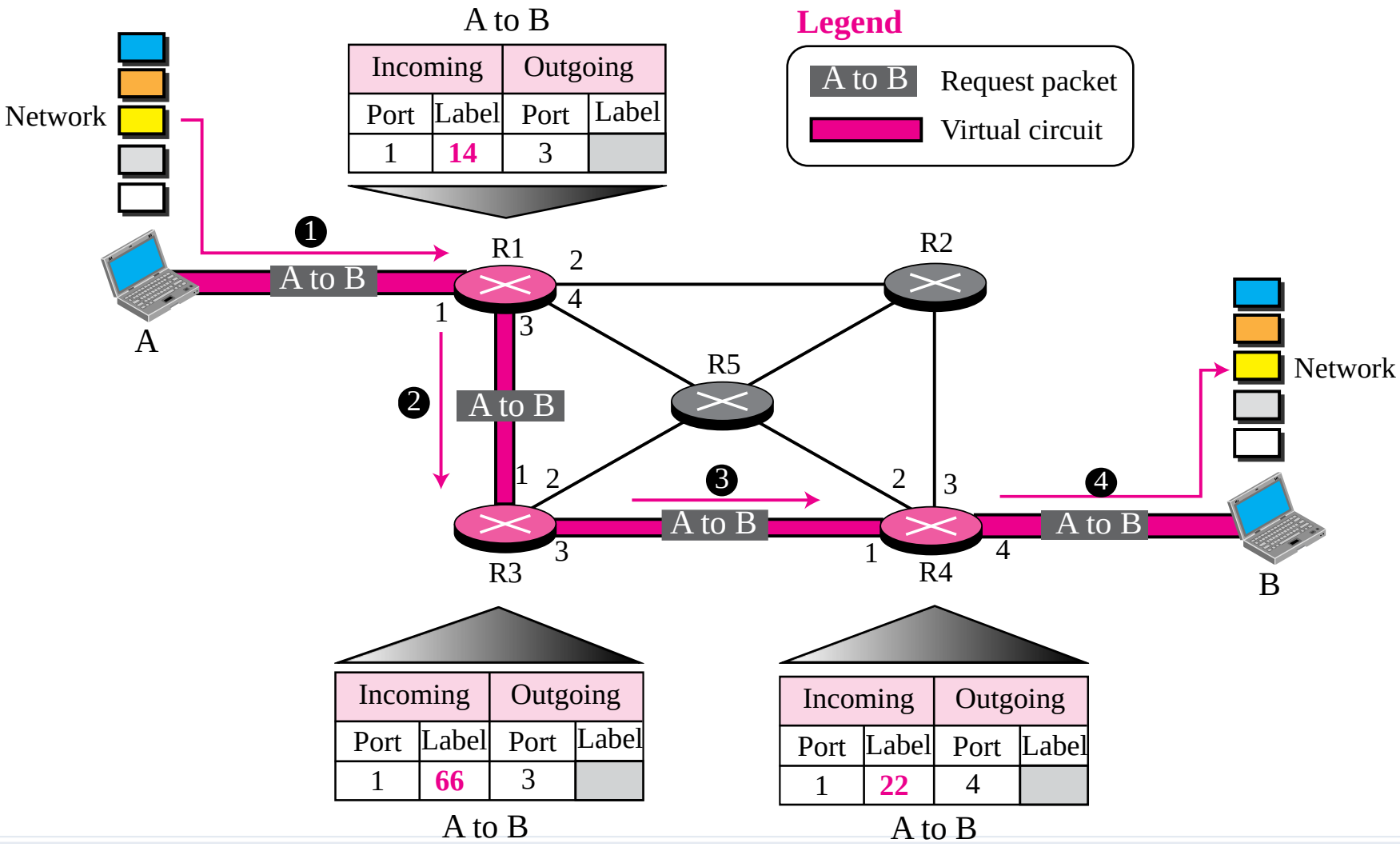
1. A virtual circuit is established between the source and destination (using a setup message).
2. R1 receives the packet with VCI = 12 and looks up its routing table. It forwards the packet to R2 with VCI = 34.
3. R2 receives the packet with VCI = 34 and forwards it to R3 with VCI = 56.
4. R3 receives the packet with VCI = 56 and forwards it to R4 with VCI = 78.
5. R4 receives the packet with VCI = 78 and delivers it to the destination.

Legend

- VCI = Virtual circuit identifier
- = Packet flow
- = Established virtual circuit path
- = Router
- = Host (sender/receiver)

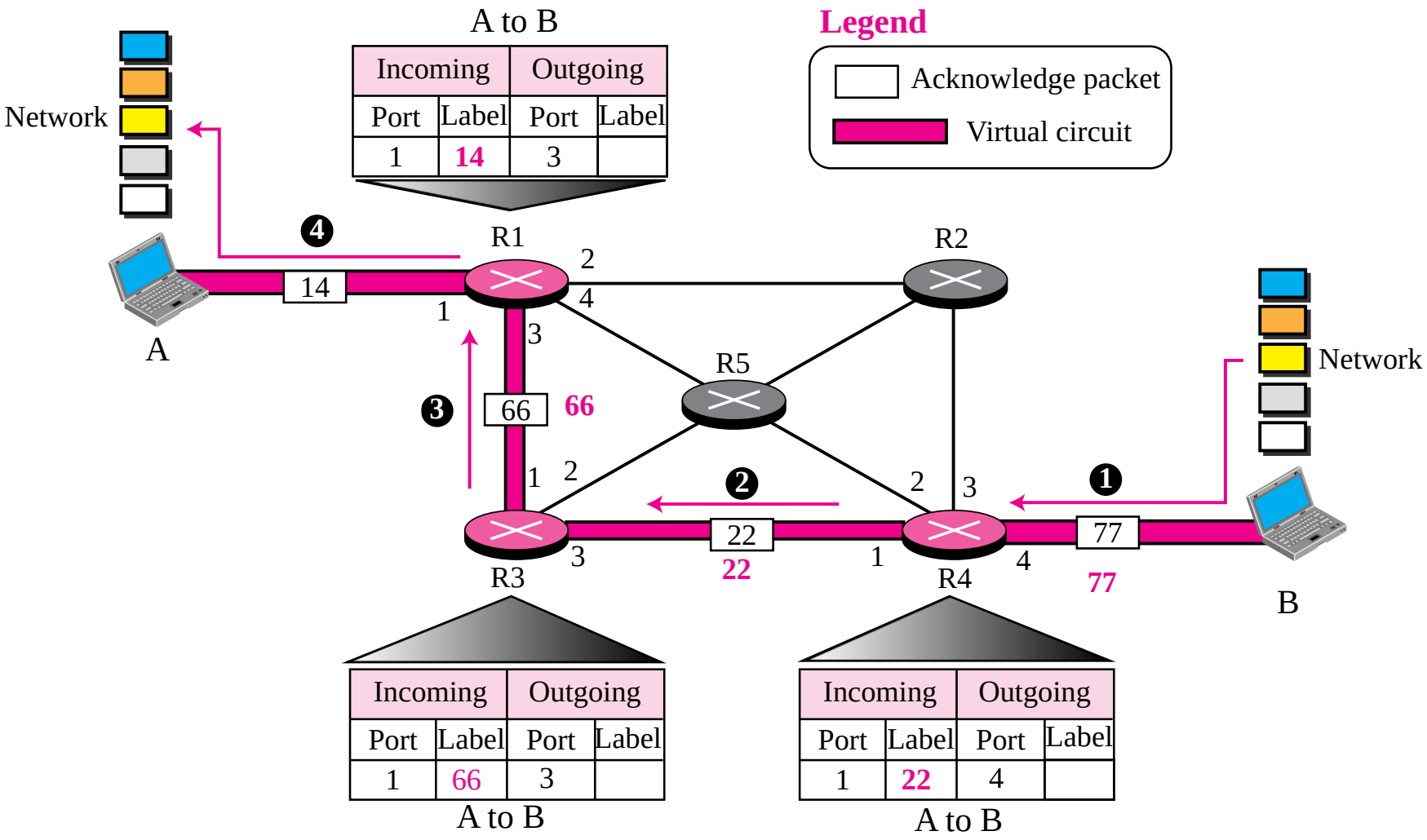
Routing Within a Virtual-Circuit Network

Example of routing tables mapping incoming interfaces to outgoing interfaces.



Routing Within a Virtual-Circuit Network

Example of routing tables mapping incoming interfaces to outgoing interfaces.



Virtual-Circuit vs. Datagram Networks

A direct comparison of the fundamental architectural choices in network layer design.

Comparison

Issue	Datagram Network	Virtual-Circuit Network
Circuit Setup	Not needed	Required
Addressing	Each packet contains the full source and destination address	Each packet contains a short VC number
State Information	Routers do not hold state information about connections	Each VC requires router table space per connection
Routing	Each packet is routed independently	Route chosen when VC is set up; all packets follow it
Effect of Router Failures	None, except for packets lost during the crash	All VCs that passed through the failed router are terminated
Quality of Service	Difficult to guarantee	Easy if enough resources can be allocated in advance for each VC

Routing Algorithms (1)

The logic utilized by network devices to determine the optimal paths across subnets.

Routing Vs. Forwarding

Routing (The Brains)

Routing is the process of making the decision regarding which paths or routes to use.

It involves algorithms running in the background to build the network maps and calculate optimal trajectories.

Forwarding (The Brawn)

Forwarding is what happens when a packet actually arrives at a router.

The router looks up the outgoing line to use for it in the pre-computed routing tables. This process is immediate and hardware-driven.

Desirable Properties of Routing Algorithms

- **Correctness:** The right path is chosen reliably.
- **Simplicity:** Ease of use and implementation within router hardware.
- **Robustness:** Reliability in the event of software or hardware failure.
- **Stability:** Algorithms converge to a fixed set of paths without severe oscillation.
- **Fairness:** Fair to all hosts trying to transmit data.
- **Efficiency:** Maximizing network utilization and minimizing delay.

Fairness vs. Efficiency

Routing algorithms must often compromise between fairness and efficiency.

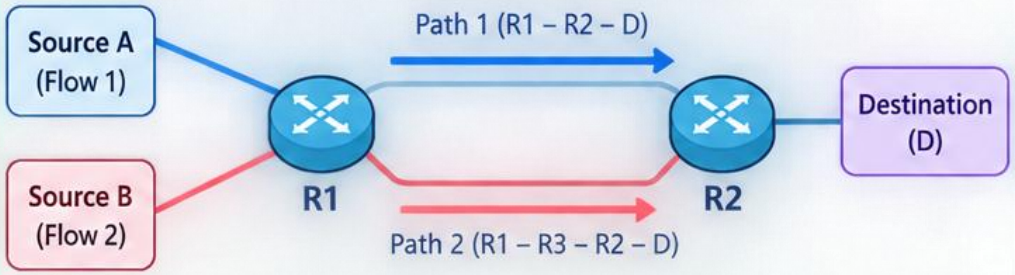
- ⚖️ Maximizing overall efficiency (throughput) might mean completely starving certain hosts of bandwidth.
- 🤝 Enforcing strict fairness ensures everyone gets to transmit, but can severely reduce the total data volume the network carries.

Fairness vs. Efficiency Example

Routing can be designed to be fair (equal share for all flows) or efficient (use network resources better), **but both goals may not be achieved together.**

Fairness (Equal Share)

Each flow gets an equal share of the available bandwidth, even if some paths are less efficient.



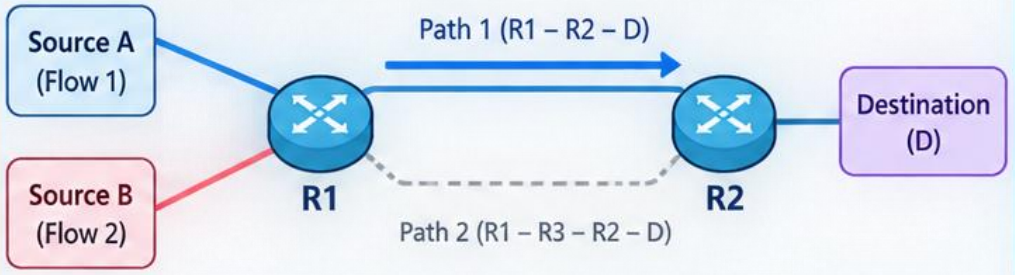
Bandwidth Allocation (Fairness)

Flow	Path Used	Bandwidth Share
Flow 1 (A → D)	Path 1	50%
Flow 2 (B → D)	Path 2	50%

- Both flows get equal share (fair).
- But Flow 2 uses a longer path (less efficient).

Efficiency (Shortest Path / Least Cost)

Routes are chosen to minimize total cost / delay, even if it leads to unequal bandwidth allocation.



Bandwidth Allocation (Efficiency)

Flow	Path Used	Bandwidth Share
Flow 1 (A → D)	Path 1	100%
Flow 2 (B → D)	Path 2 (not used)	0%

- Flow 1 gets full bandwidth (more efficient).
- Flow 2 gets no bandwidth (unfair).

Types of Routing Algorithms

Nonadaptive (Static)

Do not base their routing decisions on any measurements or estimates of the current topology and traffic.

Routes are computed in advance, offline, and downloaded to the routers when the network boots.

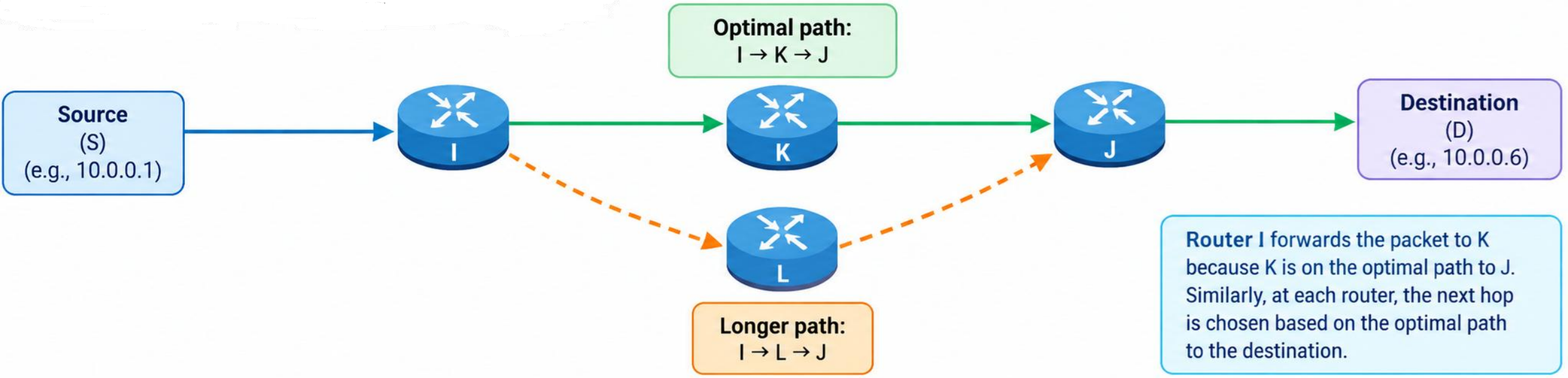
Adaptive (Dynamic)

Change their routing decisions to reflect changes in the topology, and sometimes changes in the traffic as well.

They differ in where they get their information, when they change routes, and what metric is used for optimization (e.g., Distance Vector, Link State).

The Optimality Principle

A general statement about optimal routes without regard to network topology or traffic.



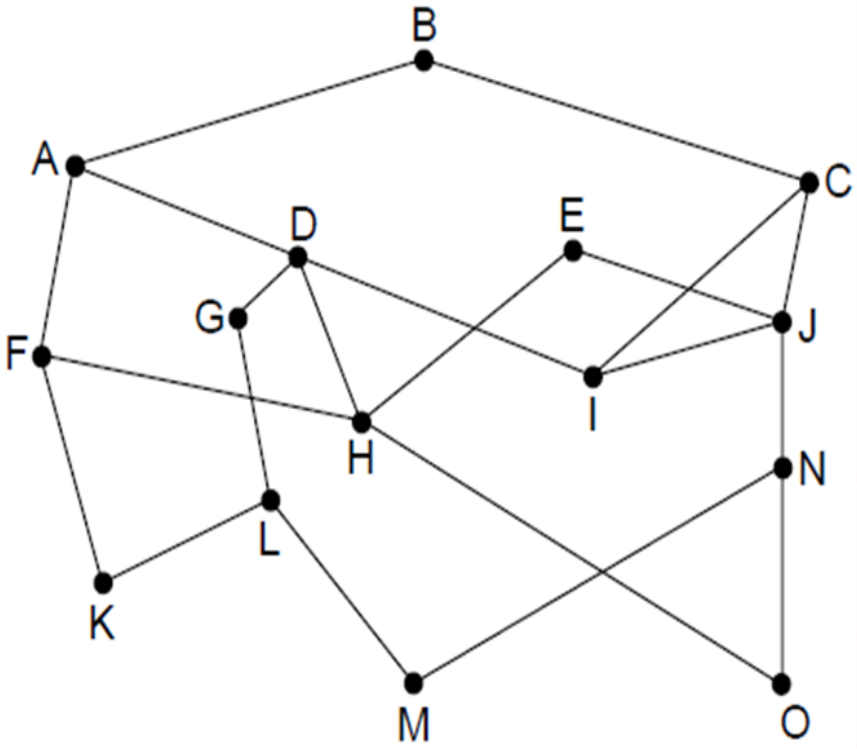
If a router is on the optimal path from source to destination, then the optimal path from that router to the destination is the same as the overall optimal path.

The Sink Tree

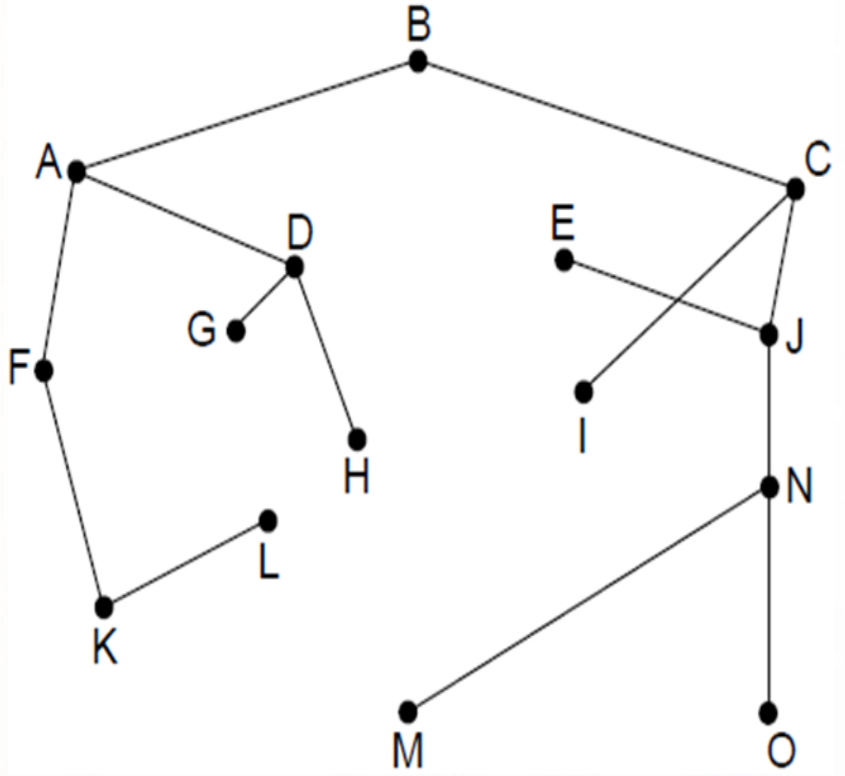
Definition

By the optimality principle, the set of optimal routes from all sources to a given destination forms a **tree** rooted at the destination, called a **sink tree**.

(a) A network topology



(b) Sink tree for router B

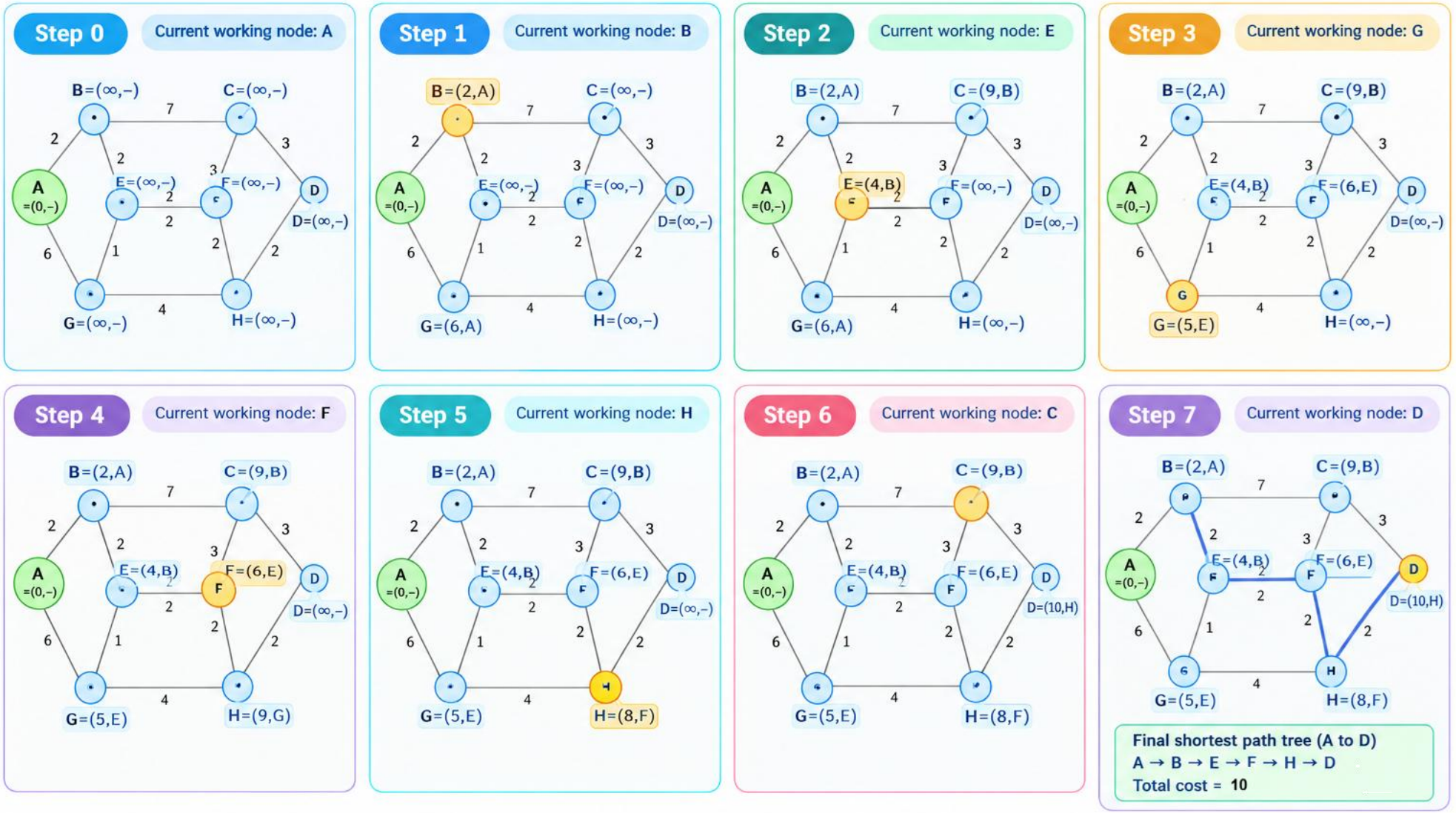


Shortest Path Routing

The goal is to find the shortest path between two nodes in a graph representing the network.

- The network is modeled as a graph, where routers are nodes and communication lines are edges.
- Each edge is assigned a "weight" or cost, which could represent physical distance, delay, bandwidth cost, or a combination of metrics.
- **Dijkstra's Algorithm** is universally used to compute the shortest path through a network graph.

Dijkstra's Shortest Path Algorithm



Dijkstra's Algorithm (Implementation 1)

```
#define MAX_NODES 1024                  /* maximum number of nodes */
#define INFINITY 1000000000             /* a number larger than every maximum path */
int n, dist[MAX_NODES][MAX_NODES];      /* dist[i][j] is the distance from i to j */

void shortest_path(int s, int t, int path[])
{ struct state {                        /* the path being worked on */
      int predecessor;                  /* previous node */
      int length;                       /* length from source to this node */
      enum {permanent, tentative} label;    /* label state */
  } state[MAX_NODES];

  int i, k, min;
  struct state *p;

for (p = &state[0]; p < &state[n]; p++) {    /* initialize state */
     p->predecessor = –1;
     p->length = INFINITY;
     p->label = tentative;
}
state[t].length = 0;  state[t].label = permanent;
k = t;                                  /* k is the initial working node */
```

Dijkstra's Algorithm (Implementation 2)

```
do {                                          /* Is there a better path from k? */
    for (i = 0; i < n; i++)                   /* this graph has n nodes */
        if (dist[k][i] != 0 && state[i].label == tentative) {
            if (state[k].length + dist[k][i] < state[i].length) {
                state[i].predecessor = k;
                state[i].length = state[k].length + dist[k][i];
            }
        }
        /* Find the tentatively labeled node with the smallest label. */
        k = 0; min = INFINITY;
        for (i = 0; i < n; i++)
            if (state[i].label == tentative && state[i].length < min) {
                min = state[i].length;
                k = i;
            }
        state[k].label = permanent;
    } while (k != s);

    /* Copy the path into the output array. */
    i = 0;  k = s;
    do {path[i++] = k; k = state[k].predecessor; } while (k >= 0);
}
```

Distance Vector Routing

- An adaptive routing algorithm where routers maintain a vector (table) giving the best known distance to each destination and which line to use to get there.
- These tables are updated solely by exchanging information with direct neighbors.
- It is also known as the Bellman-Ford algorithm and was the original ARPANET routing algorithm.
- Key Characteristics:
  - Distributed: No router has complete knowledge of the network.
  - Neighbor-based: Routers learn routes from neighboring routers.
  - Iterative: Routes are progressively improved.
  - Dynamic: Changes in link costs or topology trigger route updates.
  - Limitation: Can suffer from the count-to-infinity problem and slow convergence.

Distance Vector Updates

How the routing tables are maintained and updated:



Periodic Broadcasts

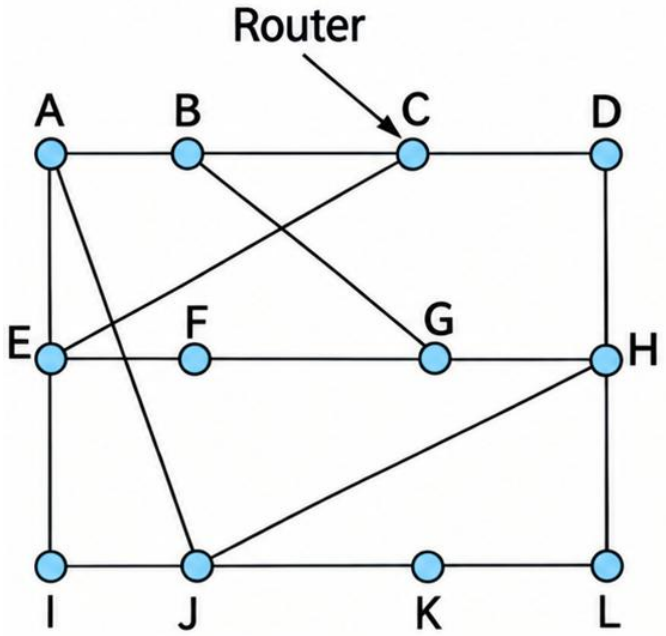
Each router periodically receives a routing table from each of its neighbors showing their current best paths.



Calculation

It computes its own new table by adding the cost to reach the neighbor to the neighbor's advertised distance to the ultimate destination.

Distance Vector Routing Example



(a) A network.

To	A	I	H	K	New estimated delay from J	Line
A	0	24	20	21	8	A
B	12	36	31	28	20	A
C	25	19	19	36	28	I
D	40	27	8	24	20	H
E	14	7	30	22	17	I
F	23	20	19	40	30	I
G	18	31	6	31	18	H
H	17	20	0	19	12	H
I	21	0	14	22	10	I
J	9	11	7	10	0	–
K	24	22	22	0	6	K
L	29	33	9	9	15	K
	JA delay is 8	JI delay is 10	JH delay is 12	JK delay is 6	New routing table for J	

Vectors received from J's four neighbors

(b) Input from A, I, H, K, and the new routing table for J.

The Count-to-Infinity Problem

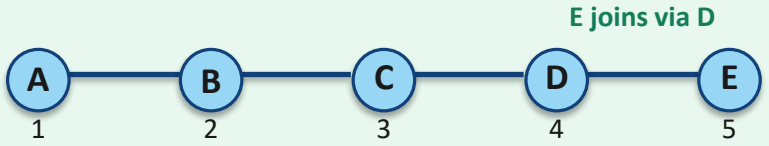
- A major flaw in the Distance Vector algorithm is its reaction to bad news
- The algorithm reacts rapidly to good news (e.g., discovering a new shorter path).
- However, it reacts very slowly to bad news (e.g., a link failing or going down).

∞ Because nodes blindly trust neighbors' outdated metrics, they can enter routing loops where node distances slowly increment to infinity.

The Count-to-Infinity Problem

Good news: New node joins (E) — converges quickly

A new node E becomes reachable through D. Its distance information propagates hop-by-hop and quickly reaches all routers.



Distance to destination E after each exchange

Exchange	A→E	B→E	C→E	D→E	E→E
Initial	∞	∞	∞	∞	0
1	∞	∞	∞	1	0
2	∞	∞	2	1	0
3	∞	3	2	1	0
4	4	3	2	1	0

Good news: information moves in the correct direction.
A better/new route is learned, and the network stabilizes after only a few exchanges.

Bad news: Link D–E fails — converges very slowly

After D–E fails, D sets its distance to E to ∞. C still advertises its old route, so D temporarily believes E is reachable via C; the estimate then increases one hop at a time.

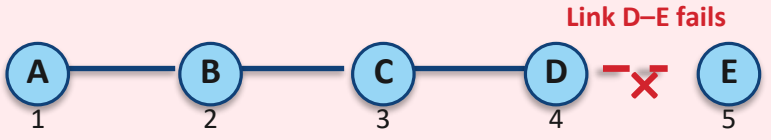


Illustration of the count-to-infinity sequence

Exchange	B→E	C→E	D→E	What happens
Before failure	3	2	1	Stable route
1	3	2	-	D detects failure
2	5	4	3	D uses C's stale route
3	5	4	5	C and D increase estimates
4	7	6	5	Old information persists
5	7	6	7	Count continues
…	..	..	..	..
N	∞	∞	∞	

C advertises → D updates → D advertises → C updates → …

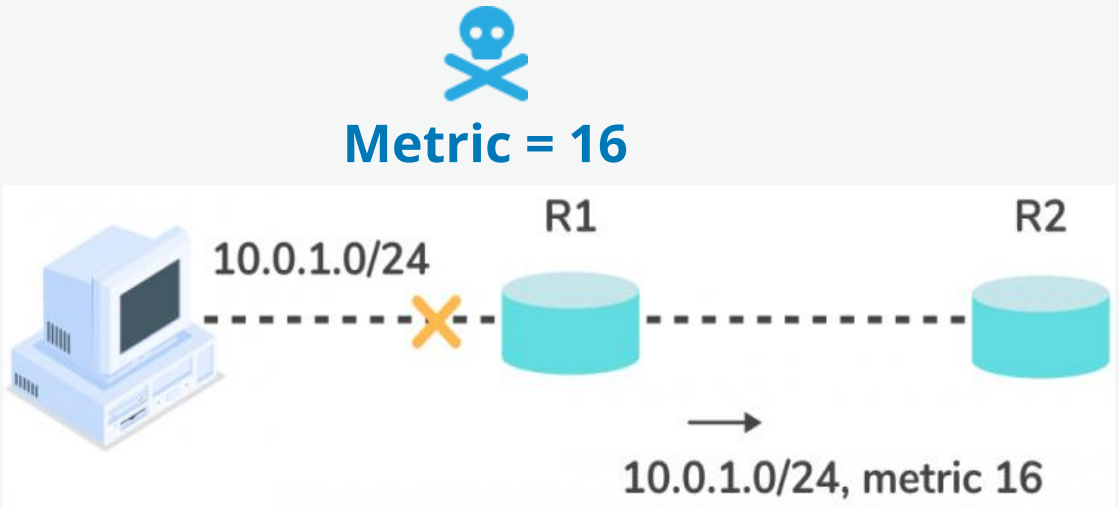
Count-to-infinity: stale routing information creates a loop and the metric rises slowly toward ∞.

Concept shown without split horizon / poison reverse, so the classic count-to-infinity behavior is visible.

Route Poisoning

Spreading Bad News

- When a route fails, distance vector protocols spread the "bad news" about a route failure by poisoning the route.
- This means advertising the route with a special metric value called **Infinity**.
- The actual value of Infinity is a known constant, which may be different for different routing protocols (e.g., 16 for RIP).

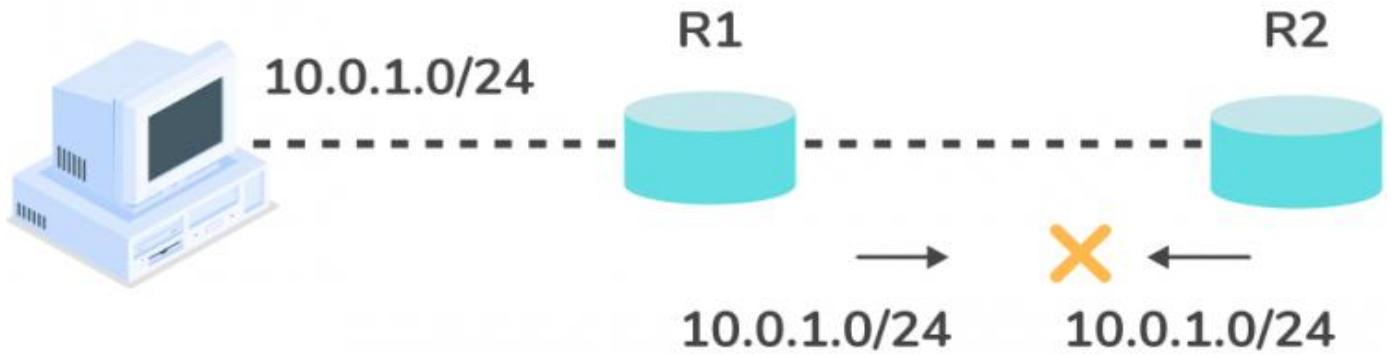


Any router receiving a route with metric 16 instantly knows the destination is unreachable.

Split Horizon

A technique to prevent two-node routing loops in Distance Vector protocols.

- Each node delivers a portion of its table across each interface rather than flooding the entire table everywhere.
- Specifically, a router will not broadcast a route back to the neighbor from which it learned that route.
- If node R2 believes that the best way to go to X is through node R1, then node R2 need not inform node R1 of this information because it has already been provided by node R1.



Link State Routing

A modern alternative to Distance Vector, used in widely deployed protocols like OSPF and IS-IS. Each router builds a complete map of the network topology.

Link State Routing Steps

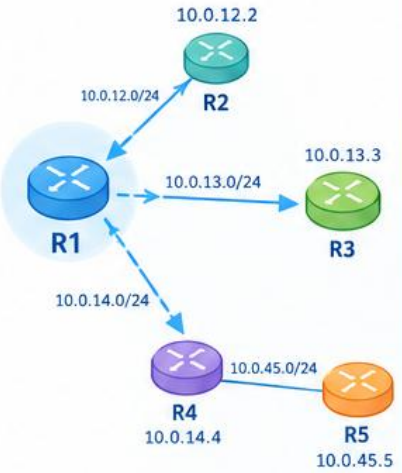
To implement Link State Routing, each router must accomplish five things:

1. Discover its neighbors and learn their network addresses.
2. Measure the delay or cost to each of its neighbors.
3. Construct a packet telling all it has just learned.
4. Send this packet to, and receive packets from, all other routers.
5. Compute the shortest path to every other router locally.

Link State Routing Steps

1 Discover neighbors

R1 sends HELLO messages to directly connected routers.

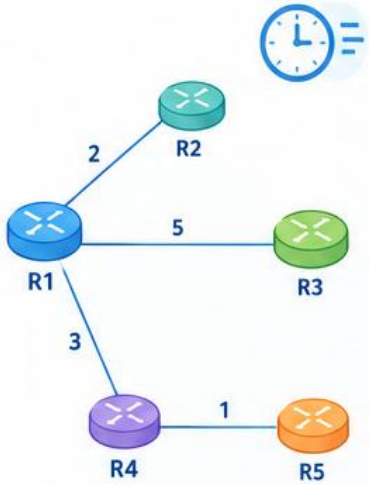


R1's neighbors

- R2 (10.0.12.2)
- R3 (10.0.13.3)
- R4 (10.0.14.4)

2 Measure link cost

Determine delay / cost for each link.



Link costs (delay)

R1 – R2 : 2
R1 – R3 : 5
R1 – R4 : 3
R4 – R5 : 1

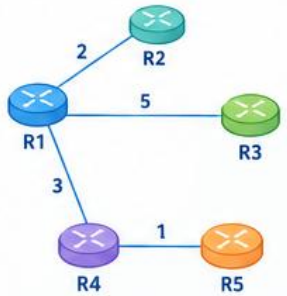
3 Create link-state packet

R1 builds an LSP (LSA) with its neighbors and costs.



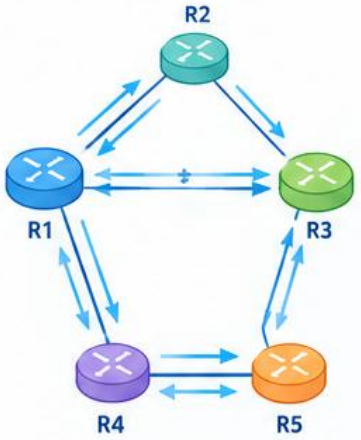
LSP / LSA (from R1)

Origin Router : R1
Neighbor List :
R2 (cost 2)
R3 (cost 5)
R4 (cost 3)



4 Flood the packet

LSP propagates hop-by-hop to all routers.



LSP reaches all routers
(R1 → R2, R3, R4, R5)

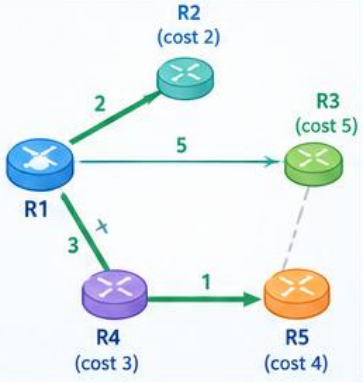
5 Compute shortest paths

R1 builds a link-state database and runs Dijkstra.

Link-State Database (R1)

Router	Neighbors (cost)
R1	R2 (2), R3 (5), R4 (3)
R2	R1 (2), R5 (4)
R3	R1 (5), R5 (2)
R4	R1 (3), R5 (1)
R5	R2 (4), R3 (2), R4 (1)

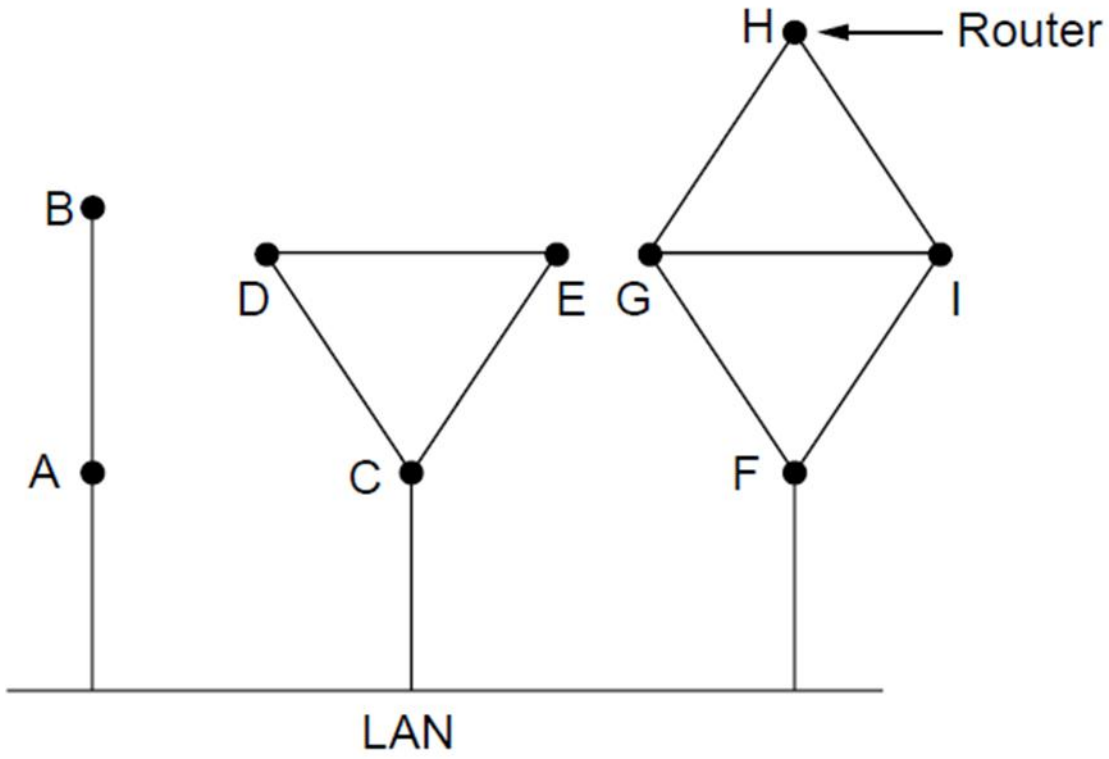
Shortest Path Tree from R1

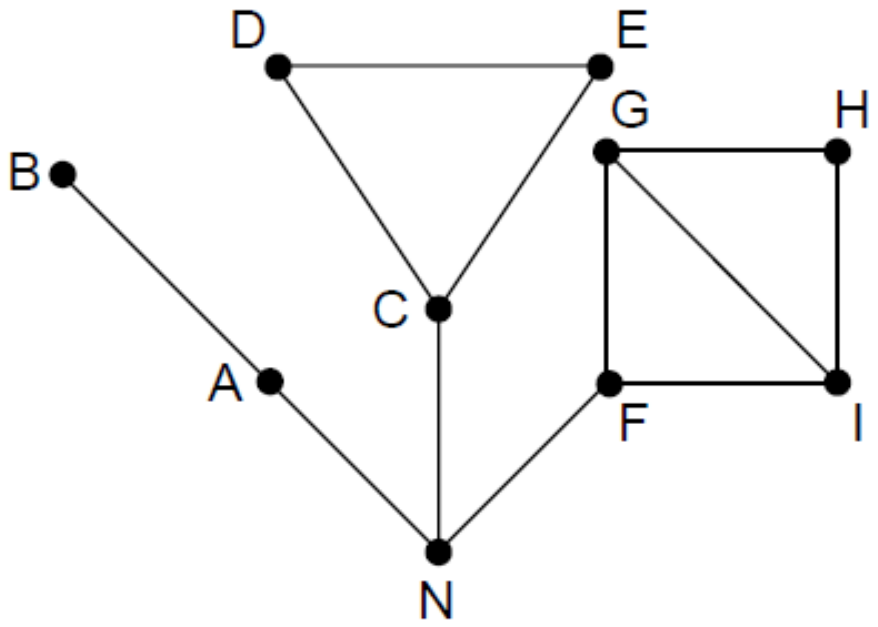


1 Discover → 2 Measure → 3 Create LSP → 4 Flood → 5 Compute Shortest Paths

Step 1: Learning about the Neighbors

Nine routers and a broadcast LAN interacting to discover adjacencies via HELLO packets.





Step 2: Measuring Line Cost

Delay as Cost

The Link State protocol requires a measure of the delay or cost to reach each neighbor. A common metric is delay.

The router sends a special ECHO packet that the other side is required to send back immediately.

Bandwidth as Cost

Alternatively, the cost can be artificially configured based on the bandwidth of the link (e.g., 1 Gbps link has lower cost than 10 Mbps link).

This encourages routing traffic over faster lines rather than purely fewer hops.

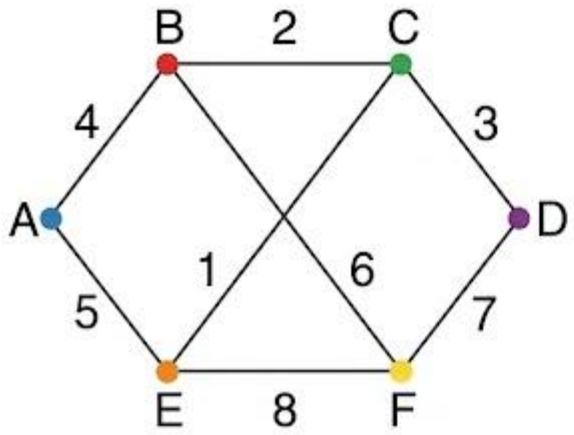
Step 3: Building Link State Packets

Once the information needed for the exchange has been collected, the next step is for each router to build a packet containing all the data.

- The packet contains the identity of the sender.
- A sequence number and age to manage packet freshness.
- A list of neighbors and the delay/cost to each neighbor.

Building Link State Packets (Graph)

(a) A network. (b) The Link State packets generated for this network.



(a)

Link State Packets

A	B	C	D	E	F
Seq.	Seq.	Seq.	Seq.	Seq.	Seq.
Age	Age	Age	Age	Age	Age
B 4	A 4	B 2	C 3	A 5	B 6
E 5	C 2	D 3	F 7	C 1	D 7
	F 6	E 1		F 8	E 8

(b)

Step 4: Distributing Link State Packets

The trickiest part of the algorithm is reliably distributing the link state packets.

The fundamental idea is to use **flooding** to distribute the packets rapidly.

To keep the flood in check, each packet contains a 32-bit sequence number that is incremented for each new packet sent.

Routers keep track of all pairs of (source router, sequence). If a duplicate packet arrives, it is discarded.

Packet buffer for Router B

			Send flags			ACK flags			
Source	Seq.	Age	A	C	F	A	C	F	Data
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

Step 5: Computing the New Routes

Global Map

Once a router has accumulated a full set of Link State packets, it has a complete graph of the entire network topology.

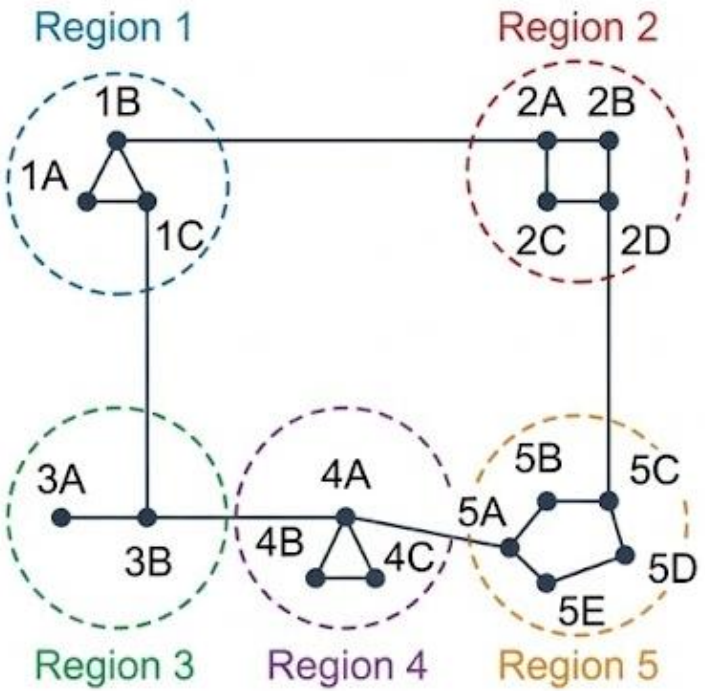
Local Dijkstra

Dijkstra's shortest path algorithm is run locally on this graph to construct the shortest path from this router to all possible destinations, filling the routing table.

Hierarchical Routing

- As networks grow in size, routing tables grow proportionally. More memory, CPU time, and bandwidth are consumed.
- To solve this, routers are divided into **regions**.
- A router knows how to route packets to destinations within its own region locally.
- But it knows nothing about the internal structure of other regions, relying on summarized gateway paths.
- Two levels may not be sufficient for huge networks
- It may be necessary to group the regions into clusters, the clusters into zones, and the zones into groups
- How many levels should the hierarchy have?
- For example, consider a network with 720 routers.
  - Without a hierarchy, each router needs 720 routing table entries.
  - If the network is partitioned into 24 regions of 30 routers each, each router needs 30 local entries plus 23 remote entries for a total of 53 entries.
  - If a three-level hierarchy is chosen, with 8 clusters each containing 9 regions of 10 routers, each router needs 10 entries for local routers, 8 entries for routing to other regions within its own cluster, and 7 entries for distant clusters, for a total of 25 entries.

Hierarchical Routing



(a)

Full table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

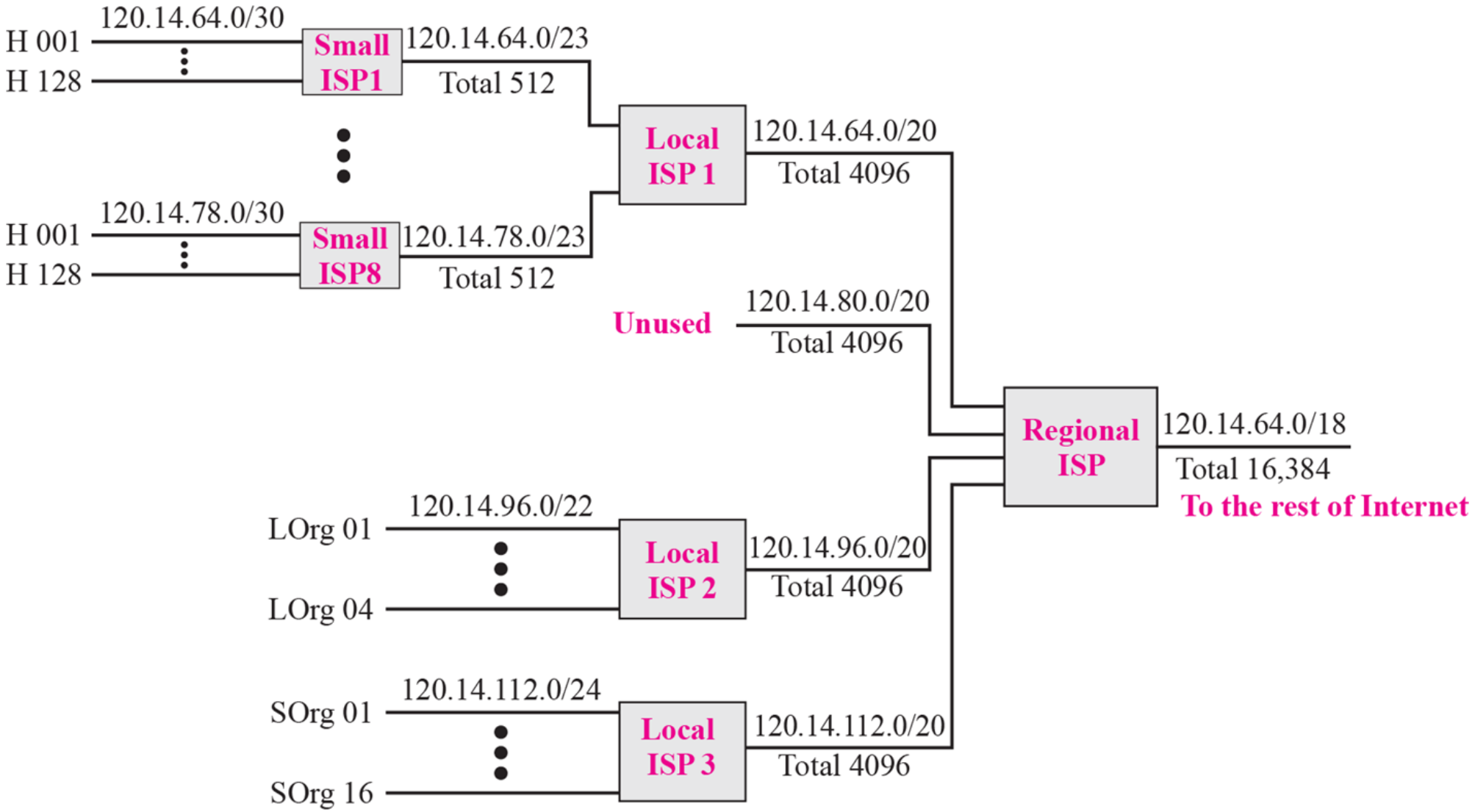
Hierarchical table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

(c)

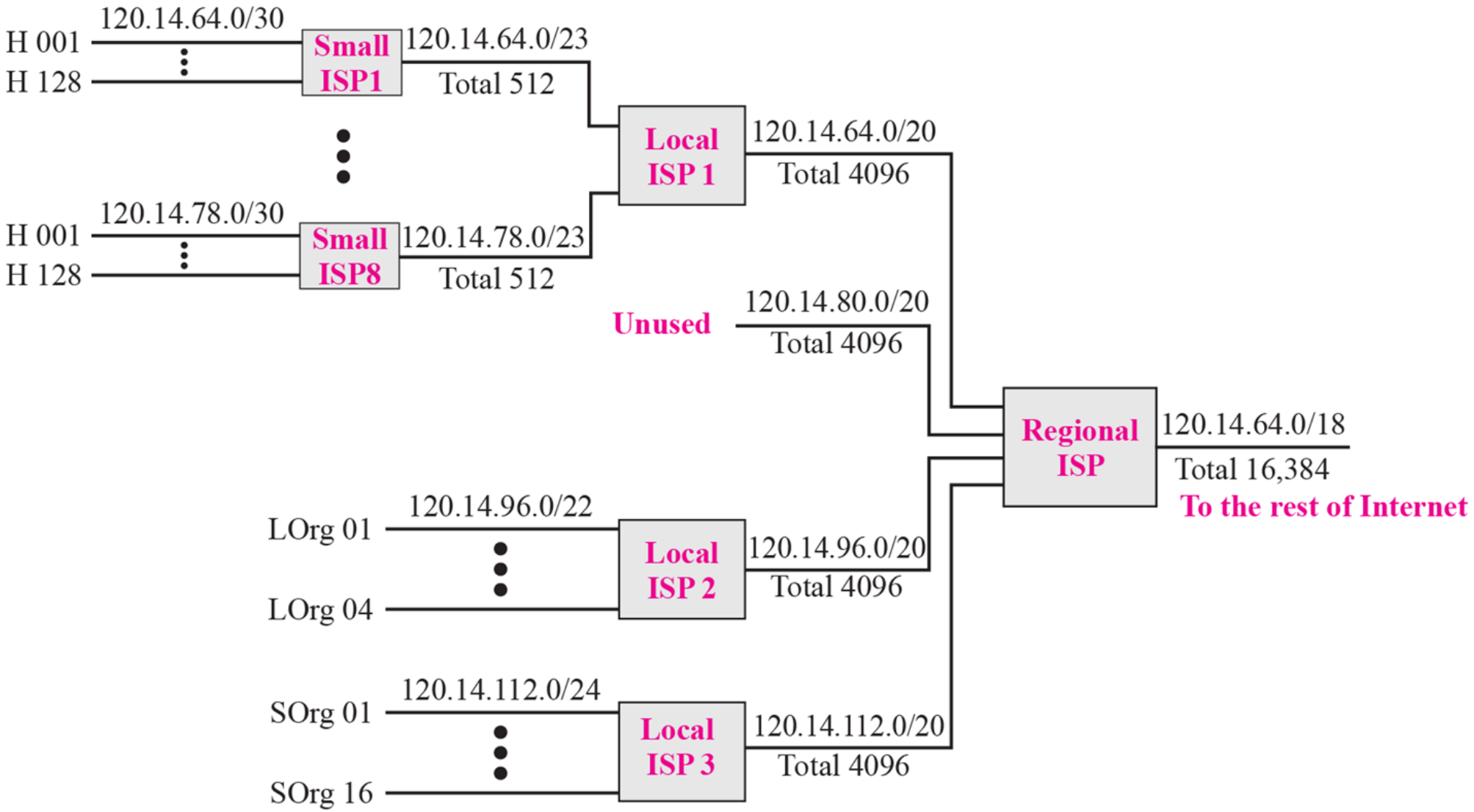
Hierarchical Routing with ISPs

A regional ISP is granted 16,384 addresses starting from 120.14.64.0. The regional ISP has decided to divide this block into 4 subblocks, each with 4096 addresses.



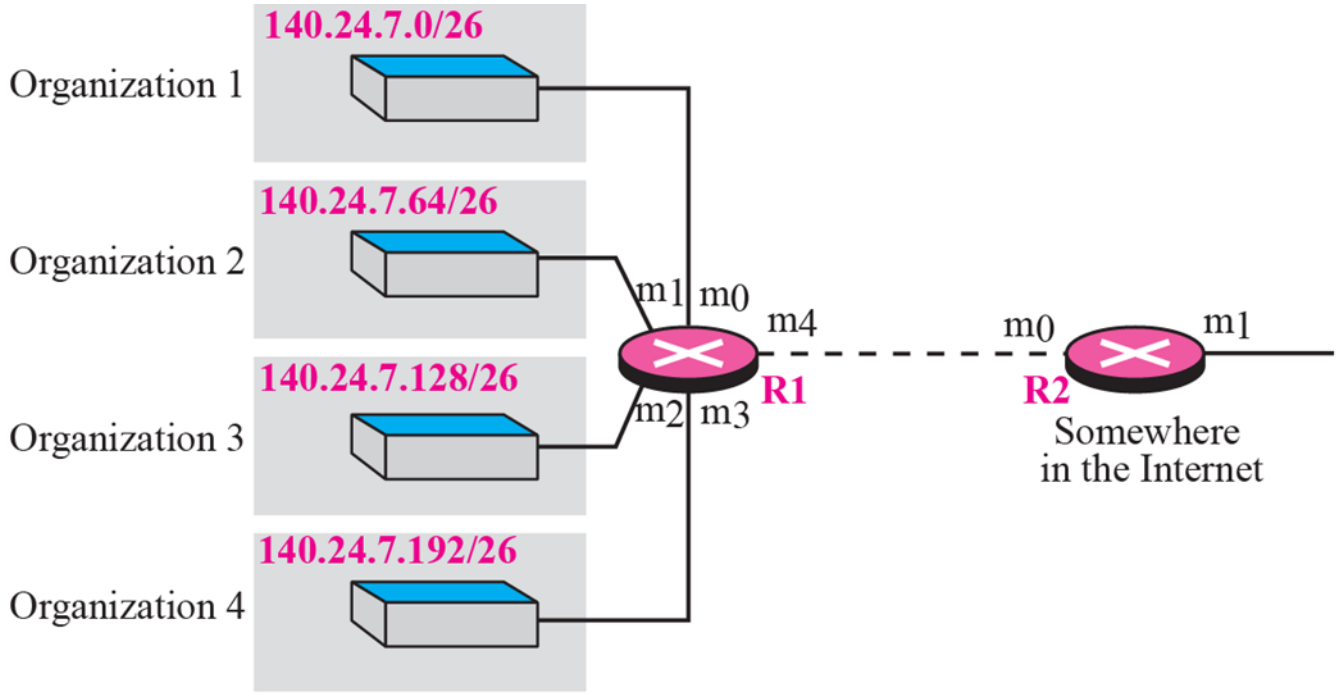
Hierarchical Routing with ISPs

A regional ISP is granted 16,384 addresses starting from 120.14.64.0. The regional ISP has decided to divide this block into 4 subblocks, each with 4096 addresses.



Hierarchical Routing with IPv4

Route Aggregation/Route Summarization



Mask	Network address	Next-hop address	Interface
/26	140.24.7.0	----------	m0
/26	140.24.7.64	----------	m1
/26	140.24.7.128	----------	m2
/26	140.24.7.192	----------	m3
/0	0.0.0.0	default router	m4

Routing table for R1

Mask	Network address	Next-hop address	Interface
/24	140.24.7.0	----------	m0
/0	0.0.0.0	default router	m1

Routing table for R2

Broadcast Routing Approaches

Sending a packet to all destinations simultaneously.

Multidestination Routing

Each packet contains a list of destinations. The router checks the destinations and generates a new copy for each required output line, saving bandwidth on shared segments.

Flooding

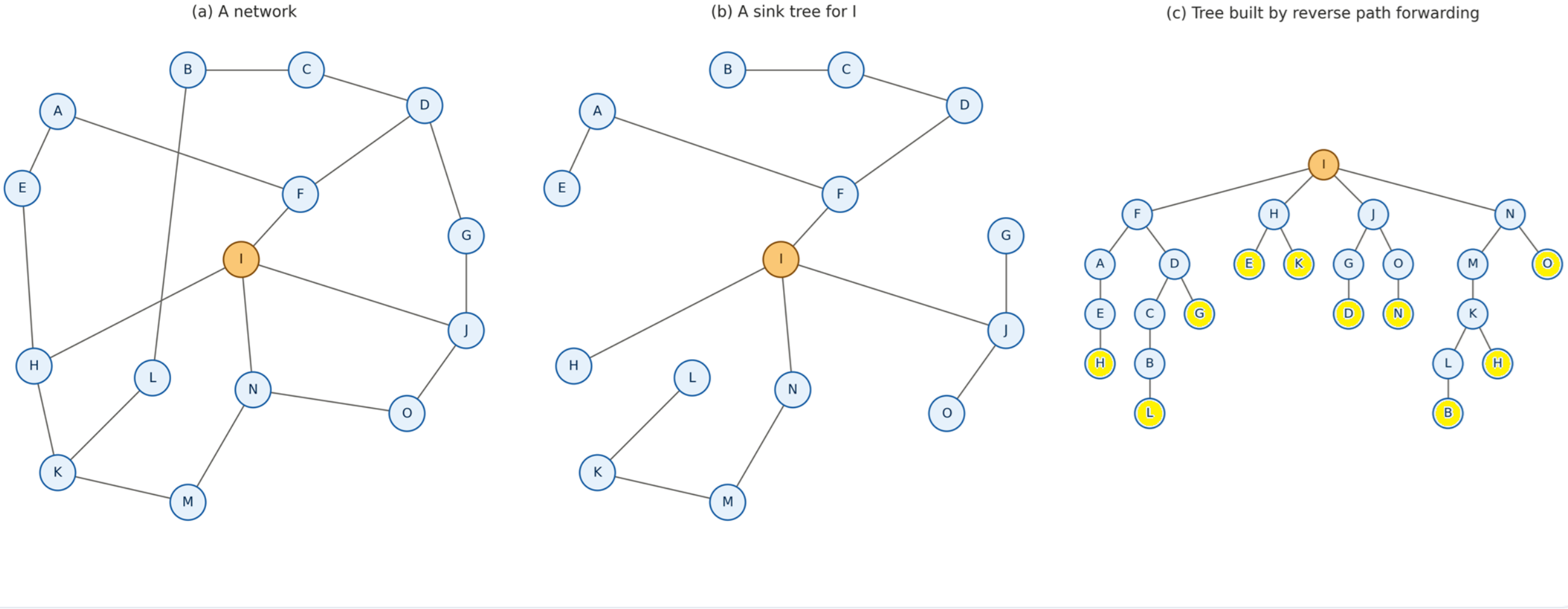
Every incoming packet is sent out on every outgoing line except the one it arrived on. Simple but highly inefficient without mechanisms to stop loops.

Reverse Path Forwarding

An elegant and highly efficient algorithm for broadcast routing.

- When a broadcast packet arrives, the router checks to see if the packet arrived on the link that is normally used for sending packets *toward* the source.
- If so, it forwards copies onto all other links, assuming it took the best path.
- If it arrived on a different link, it is discarded as a likely duplicate.

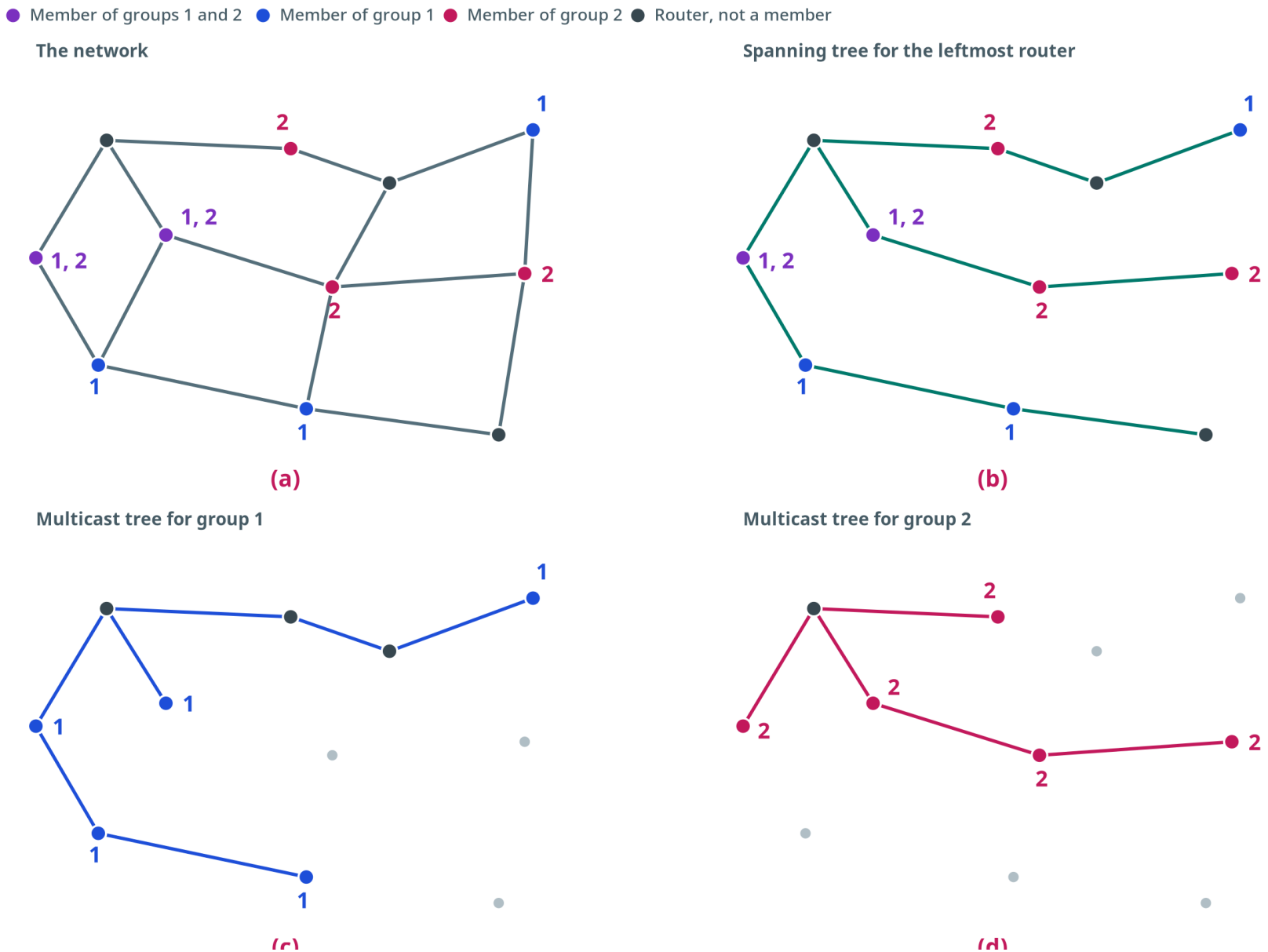
Reverse Path Forwarding (Graph)



Multicast Routing

- Sending to a specific group of nodes rather than all nodes (Broadcast) or one node (Unicast).
- Multicasting requires creating and destroying groups, and identifying which routers belong to which group.
- Assume that each group is identified by a multicast address and that routers know the groups to which they belong
- If the group is dense, broadcast is a good start, but it reaches all routers. The solution is pruning the broadcast tree by removing links that do not lead to members.
- If OSPF is used, each router knows the complete topology; it can construct its own pruned spanning tree for each sender in the group. E.g., MOSPF (Multicast OSPF)
- Alternatively, routers can build a Core-Based Tree in which all members send packets to a central rendezvous point, forming a shared distribution tree.

Multicast Routing



(a) A network.

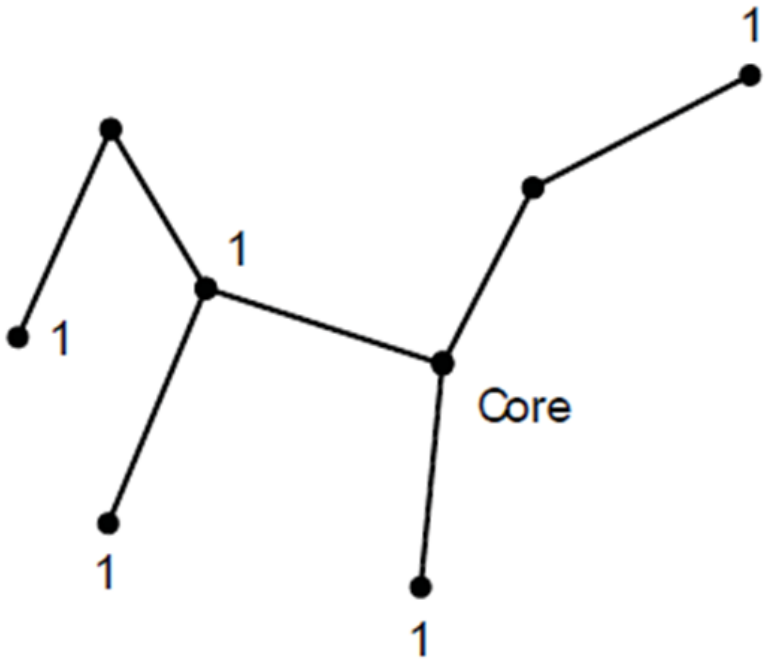
(b) A spanning tree for the leftmost router.

(c) A multicast tree for group 1.

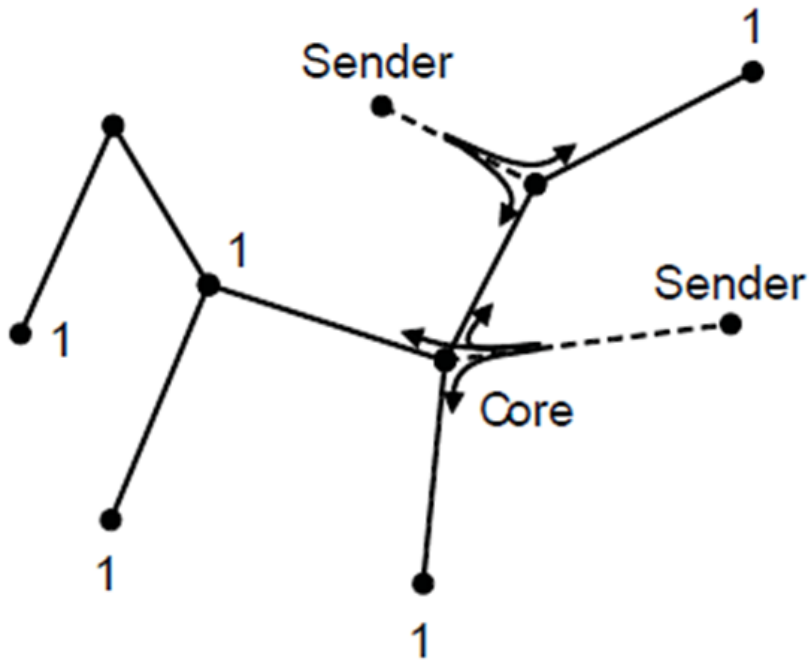
(d) A multicast tree for group 2.

Multicast Routing

- All of the routers agree on a root (called the core or rendezvous point)
- Build the tree by sending a packet from each member to the root
- The tree is the union of the paths traced by these packets
- To send to this group, a sender sends a packet to the core



a) Core-based tree for group 1.



b) Sending to group 1.

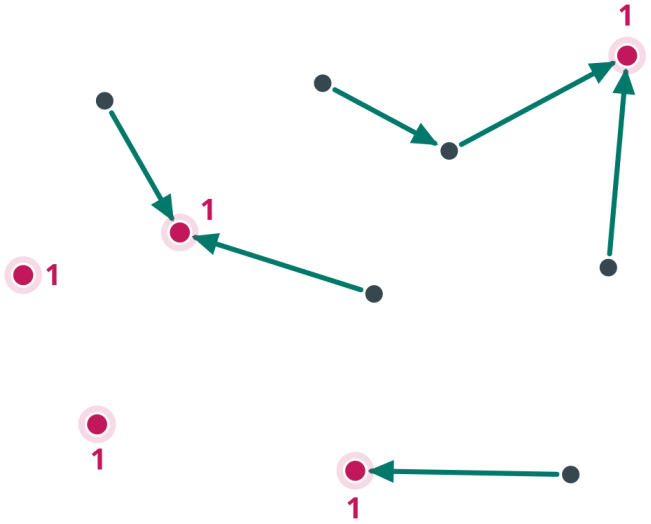
Multicast Routing in IP network

- An IP multicast packet is routed through a router using a process called Reverse Path Forwarding (RPF) combined with multicast routing protocols
- Core Steps in Multicast Routing
  - The RPF Check: The router checks its regular unicast routing table to see if the interface it received the packet on matches the best path back to the packet's source
    - If it matches, the packet passes the RPF check (ensuring a loop-free path), and the router proceeds.
    - If it does not match, the packet fails and is dropped
  - Checking the Multicast Table (mroute): The router consults its multicast forwarding cache using the source address and group address (IGMP helps in preparing and maintaining this cache)
  - Forwarding and Replication: If approved, the router replicates the packet and forwards a copy out of every interface listed in its Outgoing Interface List (OIL)

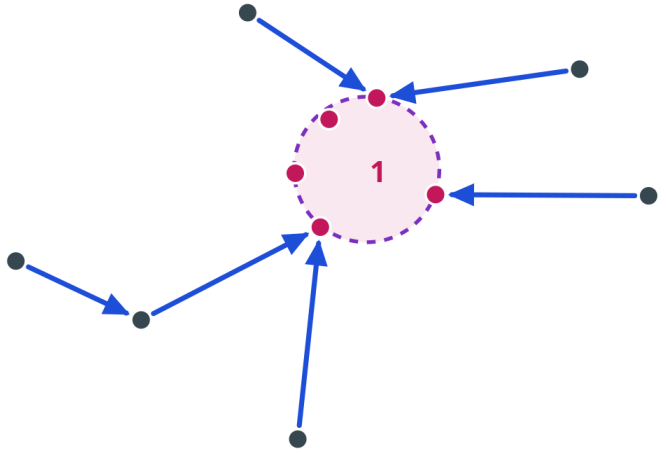
Anycast Routing

- A packet is delivered to the nearest member of a group
- Can be achieved using regular DV or LSR
- All members of the group will be given the same address, say 1
- This procedure works because the routing protocol does not realize that there are multiple instances of destination 1.

  That is, it believes that all the instances of node 1 are the same node



a) Anycast routes to group 1.



b) Topology seen by the routing protocol.

Congestion Control Algorithms (1)

Approaches to Congestion Control

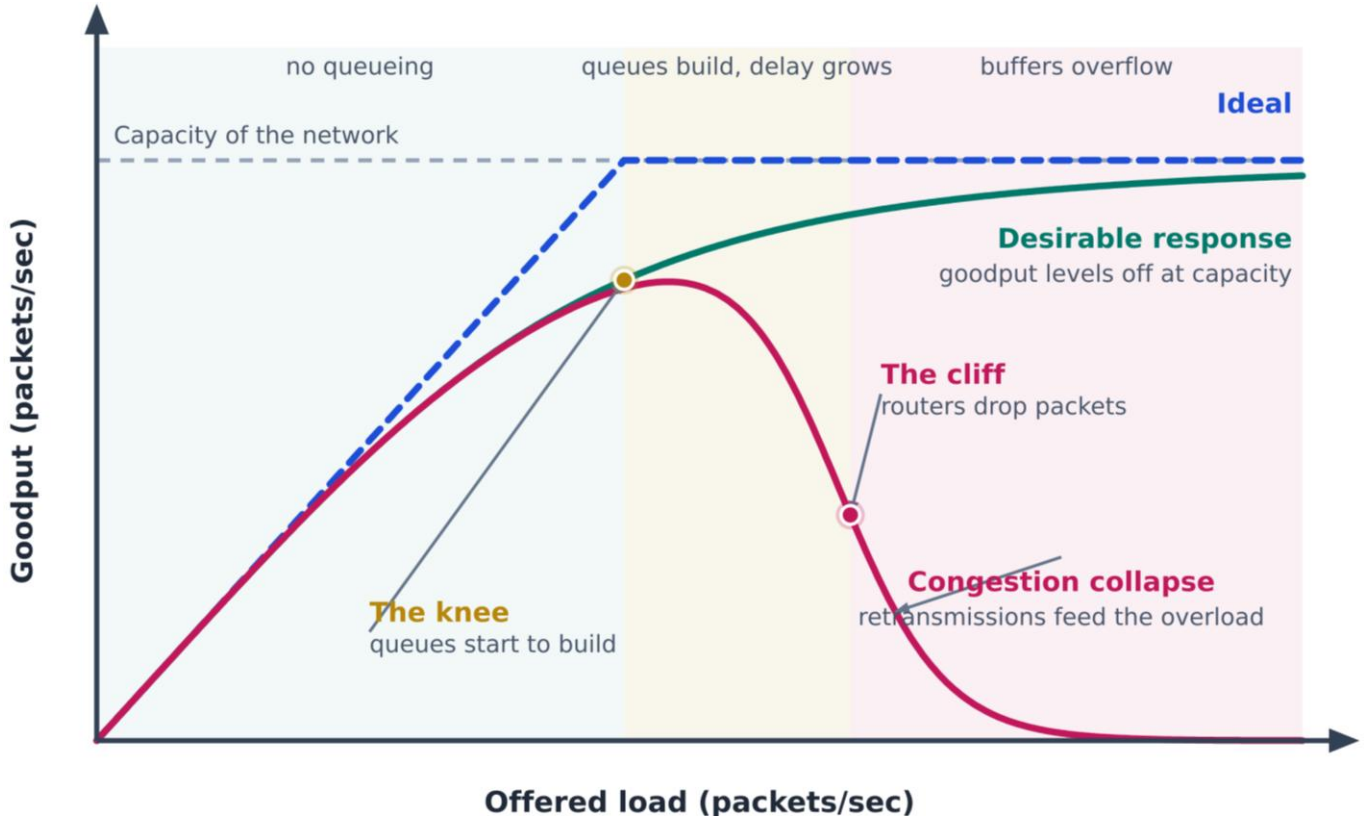
- **Traffic-aware routing:** Shifting traffic away from hotspots.
- **Admission control:** Refusing new connections if the network is overloaded.
- **Traffic throttling:** Forcing senders to slow down their transmission rates.
- **Load shedding:** Dropping packets when the router's buffers are full.

> **Perspective:** Think of congestion control as managing highway traffic. You can build detours (routing), close on-ramps (admission), lower the speed limit (throttling), or—in the worst case—start forcing cars off the road (load shedding).

Congestion Control Algorithms (2)

When too much traffic is offered, congestion sets in and performance degrades sharply.

The Knee vs. The Cliff:
The "Knee" is where queues start building, increasing delay. The "Cliff" is where router buffers overflow, causing packet drops. This triggers a destructive feedback loop of retransmissions, plunging goodput to near zero (Congestion Collapse).



Approaches to Congestion Control

Timescales of approaches to congestion control:

Slow / Preventive

Network Provisioning &
Traffic-Aware Routing

Medium / Setup

Admission Control
(during VC setup)

Fast / Reactive

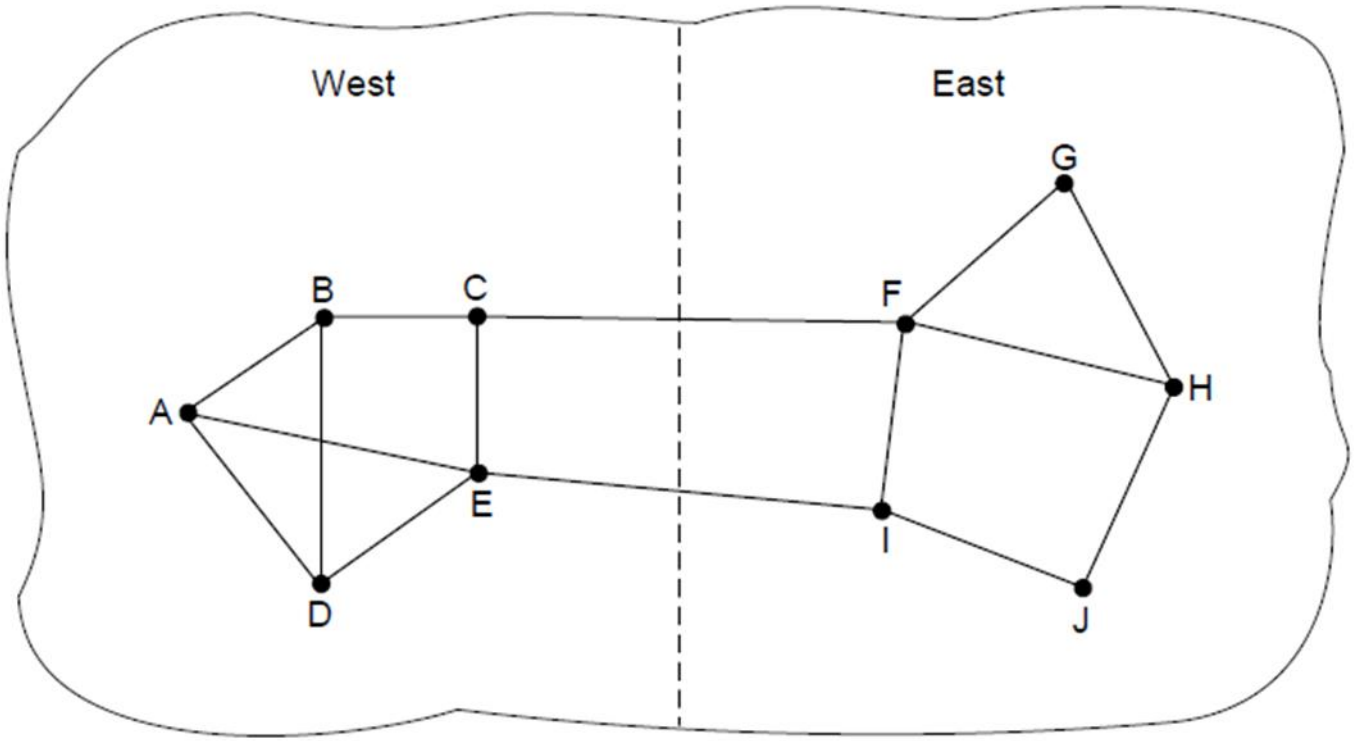
Traffic Throttling &
Load Shedding

Traffic-Aware Routing (1)

- The most direct way to route around congestion is to set the link weight to be a function of the (fixed) link bandwidth plus the (variable) measured load.
- **Risk of Oscillation:** If everyone routes away from a congested link simultaneously, the alternate route becomes congested, causing traffic to swing wildly back and forth.
- **Solution:** Multipath routing (like ECMP) and shifting traffic slowly allows the routing scheme to converge stably.

Traffic-Aware Routing (2)

A network in which the East and West parts are connected by two links.



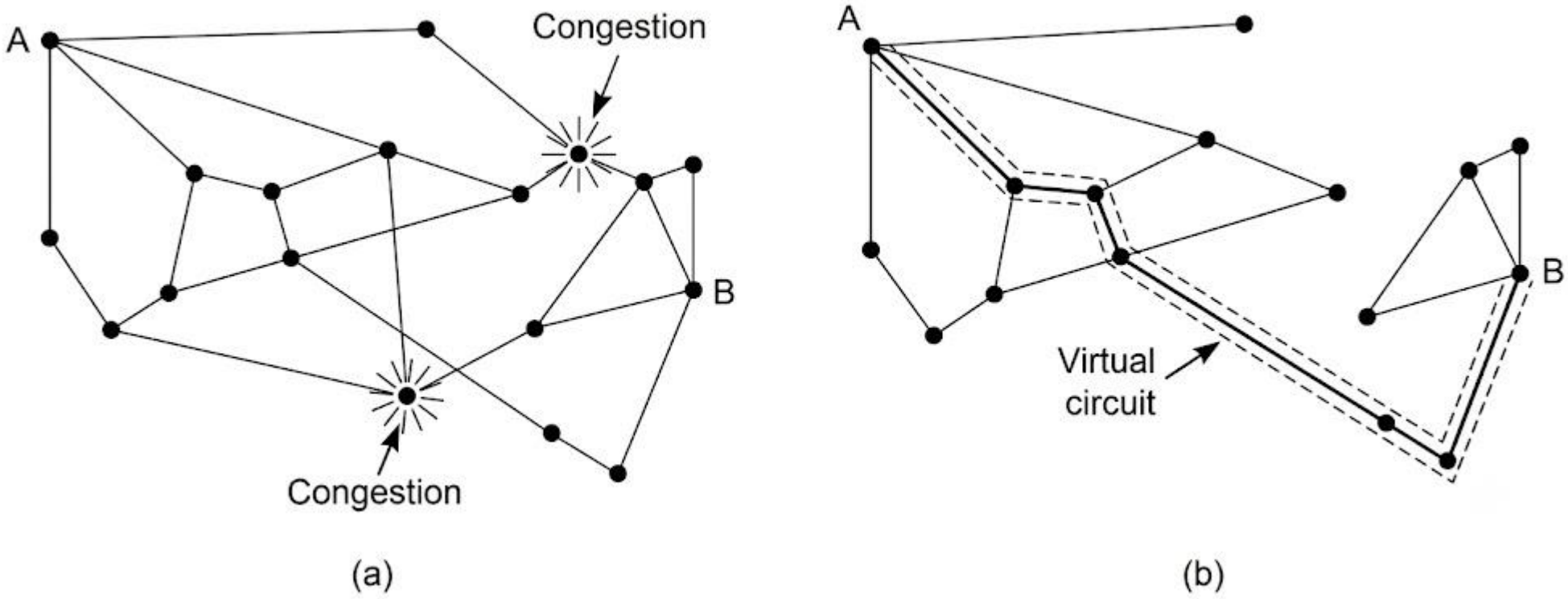
Admission Control (1)

The idea: do not set up a new virtual circuit unless the network can carry the added traffic without becoming congested.

- Traffic is described in terms of its rate and shape (e.g., Leaky Bucket or Token Bucket descriptors).
- Armed with traffic descriptions, the network can decide whether to admit the new virtual circuit.
- Admission control can also be combined with traffic-aware routing by routing *around* traffic hotspots during the setup procedure.

Admission Control (2)

(a) A congested network. (b) The portion of the network that is not congested. A virtual circuit from A to B is routed strictly through the uncongested zone.



Traffic Throttling (1)

- The network aims to operate just before the onset of congestion.
- When congestion is imminent, it must tell the senders to throttle back their transmissions and slow down.
- Routers determine approaching congestion by monitoring output link utilization, queued packet buffering, and dropped packets.

> **Proactive vs Reactive:** Traffic throttling is a *Congestion Avoidance* mechanism. It tries to pull the network back from the brink of the "Cliff" before severe packet loss occurs.

Traffic Throttling (2): EWMA

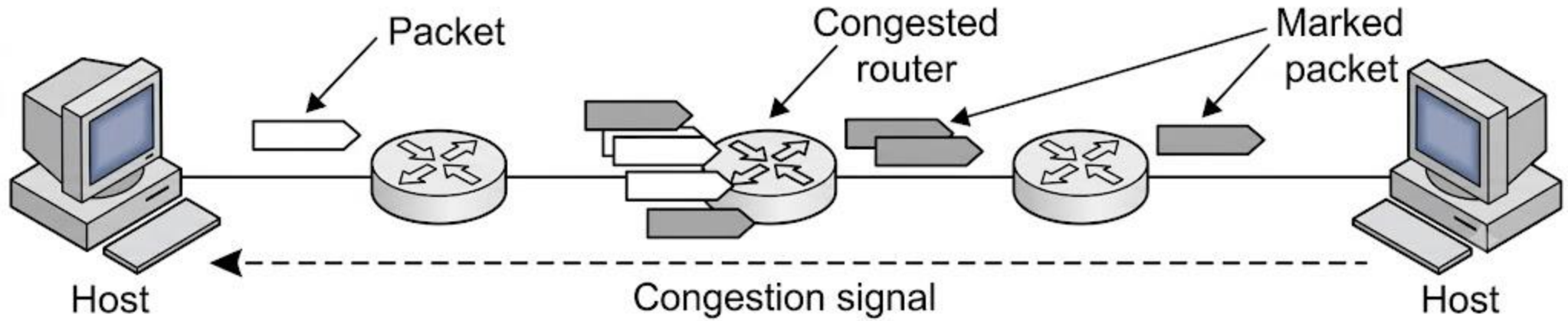
Routers use an Exponentially Weighted Moving Average (EWMA) to smooth out transient bursts and calculate true congestion:

$$d_{new} = \alpha.d_{old} + (1 - \alpha).S$$

- **d** is the estimated delay or queue length.
- S is the instantaneous sample measurement.
- **α** determines how fast the router forgets history (typically a value like 0.875).

Explicit Congestion Notification (ECN)

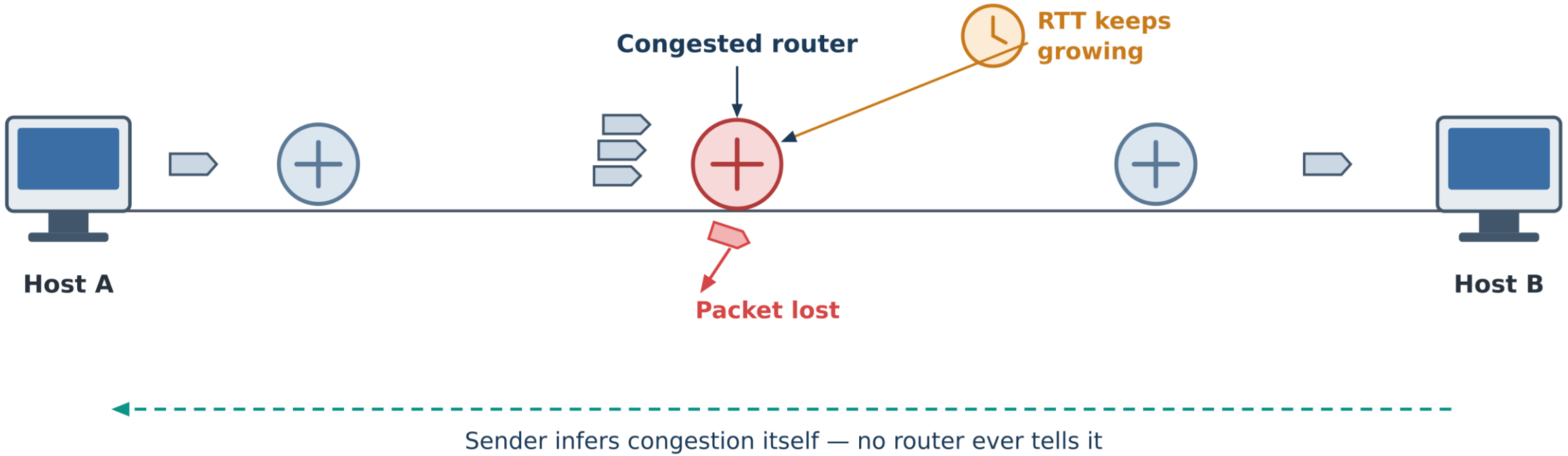
Instead of dropping packets, a congested router can mark the packet's IP header.



How it works: The receiver sees the ECN mark and echoes it back to the sender in the TCP ACK. The sender then reduces its transmission window exactly as if a packet had been lost, but without the penalty of actual data loss.

Implicit Congestion Notification

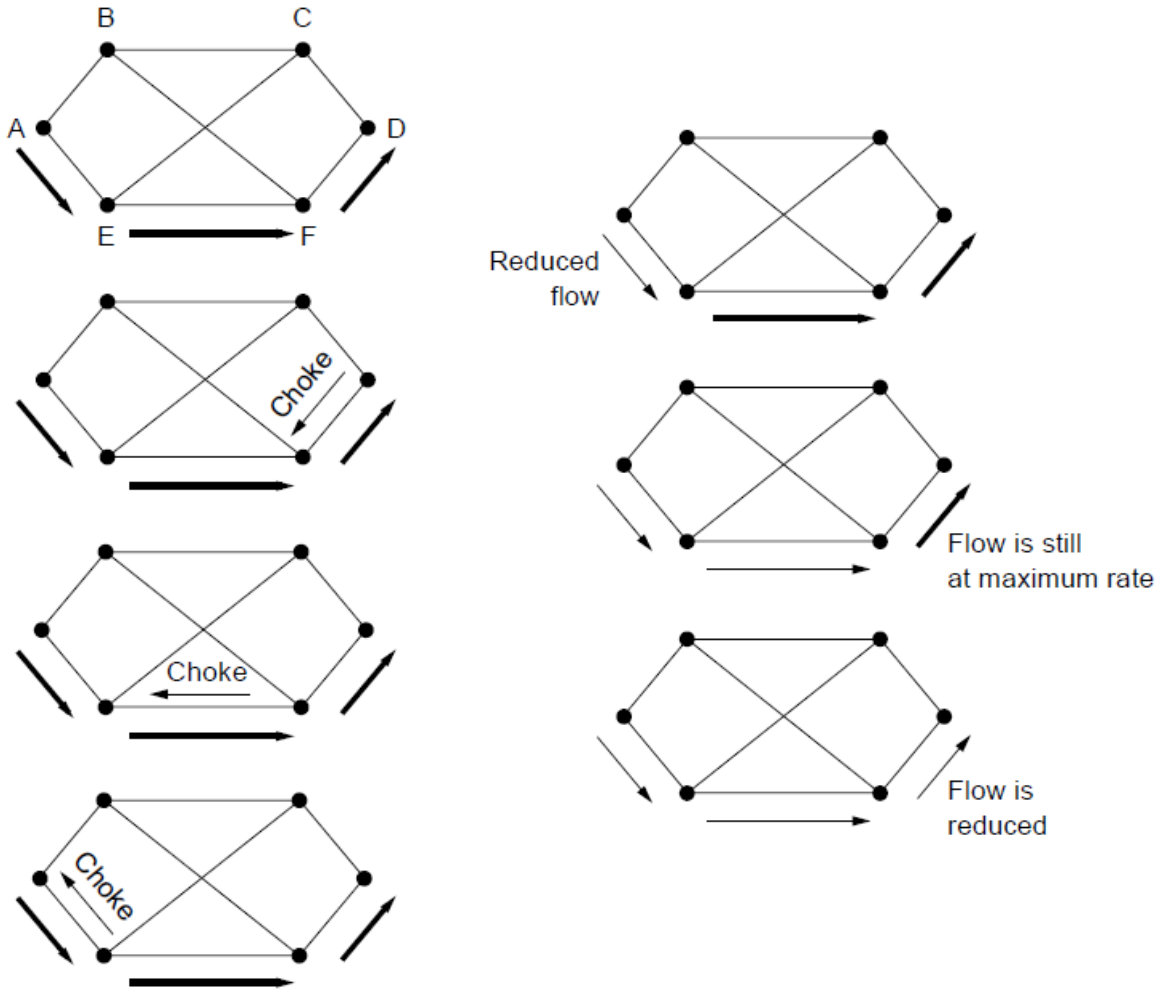
Instead of a router marking anything, the source infers congestion from what it observes end-to-end



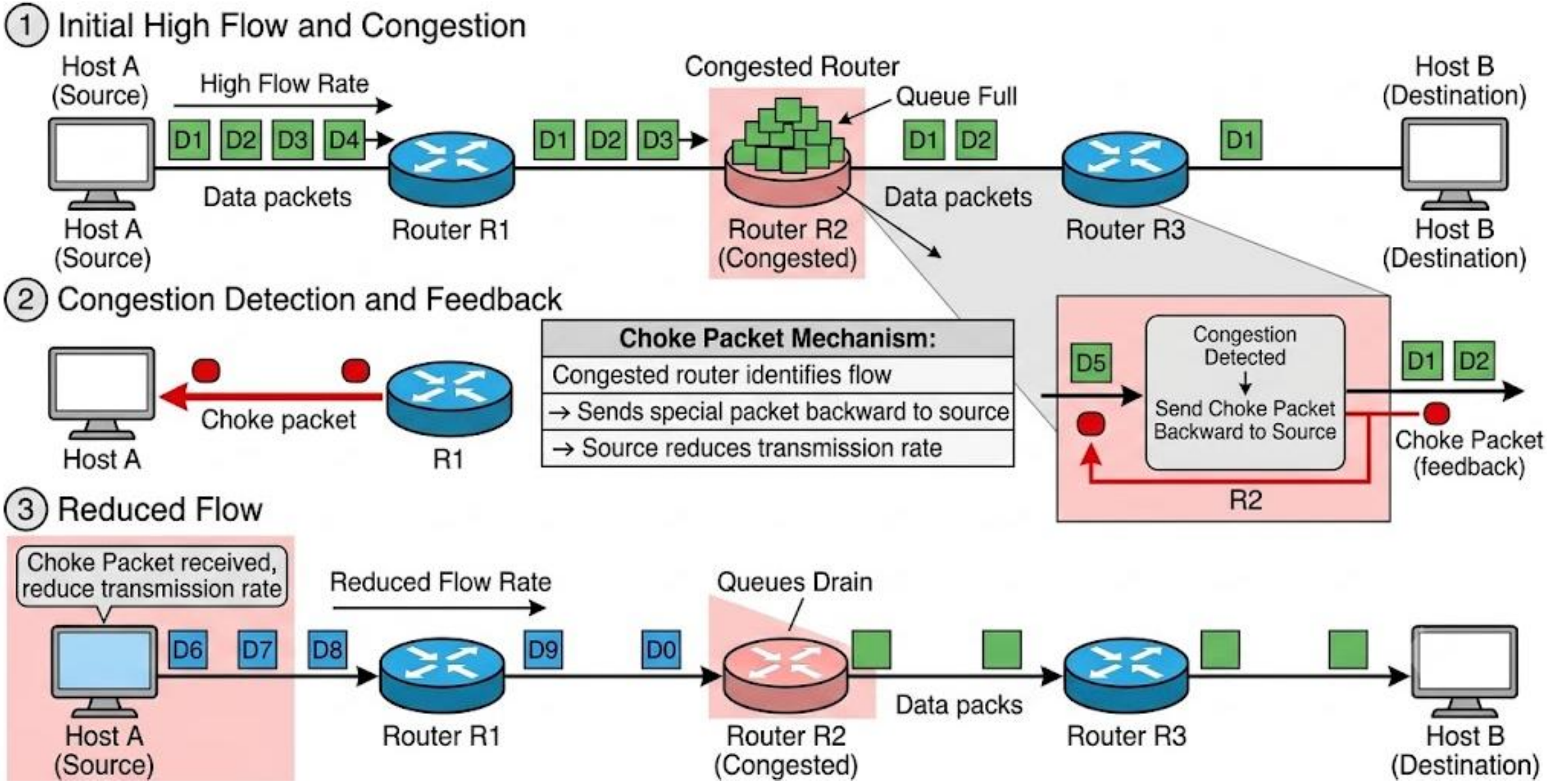
How it works: TCP infers congestion by observing timeouts or three duplicate acknowledgments. ACKs serve as evidence of congestion, and the sender cuts its congestion window accordingly – the same reaction as ECN but triggered by the sender's own measurements instead of being marked explicitly by a router.

Choke Packets (Source Only)

- A choke packet that affects only the source.
  - The router detects congestion.
  - Generates a warning packet sent directly back to the origin host.
  - The host is required to cut its transmission rate.



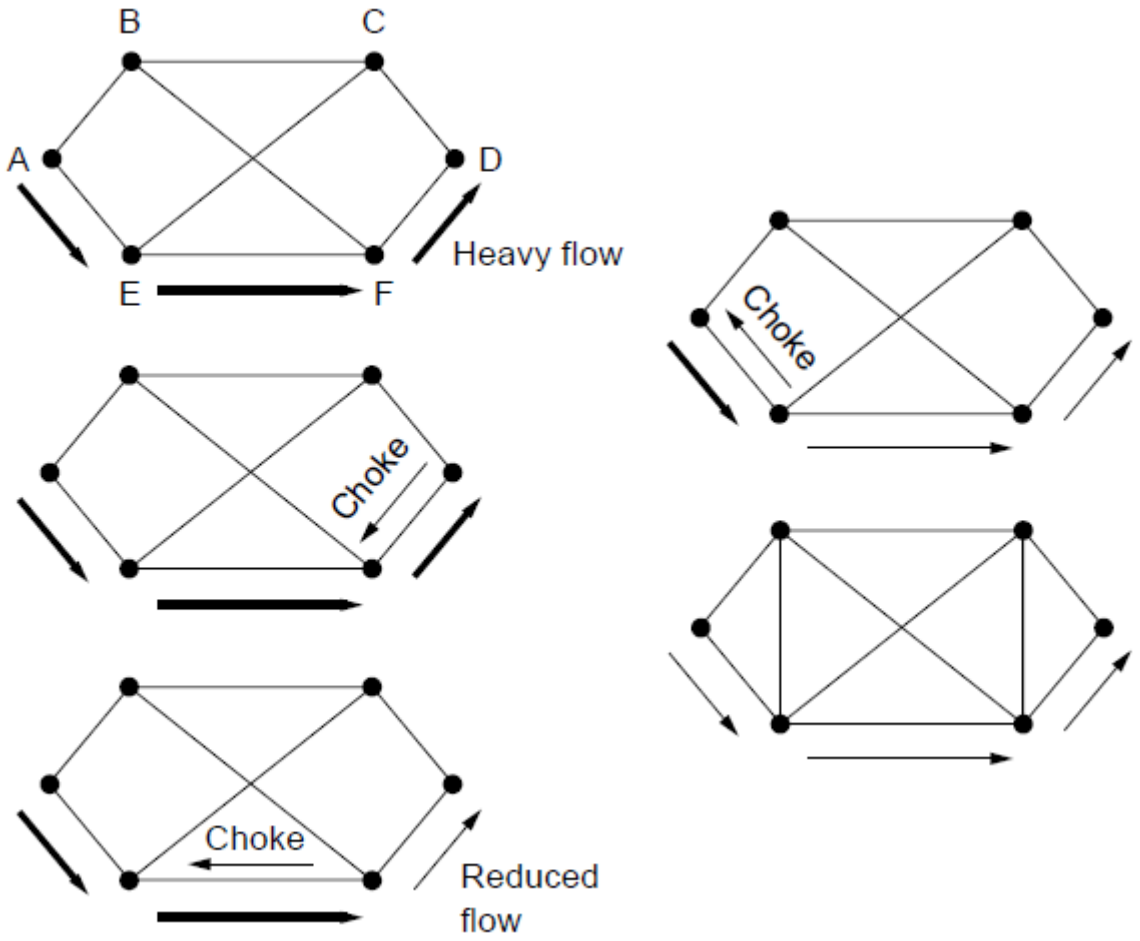
Choke Packets (Source Only)



Hop-by-Hop Choke Packets

A choke packet that affects each hop it passes through.

> **Faster Relief:** Standard choke packets take an entire round-trip time to take effect. Hop-by-hop forces immediate upstream routers to buffer data and slow down instantly, providing the congested router rapid relief.



Load Shedding

When all else fails, routers bring out the heavy artillery: **Throw packets away.**

Wine Strategy

Drop **new** packets. Keep the old ones. Good for file transfers where losing an old packet forces a massive retransmission.

Milk Strategy

Drop **old** packets. Keep the fresh ones. Good for multimedia/video where old frames are completely useless anyway.

Random Early Detection (RED)

- Dealing with congestion early is better than letting queues fill up entirely (Tail Drop).
- Routers maintain a running average of their queue lengths.
- When the average exceeds a threshold, a small fraction of packets are dropped **at random**.

> **Why Random?** Picking packets at random makes it mathematically more likely that the fastest, most aggressive senders will suffer the drop. This naturally forces the bandwidth hogs to trigger TCP backoff.

Quality of Service (QoS)

Techniques to guarantee performance levels for different types of network traffic:

- Application Requirements
- Traffic Shaping
- Packet Scheduling
- Admission Control
- Integrated Services (IntServ)
- Differentiated Services (DiffServ)

Application Requirements

Application	Bandwidth	Delay	Jitter	Loss
Email	Low	Low	Low	Medium
File sharing	High	Low	Low	Medium
Web access	Medium	Medium	Low	Medium
Remote login	Low	Medium	Medium	Medium
Audio on demand	Low	Low	High	Low
Video on demand	High	Low	High	Low
Telephony / VoIP	Low	High	High	Low
Videoconferencing	High	High	High	Low

Categories of Traffic

Constant Bit Rate (CBR)

Strict, unvarying bandwidth.

e.g., Uncompressed Telephony

Variable Bit Rate (VBR)

- Real-time VBR
  - Compressed videoconferencing
- Non-real-time VBR
  - Watching a movie on demand

Fluctuates based on content complexity.

e.g., Compressed Video/Netflix

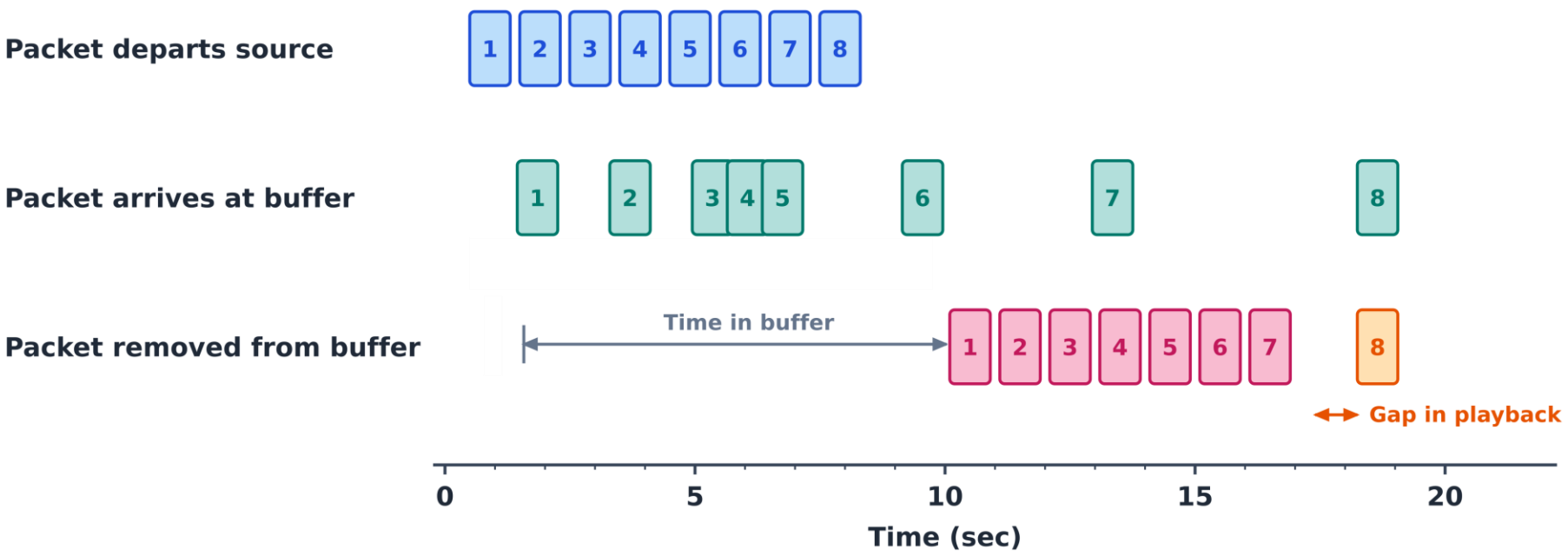
Available Bit Rate (ABR)

Takes whatever bandwidth is left over.

e.g., Background File Transfers

Buffering for Jitter Control

Smoothing the output stream by buffering packets.



Jitter vs Delay: Jitter is the *variation* in delay. Video streaming clients intentionally delay playback by a few seconds (filling a buffer) so that if a packet arrives slightly late due to jitter, the video doesn't freeze.

Buffering for Jitter Control

Small buffer

- Low playback delay - receiver starts playing back soon after the first packets arrive
- Poor jitter tolerance - if a packet is delayed more than the buffer's holding time, the buffer runs dry before the packet arrives, causing a freeze in playback.
- More sensitive to network conditions - any spike in jitter shows up immediately as a playback glitch.

Large buffer

- Smooths out jitter effectively - even packets that arrive quite late (within the buffer window) get ordered into steady, gap-free playback.
- High playback delay - the receiver must wait longer before starting playback, and the whole stream lags further behind real time. This is fine for on-demand video (Netflix, YouTube) but unacceptable for live, interactive traffic.
- Uses more memory at the receiver, and if the source is far ahead of playback, a sudden pause (e.g., user pauses) means a lot of buffered data.

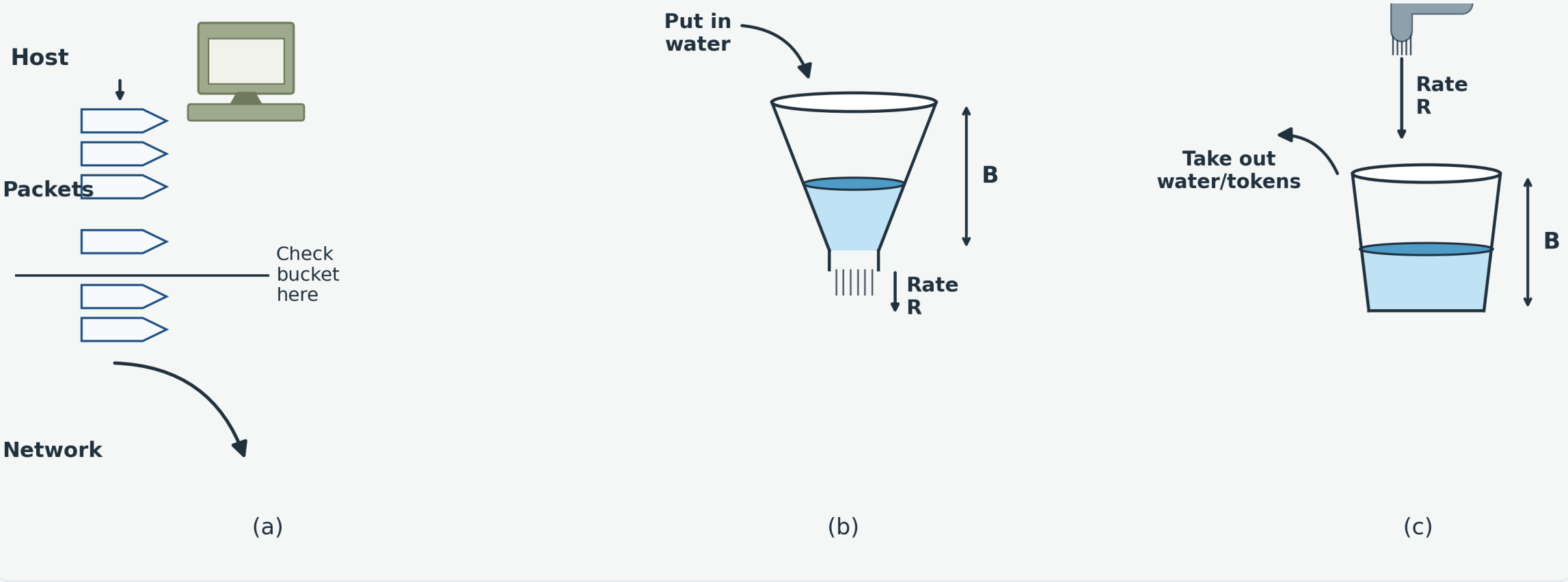
In practice, the buffer size is chosen based on the application:
Live/interactive : small buffer, tolerate occasional glitches, prioritize low latency.
Streaming/on-demand video: larger buffer, tolerate a few seconds of startup delay to guarantee smooth playback.

Traffic Shaping (1)

- Before the network can make QoS guarantees, users must agree to a **Service Level Agreement (SLA)** defining their traffic shape.
- Traffic shaping is a technique for regulating the average rate and burstiness of a flow of data that enters the network.
- Monitoring a traffic flow to ensure it obeys the SLA is called **Traffic Policing**.

Traffic Shaping (2): The Mechanisms

(a) Shaping packets. (b) A leaky bucket. (c) A token bucket.

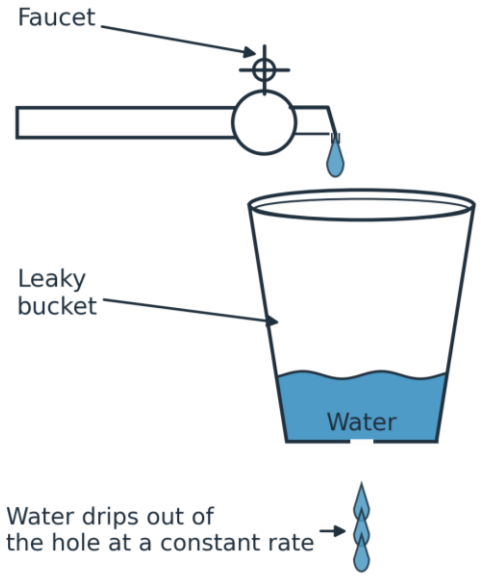


The Leaky Bucket Algorithm

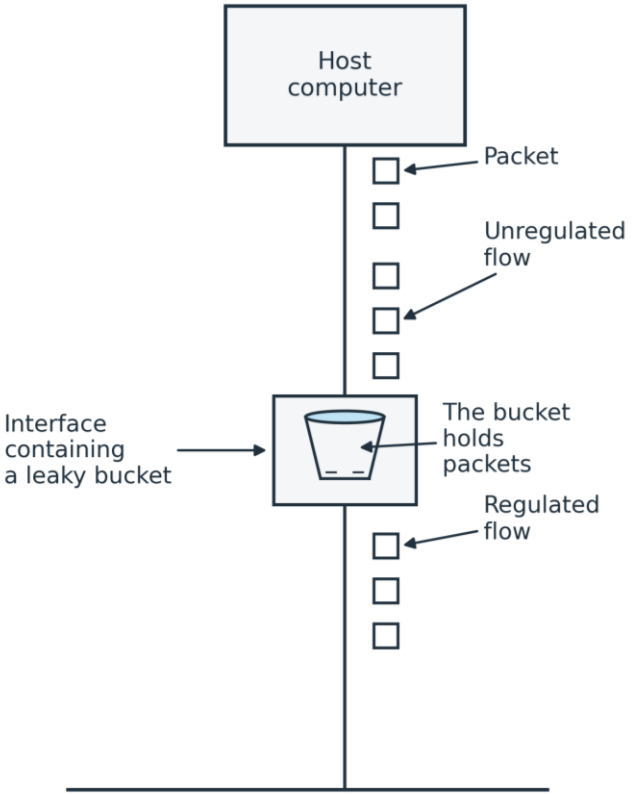
The leaky bucket enforces a rigid, constant output rate regardless of how bursty the input is.

If water (packets) pours in faster than the hole leaks, it buffers in the bucket.

If the bucket fills up, it discards excess packets.



(a)



(b)

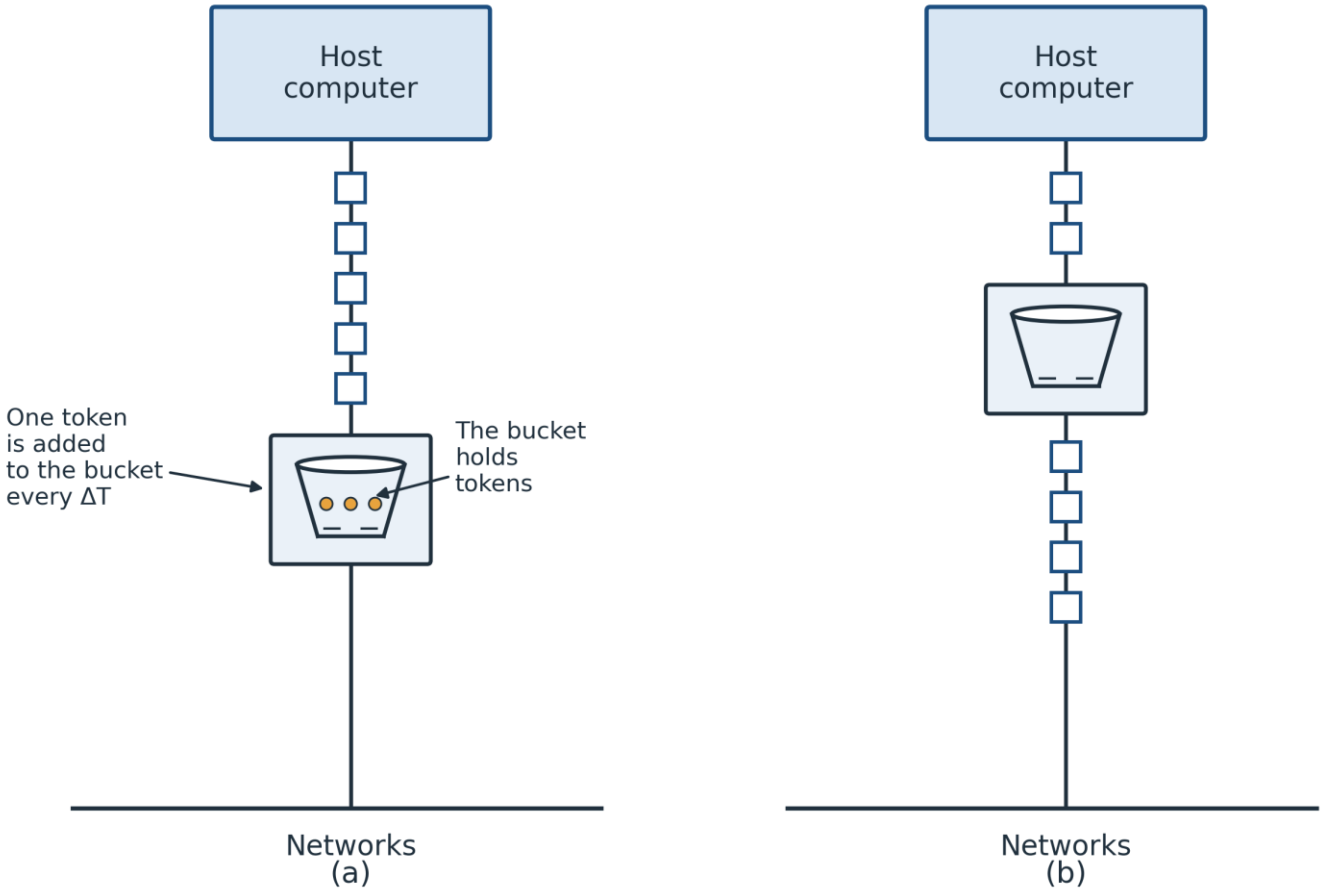
The Token Bucket Algorithm

Allows saving up "credits" to send sudden bursts of data.

Tokens are generated at a constant rate and stored in a bucket.

To transmit a packet, the host must capture and destroy a token.

Allows for bursty transmission up to the maximum size of the bucket.



Calculating Maximum Burst

How long can a Token Bucket transmit at maximum line speed?

$$B + RS = MS \qquad \text{OR} \qquad S = \frac{B}{(M - R)}$$

S = Maximum Burst Length (seconds)

B = Token Bucket Capacity (bytes)

R = Token Arrival Rate (bytes/sec)

M = Maximum Output Line Rate (bytes/sec)

The Leaky Bucket Example

(a) Input to a leaky bucket.

(b) Output from a leaky bucket.

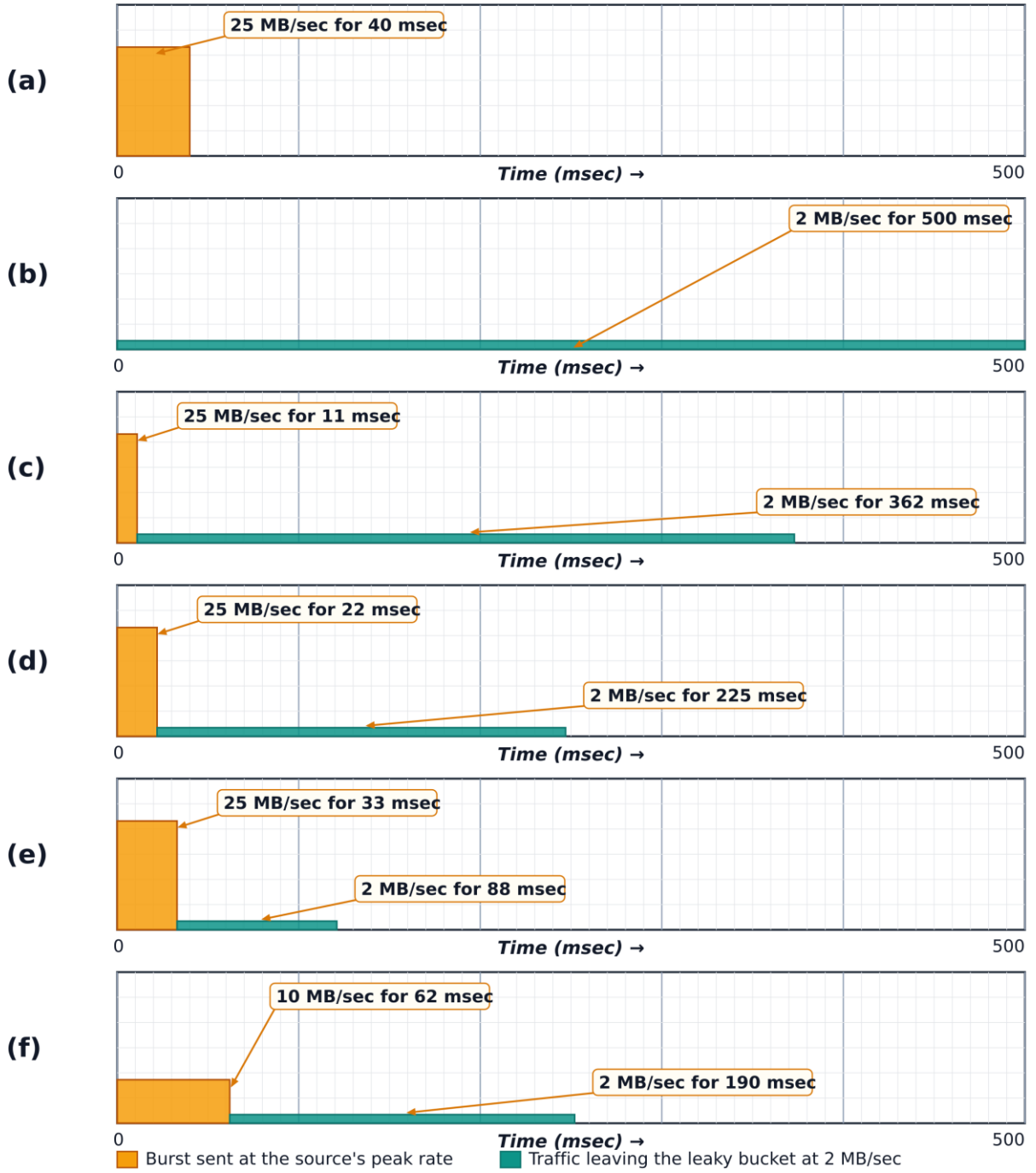
Output from a token bucket with capacities of

c) 250 KB,

d) 500 KB,

e) 750 KB,

f) Output from a 500KB token bucket feeding a 10-MB/sec leaky bucket.



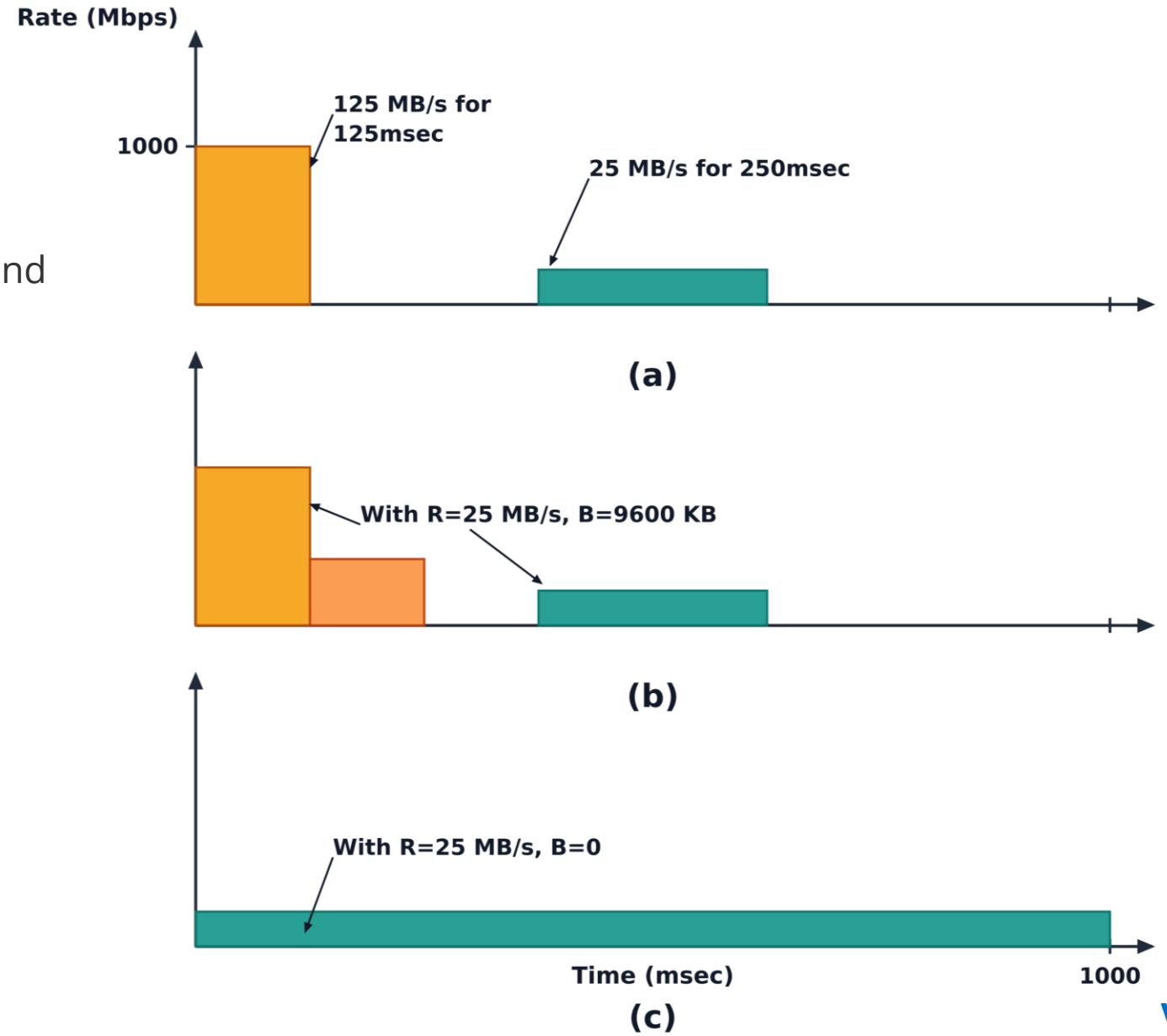
The Token Bucket Example

(a) Traffic from a host.

Output shaped by a token bucket of rate 25 MBps and capacity

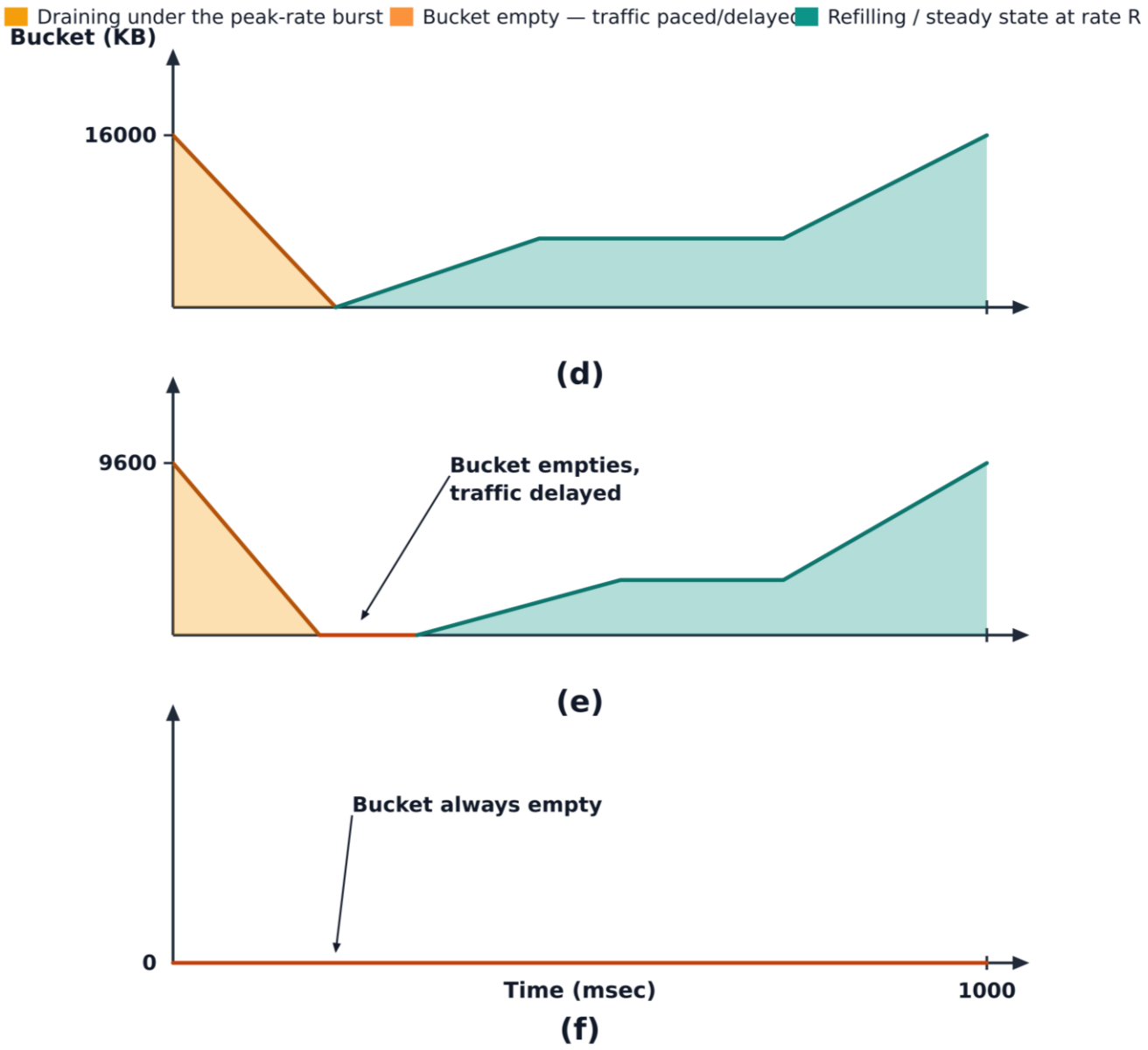
b) 9600 KB,

c) 0 KB.



The Token Bucket Example

Token bucket level for shaping with rate 200 Mbps and capacity
(d) 16000 KB,
(e) 9600 KB, and
(f) 0KB



V.U.

Packet Scheduling (1)

When multiple flows converge on a router, the router must decide which packets to send next. It must allocate three specific resources:

Bandwidth

Which flow gets priority on the outgoing link.

Buffer Space

Which packets are stored in memory vs dropped.

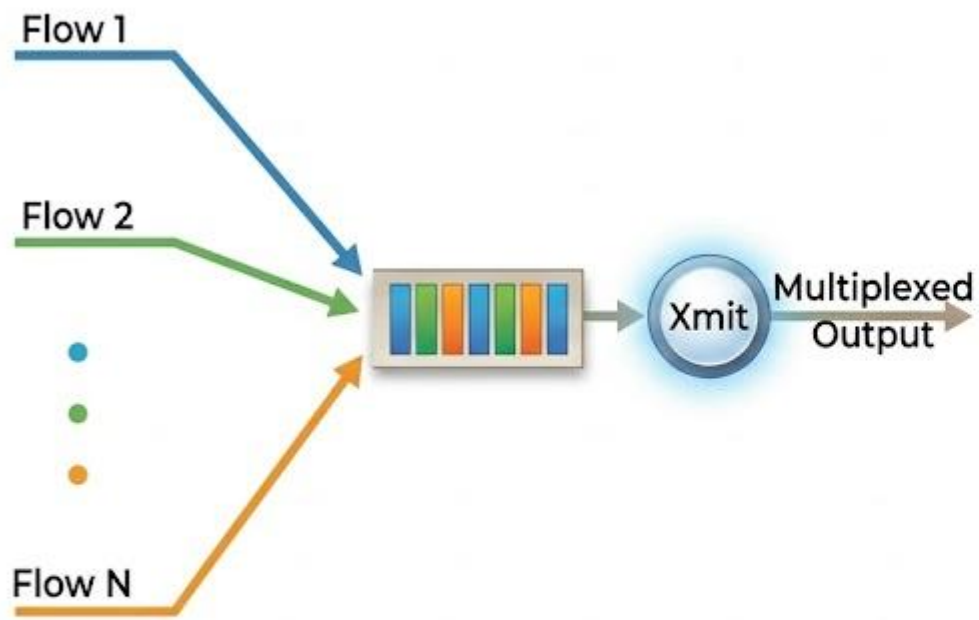
CPU Cycles

Processing power required to make routing decisions.

Packet Scheduling (2): FIFO vs Fair Queuing

FIFO (First-In, First-Out)

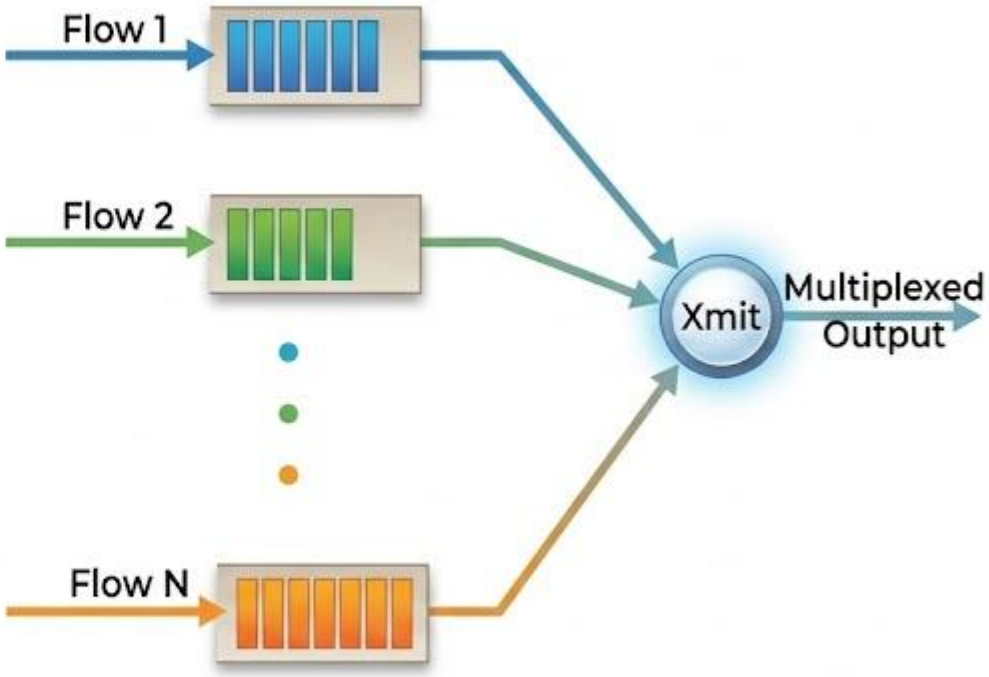
All packets go into a single queue. An aggressive sender (like a massive file transfer) can fill the queue, starving out small, delay-sensitive flows like VoIP.



(a) FIFO Queuing

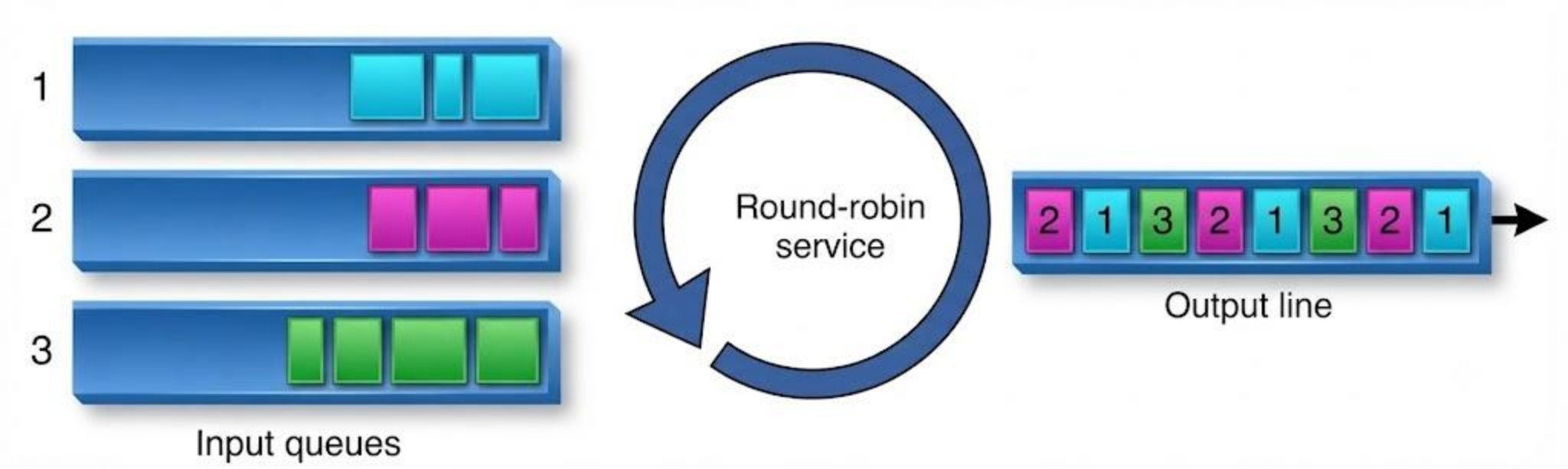
Fair Queuing

The router maintains a separate queue for each flow. It serves them in a round-robin fashion, ensuring no single flow can monopolize the link bandwidth.

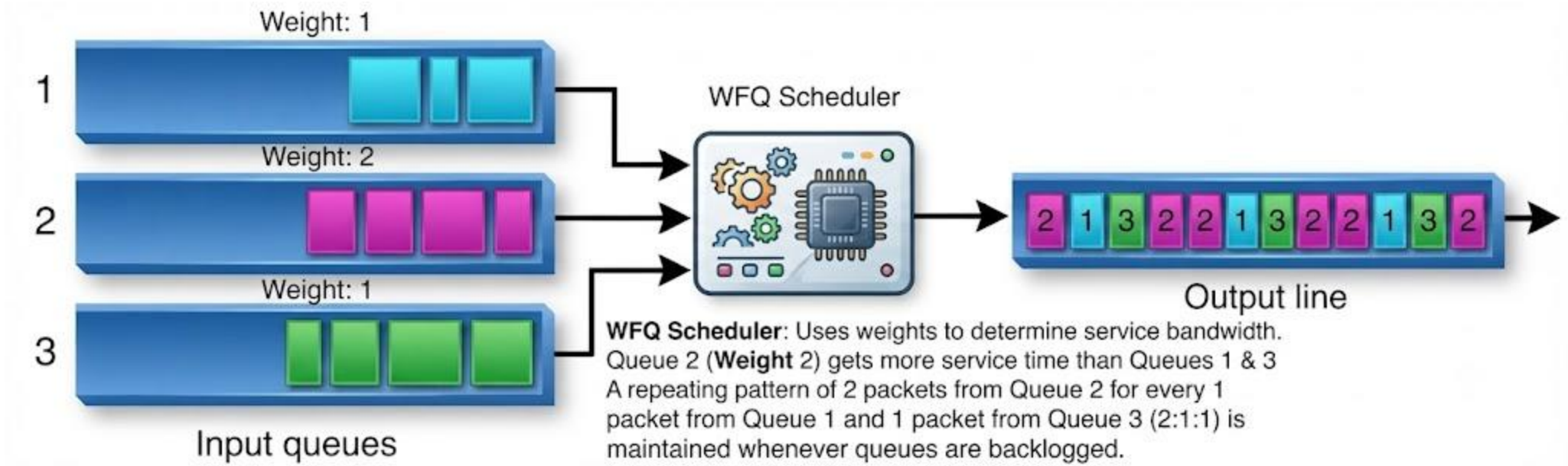


(b) Fair Queuing

Packet Scheduling (3): Round Robin Fair Queuing



Packet Scheduling (4): Weighted Fair Queuing



WFQ Scheduler: Uses weights to determine service bandwidth.
Queue 2 (**Weight** 2) gets more service time than Queues 1 & 3
A repeating pattern of 2 packets from Queue 2 for every 1 packet from Queue 1 and 1 packet from Queue 3 (2:1:1) is maintained whenever queues are backlogged.

Internetworking

Connecting radically different networks together (e.g., Ethernet to Wi-Fi to MPLS to 5G).

How do networks differ?

Item	Some Possibilities
Service offered	Connectionless versus connection-oriented
Addressing	Different sizes, flat or hierarchical
Broadcasting	Present or absent (also multicast)
Packet size	Every network has its own maximum (MTU)
Quality of Service	Present or absent; many different kinds
Reliability	Different level of loss
Security	Privacy rules, encryption
Parameters	Different timeouts, flow specifications etc
Accounting	By connect time, packets, bytes or not at all

How Networks can be connected?

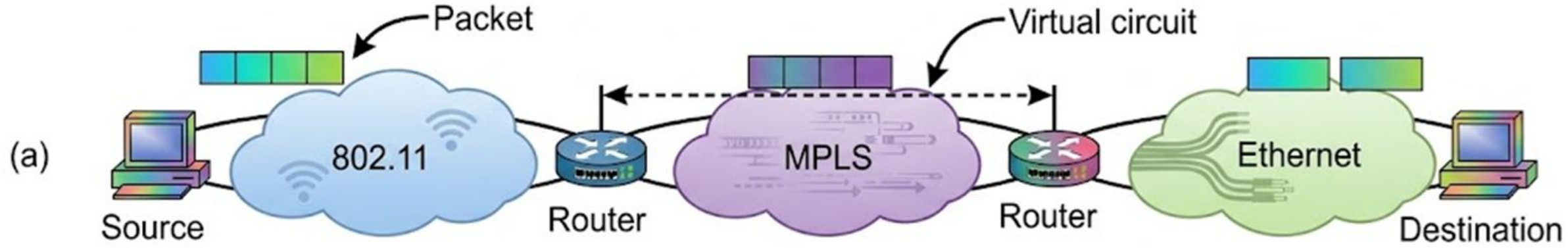


FIGURE (a): A packet crossing heterogeneous networks.

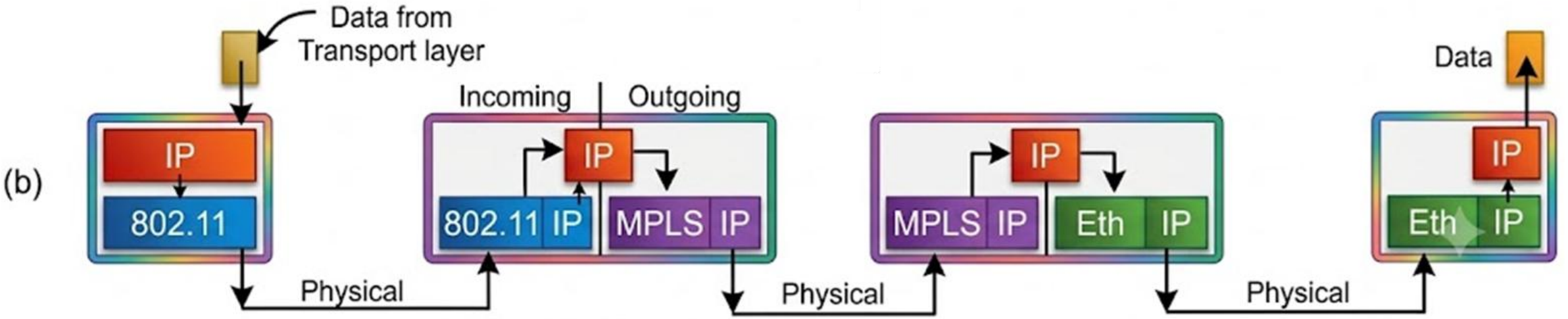


FIGURE (b): Network and link layer protocol processing.

Tunneling (1)

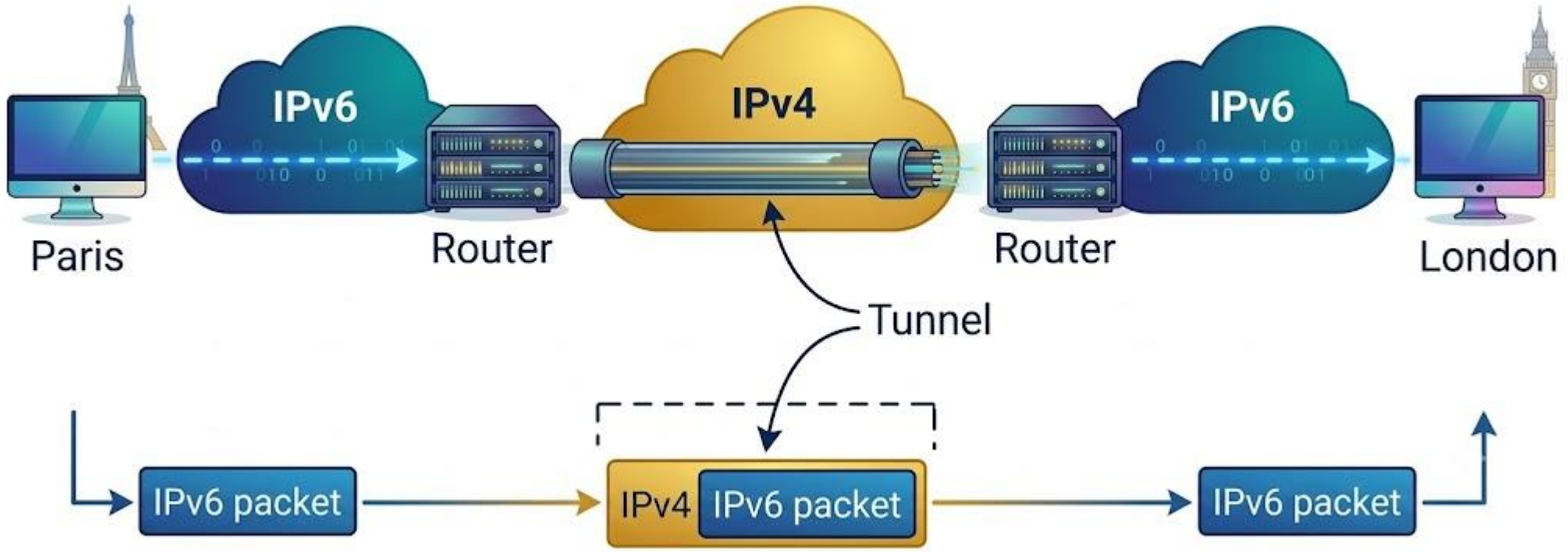
How to connect two networks of the same type across a network of a different type.

- The source router wraps the original packet completely inside the payload of a new packet formatted for the transit network.
- The transit network routes it like normal data.
- The exit/egress router decapsulates the original packet and forwards it.

> **Real World Use:** VPNs utilize tunneling. Your local computer encrypts an IP packet and tunnels it over the public Internet to the corporate firewall, which unwraps it and places it on the private corporate LAN.

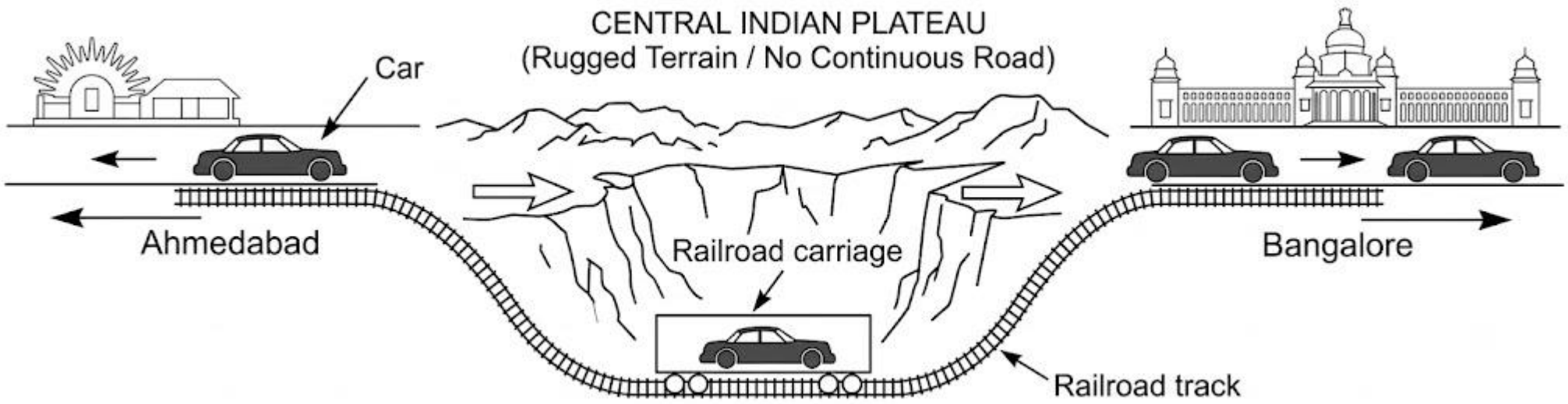
Tunneling (2)

Example: Tunneling an IPv6 packet from Paris to London across an older IPv4 backbone.



Tunneling (3)

Example: Transporting a car from Ahmedabad to Bangalore across a rugged terrain



Packet Fragmentation

Every network has a Maximum Transmission Unit (MTU) representing the largest packet it can carry. What happens when a large packet enters a network with a small MTU?

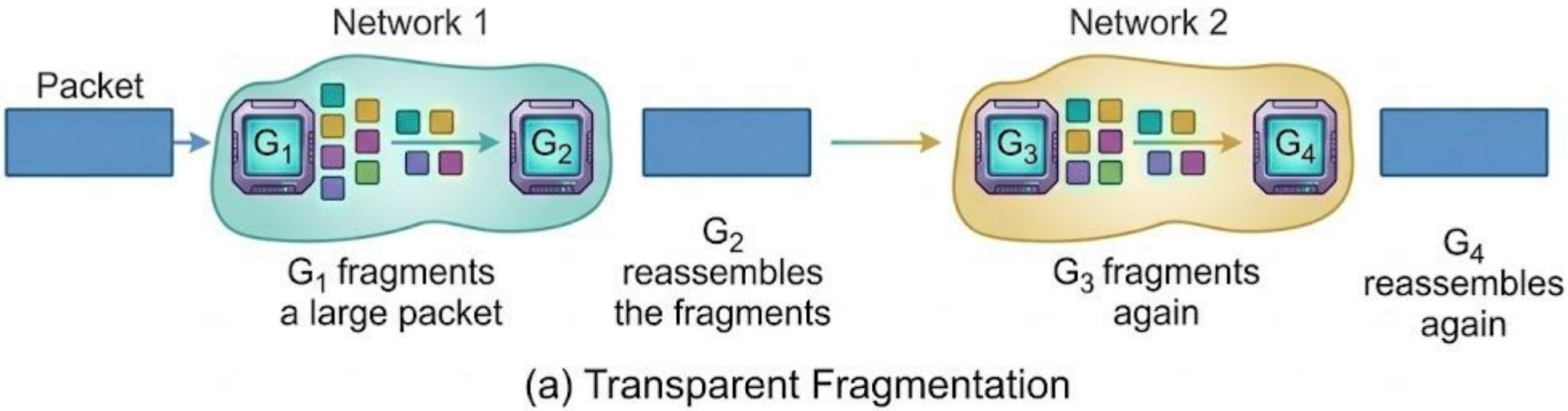
Transparent Fragmentation

- The boundary router breaks the packet into fragments, and the exit router reassembles them before forwarding. The hosts never know it happened.
- Fragments are reassembled by the very next router/gateway.
- Subsequent networks are unaware the packet was ever fragmented.
- Requires significant buffering memory on routers.

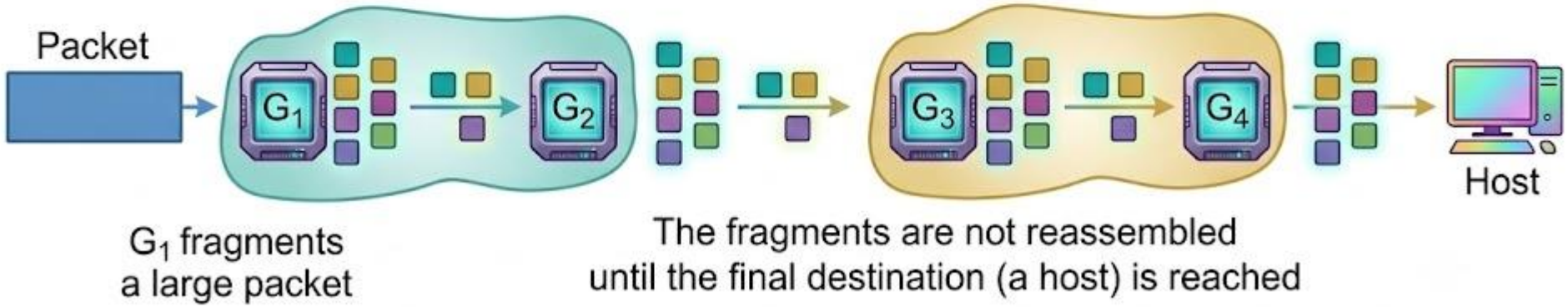
Nontransparent (IP style)

- The router breaks the packet into fragments, but they are NOT reassembled until they reach the final destination host. Much easier for routers to manage.
- The IP approach. Fragments remain fragmented until they reach the final destination host.
- Requires less work from intermediate routers.
- If one fragment is lost, the entire original packet must be retransmitted by the source.

Packet Fragmentation Overview



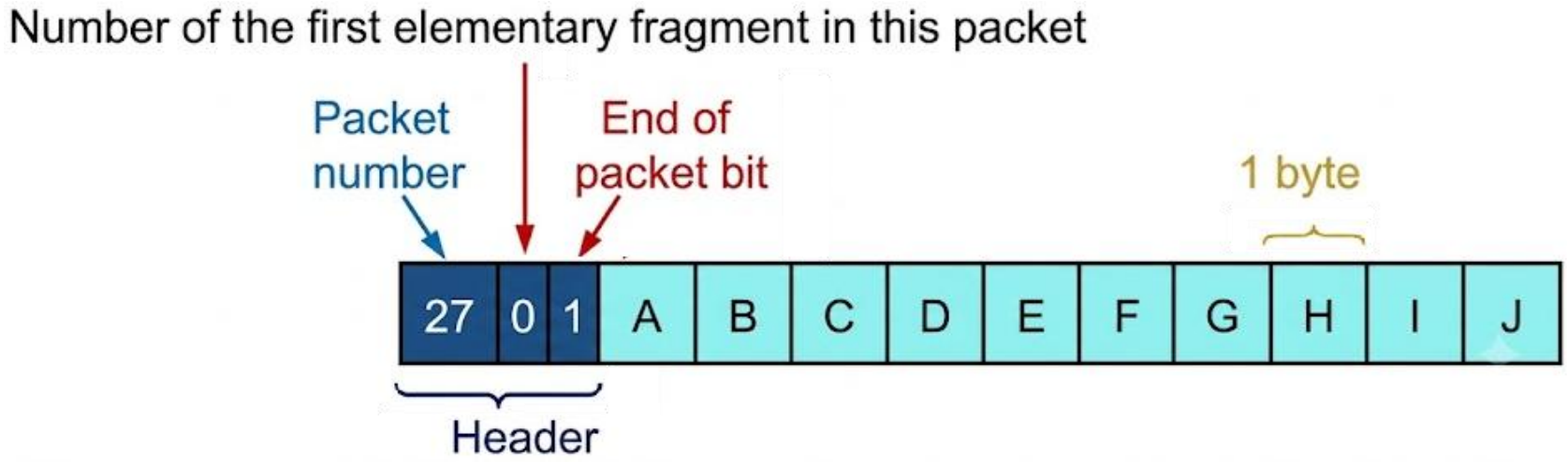
(a) Transparent Fragmentation



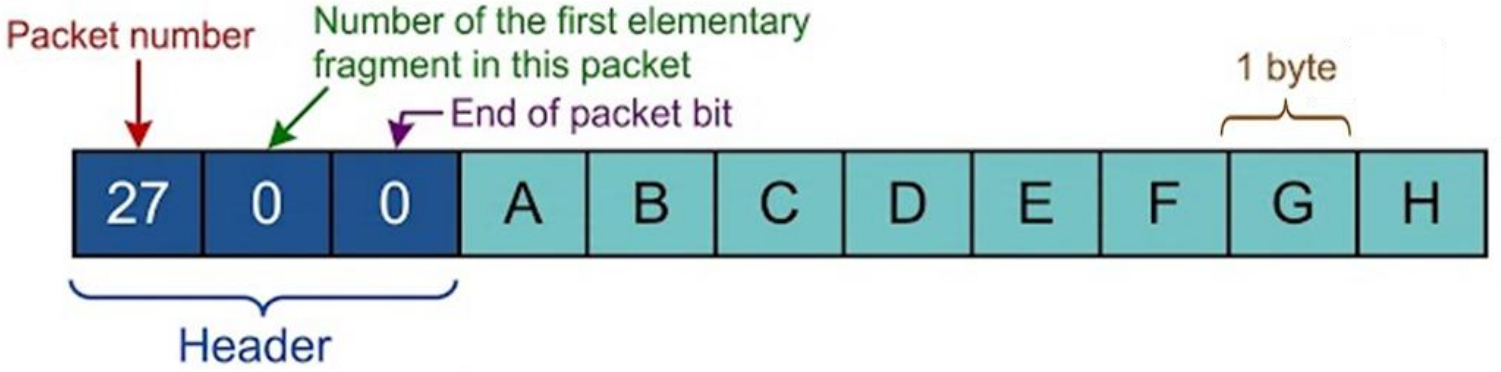
(b) Non-Transparent Fragmentation

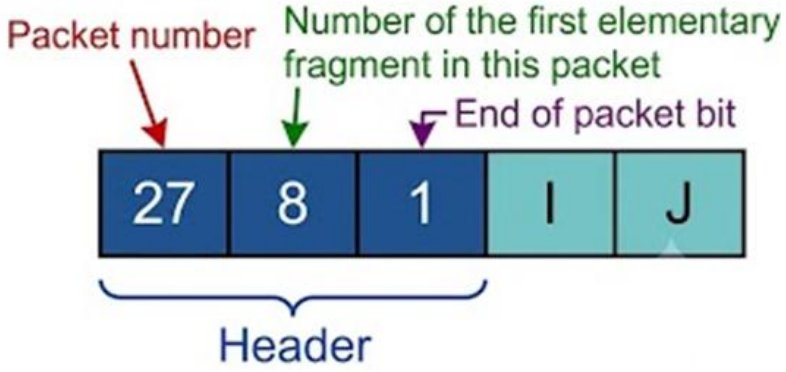
Fragmentation Trace (1)

Original packet: Contains 10 data bytes, header indicates packet number 27.



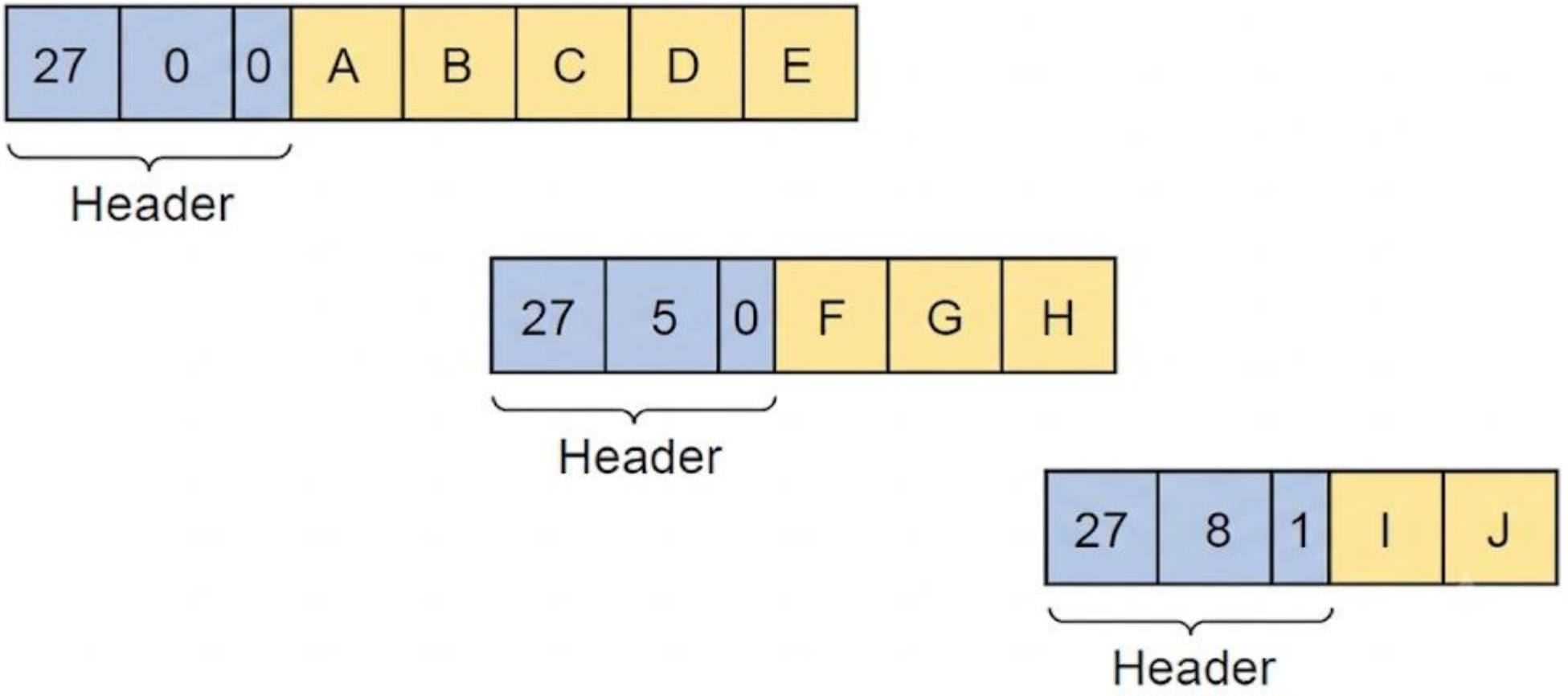
Fragments after passing through a network with maximum packet size of 8 payload bytes plus header.





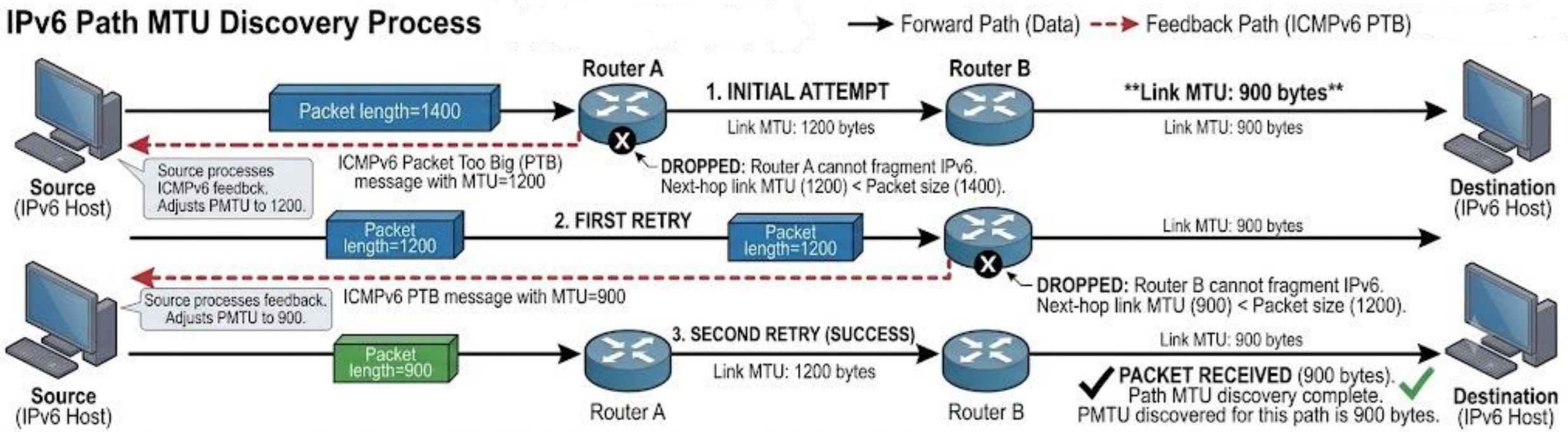
Fragmentation Trace (2)

Fragments after passing through a network with a maximum packet size of 5 payload bytes plus header.



Path MTU Discovery

- Fragmentation by routers is computationally expensive. Modern IP prefers **Path MTU Discovery**.
- Source host sends a large packet with the **DF (Don't Fragment)** bit set.
- If a router encounters an MTU too small for the packet, it drops it and sends an ICMP "Fragmentation Needed" error back to the host.
- The host lowers its transmission size and tries again until it finds a size that clears the entire path.



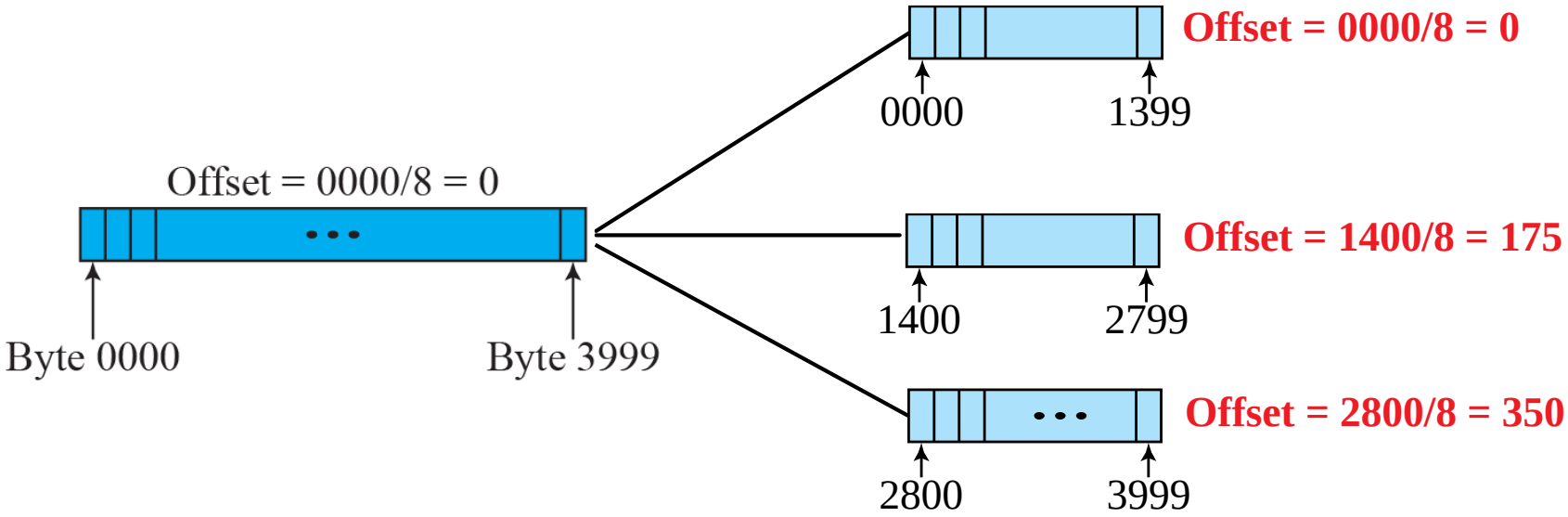
IP Fragmentation and Reassembly

- A router may need to fragment datagrams before sending to the next network
- Fragments may need further fragmentation in later networks
- Reassembly is done only at the final destination since fragments may take different routes
- If reassembly is permitted at intermediate routers
  - Large buffers are needed to store partial datagrams
  - All fragments must pass through the same gateway
- MTUs of some networks

Protocol	MTU
Hyperchannel	65,535
Token Ring (16 Mbps)	17,914
Token Ring (4 Mbps)	4,464
FDDI	4,352
Ethernet	1,500
X.25	576

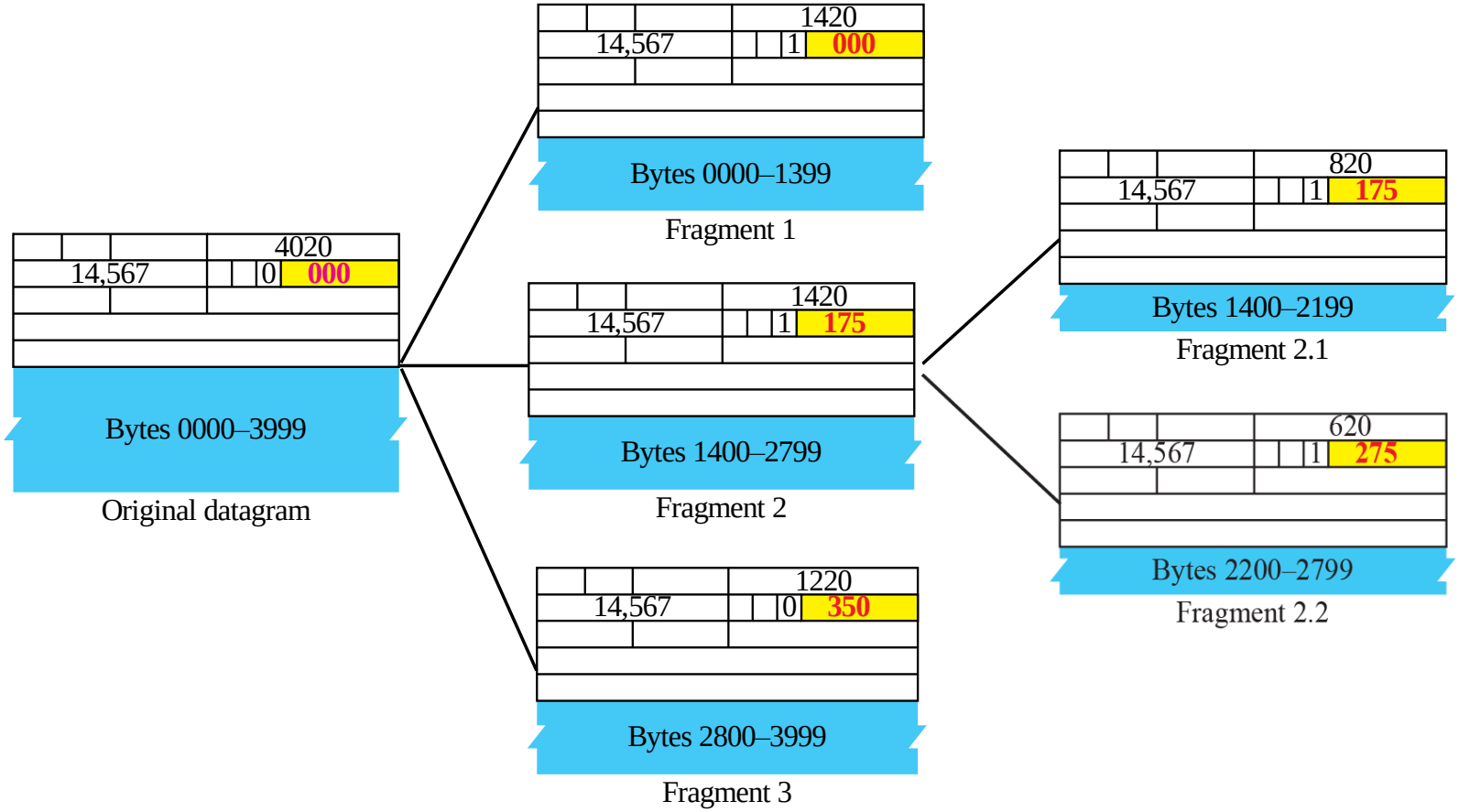
IP Fragmentation

- An IP packet carrying 4000 bytes is to be forwarded through a network with an MTU of 1400
- Fragment offset indicates the first byte number in the packet
- Fragment offset maintains byte number / 8. Fragment data size has to be multiple of 8 bytes

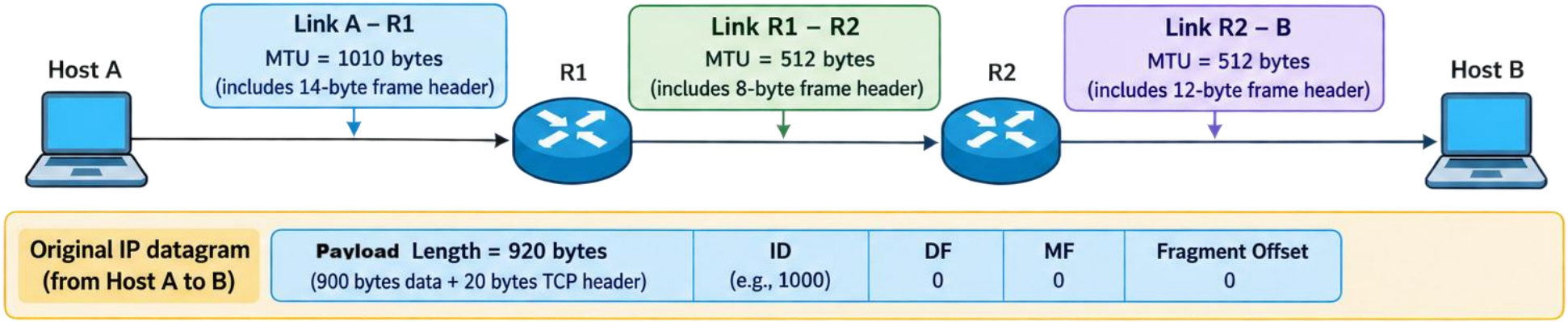


IP Fragmentation detailed example

- These fragmented datagrams are traveling through a network with an MTU of 820

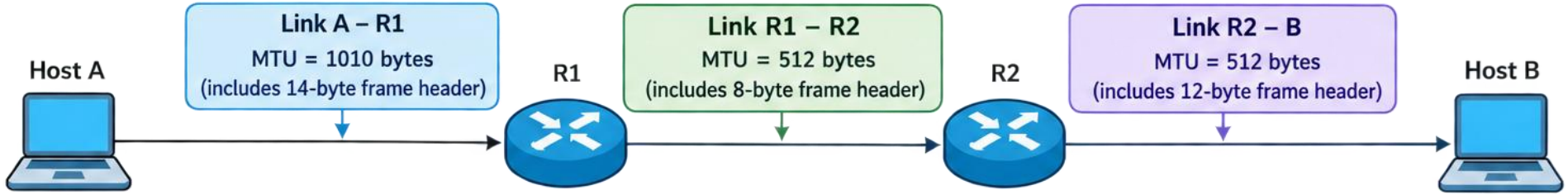


IP Fragmentation detailed example



- Suppose this TCP packet is passed to the IP code at host A for delivery to B. Show the total length, identification, DF, MF, and fragment offset fields of the IP header in each packet transmitted over the three links.

IP Fragmentation detailed example



Original IP datagram (from Host A to B)

Payload Length = 920 bytes (900 bytes data + 20 bytes TCP header)	ID (e.g., 1000)	DF 0	MF 0	Fragment Offset 0

1. Link A – R1 (MTU = 1010 bytes, includes 14-byte frame header)

Fragments sent over Link A – R1

Total Length	ID	DF	MF	Offset
940	1000	0	0	0

(includes 14-byte frame header)

No fragmentation on A – R1 because 920 bytes < 1010 bytes (MTU).

2. Link R1 – R2 (MTU = 512 bytes, includes 8-byte frame header)

Fragments sent over Link R1 – R2

Total Length	ID	DF	MF	Offset
500	1000	0	1	0

(includes 8-byte frame header)

Fragment 1

Total Length	ID	DF	MF	Offset
460	1000	0	0	60

(includes 8-byte frame header)

Fragment 2

Fragmentation occurs on R1 – R2 because 940 bytes > 512 bytes (MTU).

3. Link R2 – B (MTU = 512 bytes, includes 12-byte frame header)

Fragments sent over Link R2 – B

Total Length	ID	DF	MF	Offset
500	1000	0	1	0

(includes 12-byte frame header)

Fragment 1

Total Length	ID	DF	MF	Offset
460	1000	0	0	60

(includes 12-byte frame header)

Fragment 2

No further fragmentation on R2 – B because both fragments (500 and 460) are ≤ 512 bytes (MTU).

The Network Layer Principles

Guiding philosophies behind Internet architecture:

- **Make sure it works:** Do not finalize the standard until there are multiple, working prototype implementations.
- **Keep it simple:** When in doubt, leave it out. Complexity is the enemy of reliability.
- **Make clear choices:** If there are multiple ways to do something, pick one.
- **Exploit modularity:** This led directly to the protocol stack concept.
- **Expect heterogeneity:** Different hardware, different transmission media, different speeds.
- **Avoid static options and parameters:** Hardcoded limits usually backfire as the network scales.
- **Look for good design (not perfect):** Perfection takes too long.
- **Strict sending, tolerant receiving:** Send perfectly formatted packets, but be prepared to handle slightly malformed incoming packets
- **Think about scalability:** Can it handle billions of hosts?

The Network Layer in the Internet

The Internet is not a single entity, but an interconnected collection of many autonomous networks.

Tier 1 Backbones

Massive, high-speed fiber networks linking continents (e.g., AT&T, Level 3).

Regional ISPs

Connect to backbones and distribute connectivity to specific geographic areas.

Edge Networks

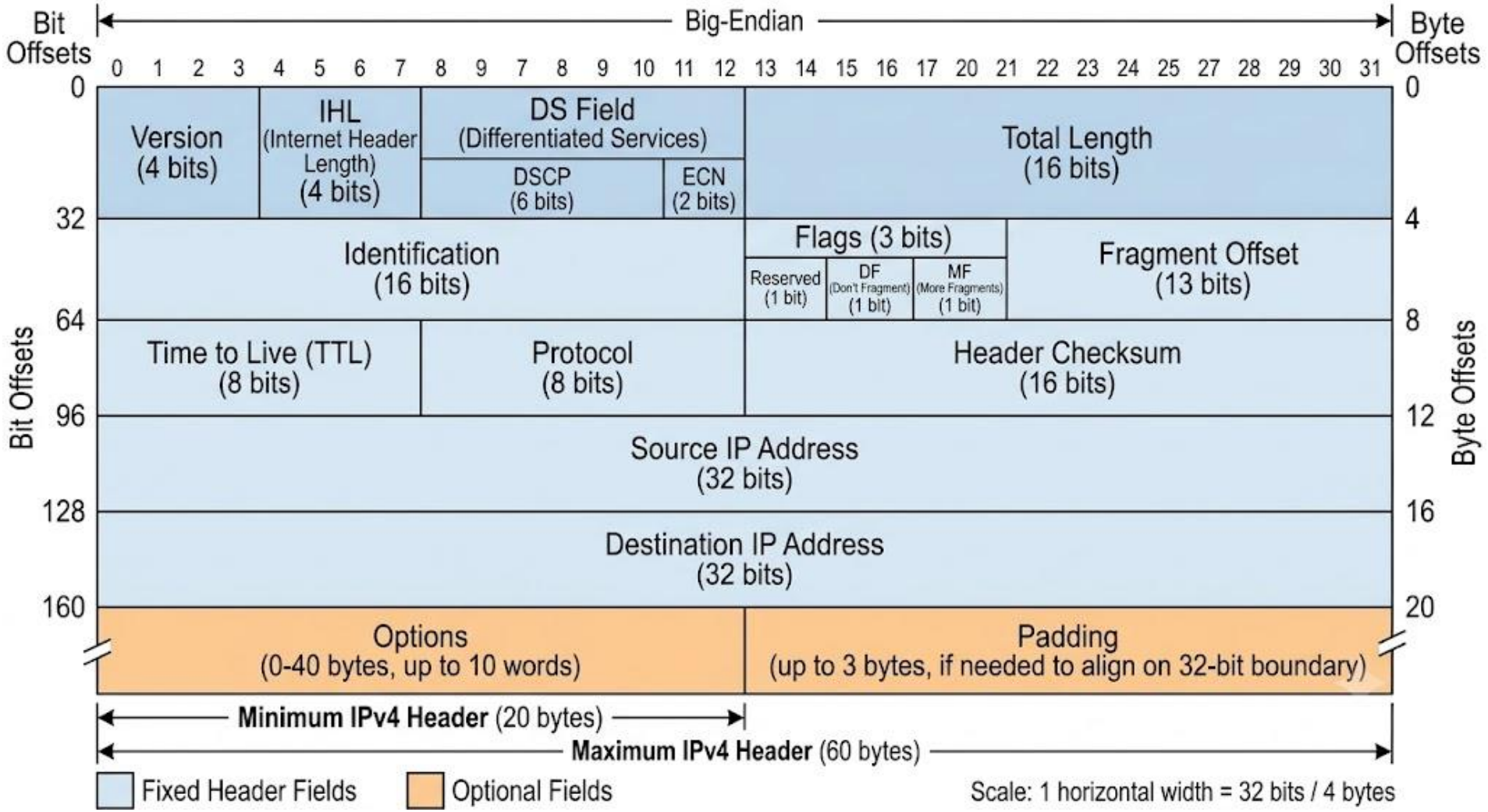
Corporate LANs, Home WiFi, and Mobile Cellular networks forming the perimeter.

The IP Version 4 Protocol

The glue that holds the entire Internet together.

The IPv4 Header Format

The IPv4 header is transmitted in 32-bit (4-byte) words. The minimum header is 20 bytes.



IPv4 Header Key Fields

IHL & Total Length

IHL: Header length in 32-bit words (usually 5 = 20 bytes).

Total Length: Entire packet size (header + data) in bytes. Max is 65,535 bytes.

Protocol

Tells the destination host which transport layer process to hand the payload to.

Examples: TCP (6), UDP (17), ICMP (1).

Fragmentation

ID: Uniquely tags the original packet.

Flags: DF (Don't Fragment) & MF (More Fragments).

Offset: Where this fragment belongs.

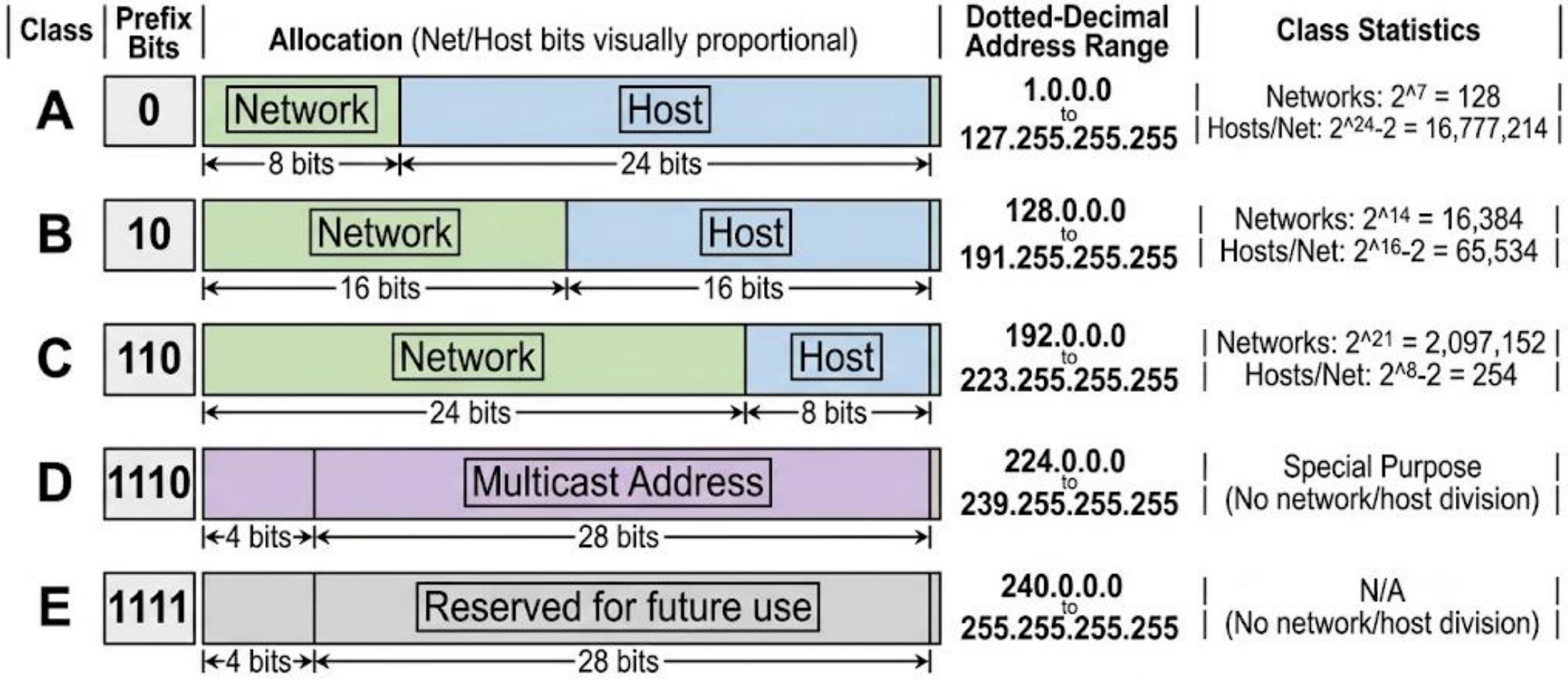
IP Options

The IPv4 header can be extended beyond 20 bytes using Options, though they are rarely used today.

Option	Description
Security	Specifies how secret the datagram is (military use).
Strict Source Routing	Gives the complete, exact path of routers to be followed.
Loose Source Routing	Gives a list of routers that *must not* be missed.
Record Route	Makes each router append its IP address to the header.
Timestamp	Makes each router append its address and a timestamp.

> **Security Warning:** Today, most core internet routers are configured to simply drop packets containing IP options (like Source Routing) because they can be exploited to bypass firewalls or launch DoS attacks.

IP Addresses : The Prefix

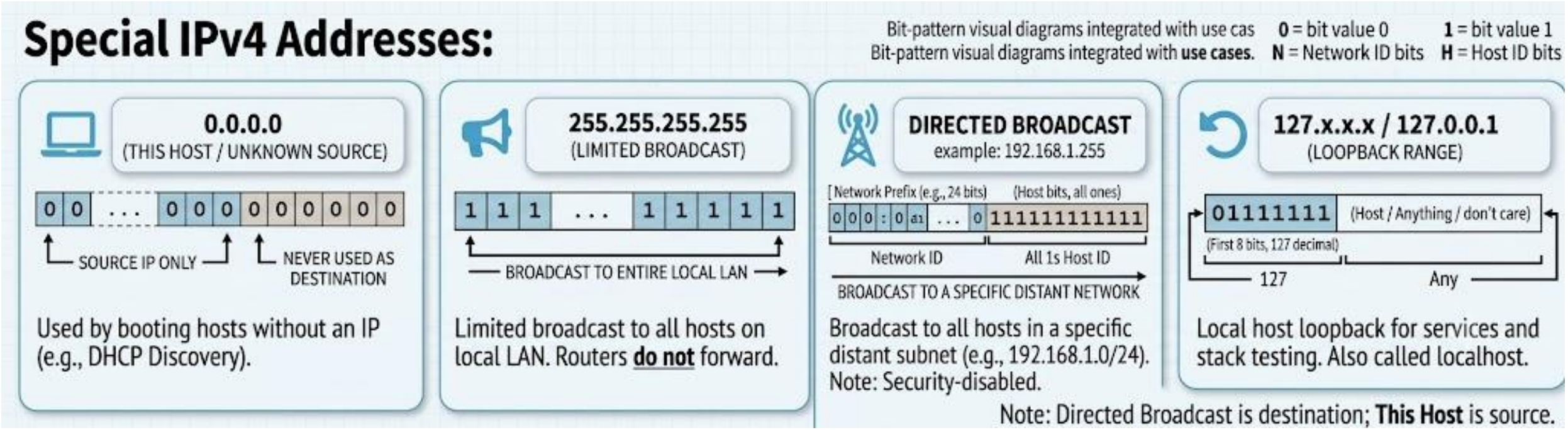


Note: Special-use and reserved ranges (Loopback, Private, etc.) are included in the full technical ranges. Usable host count per network is 2^(host bits)-2.

Two-Part Structure: An IP address is logically divided into two parts: the **Network Prefix** (which identifies the specific LAN or subnet) and the **Host ID** (which identifies the specific machine on that LAN).

Special IP Addresses

Certain IP addresses have reserved, special meanings and cannot be assigned to standard hosts.



Subnets (1)

Instead of having one massive flat network, large blocks of IPs are broken down into smaller chunks called Subnets.

1 INITIAL STATE: ONE LARGE NETWORK

Original Class C Network (Example)

IP: 192.168.1.0
Subnet Mask: 255.255.255.0 (/24)

IP ADDRESS BIT BREAKDOWN

192.	168.	1.	0

NETWORK ID (24 bits) | HOST ID (8 bits)

Large Broadcast Domain.
Limited Segmentation.
Potential for Broadcast Storms.

Subnet Mask	(255.255.255.0)
Available Networks:	($2^0 = 1$)
Hosts per Network:	($2^9 - 2 = 254$)

2 THE SUBNETTING PROCESS: BORROWING BITS

GOAL: Create 2 Subnets from the original /24 Network

/24 → /25

IP ADDRESS BIT BREAKDOWN (with Subnetting)

192.	168.	1.	0

Subnet ID (1 bit) | Host ID (7 bits)

Subnet Mask: 255.255.255.128

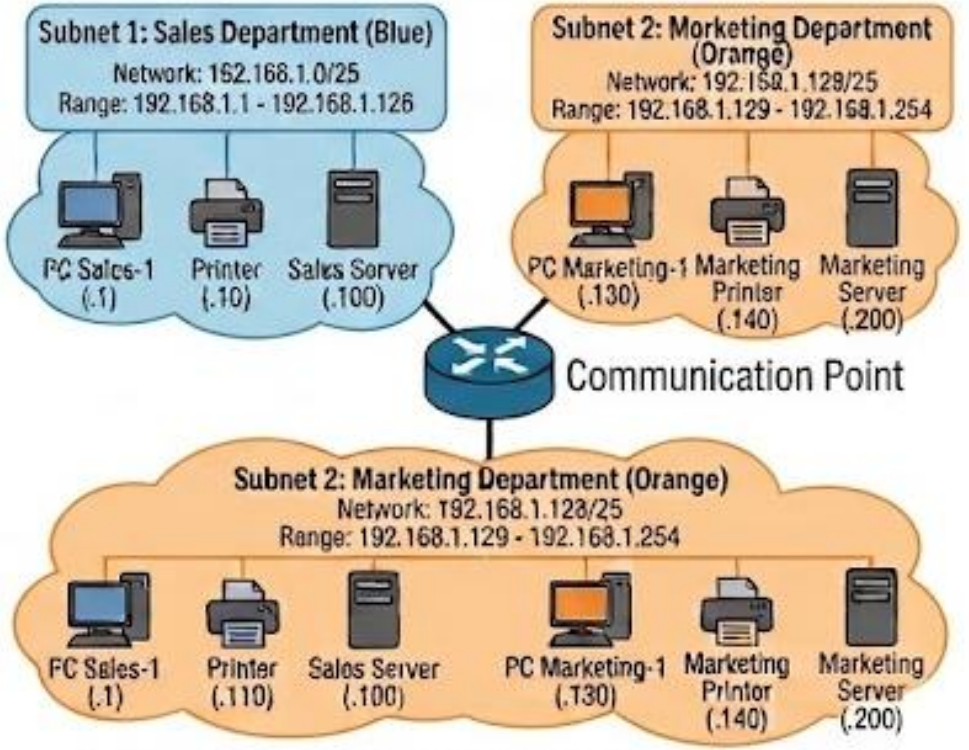
(Stats)

$2^n = 255.255.255.128$

$2^{\wedge n}$ where n is borrowed bits = 2^{n1} = 2 Subnets

Subnet Mask	(255.255.255.128)
Subnets per Network:	($2^{\wedge 1} = 2$)
Hosts per Subnet:	($2^7 - 2 = 126$)

3 THE RESULTING FIGURE: TWO SEGREGATED SUBNETS



BROADCAST DOMAIN COMPARISON

- Sales broadcast only to Sales hosts
- Marketing broadcast only to Marketing hosts

Subnet Mask	Subnet ID	Host ID	Broadcast Domain

CIDR: Classless Inter-Domain Routing

The rigid A/B/C classes caused massive IP address waste. A company needing 500 IPs had to buy a Class B (65,534 IPs), wasting 65,000 addresses.

CIDR abolishes classes. Network sizes are arbitrary.

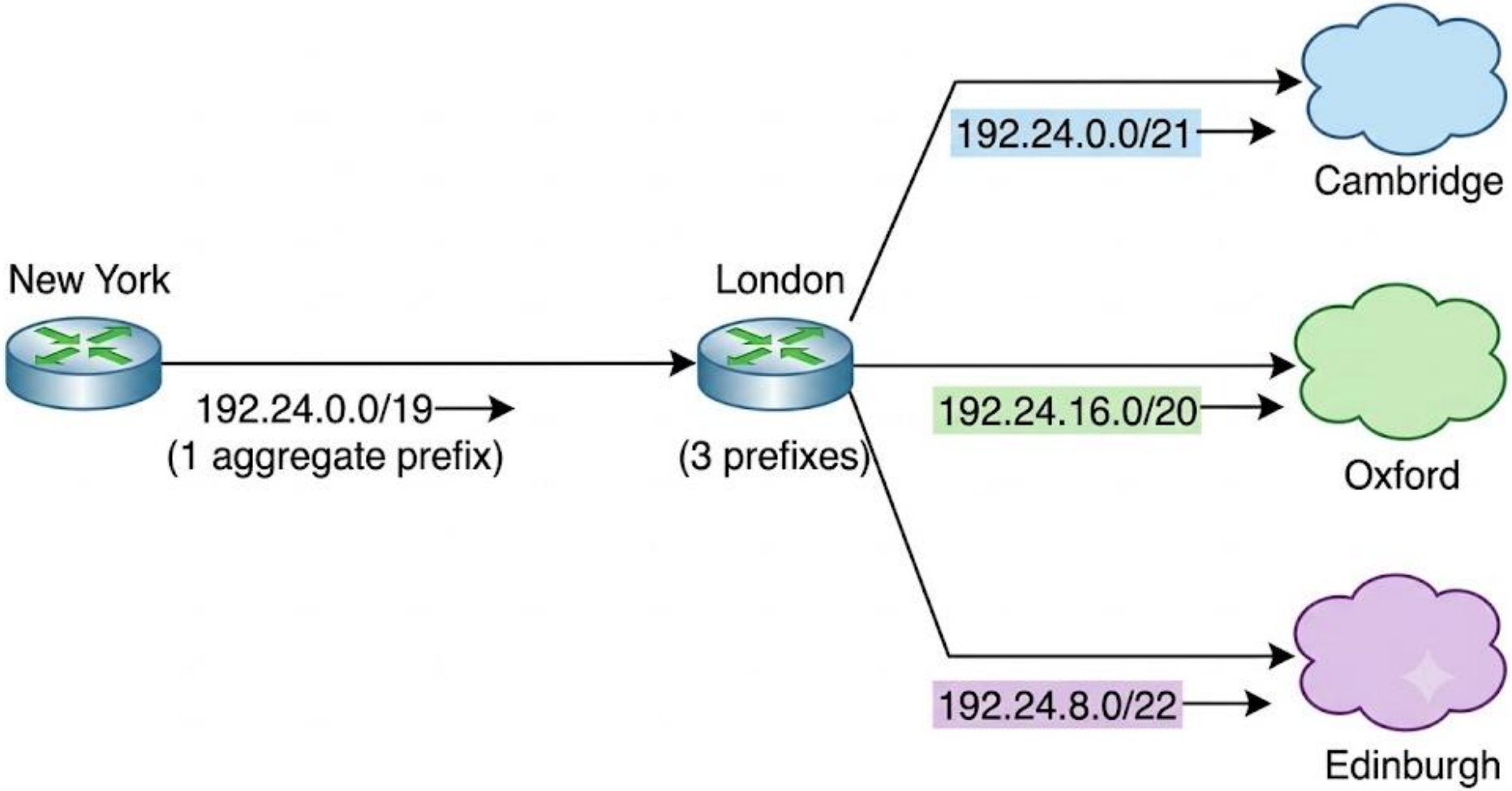
Prefixes are denoted by slash notation indicating the mask length.

Example: `194.24.0.0/21` bits define the network, leaving 11 bits for hosts (2048 hosts).

Organization	IP Block/Prefix	First Address	Last Address	Hosts
Cambridge	194.24.0.0/21	194.24.0.0	194.24.7.255	2048
Edinburgh	194.24.8.0/22	194.24.8.0	194.24.11.255	1024
Free	194.24.12.0/20	194.24.12.0	194.24.15.255	1024
Oxford	194.24.16.0/20	194.24.16.0	194.24.31.255	4096

Route Aggregation (Summarization)

CIDR allows adjacent IP blocks to be merged into a single routing table entry, shrinking internet routing tables.



The Post Office Analogy: The New York router doesn't need to know the specific path to Cambridge, Edinburgh, and Oxford. It just needs a single rule: "Send everything starting with 192.24.0.0/19 to London. London will sort it out."

Longest Matching Prefix

Routing Table
at London Router

	Address	Mask	Interface
Cambridge	194.24.0.0	21	Eth0
Edinberg	194.24.8.0	22	Eth1
Oxford	194.24.16.0	20	Eth2

Consider a packet with 194.24.17.4. Compare it to all the entries and use the longest matching entry

Routing Table
at New York Router

	Address	Mask	Interface
Cambridge	194.24.0.0	21	Eth0
Edinberg	194.24.8.0	22	Eth0
Oxford	194.24.16.0	20	Eth0

Aggregated Routing
Table at NY Router

	Address	Mask	Interface
All	194.24.0.0	19	Eth0

The Rule: Routers always choose the **Longest Match** (the rule with the most specific /XX number). If a packet arrives for 194.24.8.5, it matches both /19 and /22. The router selects the /22 path because it provides a more exact, specific route.

The IPv4 Shortage

The 32-bit IPv4 address space yields roughly 4.3 billion addresses. This is not enough for the modern world.

Consumer ISPs typically only issue one IP address per household.

Today's households have dozens of networked devices (Laptops, TVs, Phones, Tablets, IoT appliances).

How do we get all these devices online with only one public IP?



Private IP Networks

The Internet Engineering Task Force (IETF) reserved three blocks of IP addresses for internal, private use.

Class A Range

`10.0.0.0`

to

`10.255.255.255`

Class B Range

`172.16.0.0`

to

`172.31.255.255`

Class C Range

`192.168.0.0`

to

`192.168.255.255`

Not Globally Unique: Reused numerous times across networks

Non-Routable: Core internet routers are programmed to instantly drop any packet with a destination address in these ranges. They are strictly for use *inside* a Local Area Network (LAN).

Network Address Translation (NAT)

NAT allows multiple hosts on a private network to communicate with the Internet using a single public IP address.

A special router sits at the boundary of the private network.

- It intercepts outgoing packets and replaces the internal private IP with its own external public IP.
- It maintains a translation table of active flows to map returning internet traffic back to the correct internal machine.

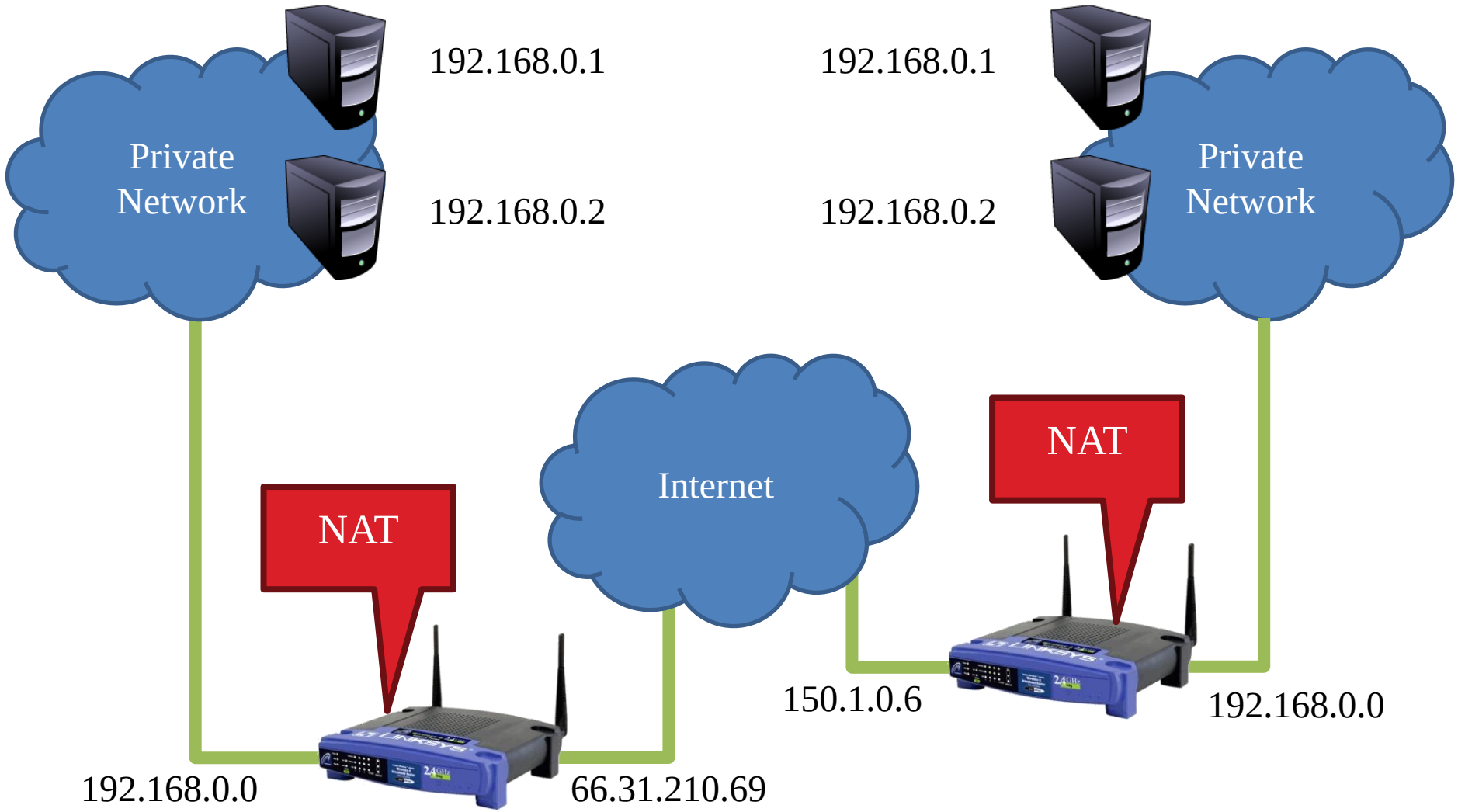
Basic NAT Operation

Because multiple internal devices might connect to the same external server, NAT must translate **TCP/UDP Port numbers** as well as IP addresses to keep flows separate (often called PAT - Port Address Translation).

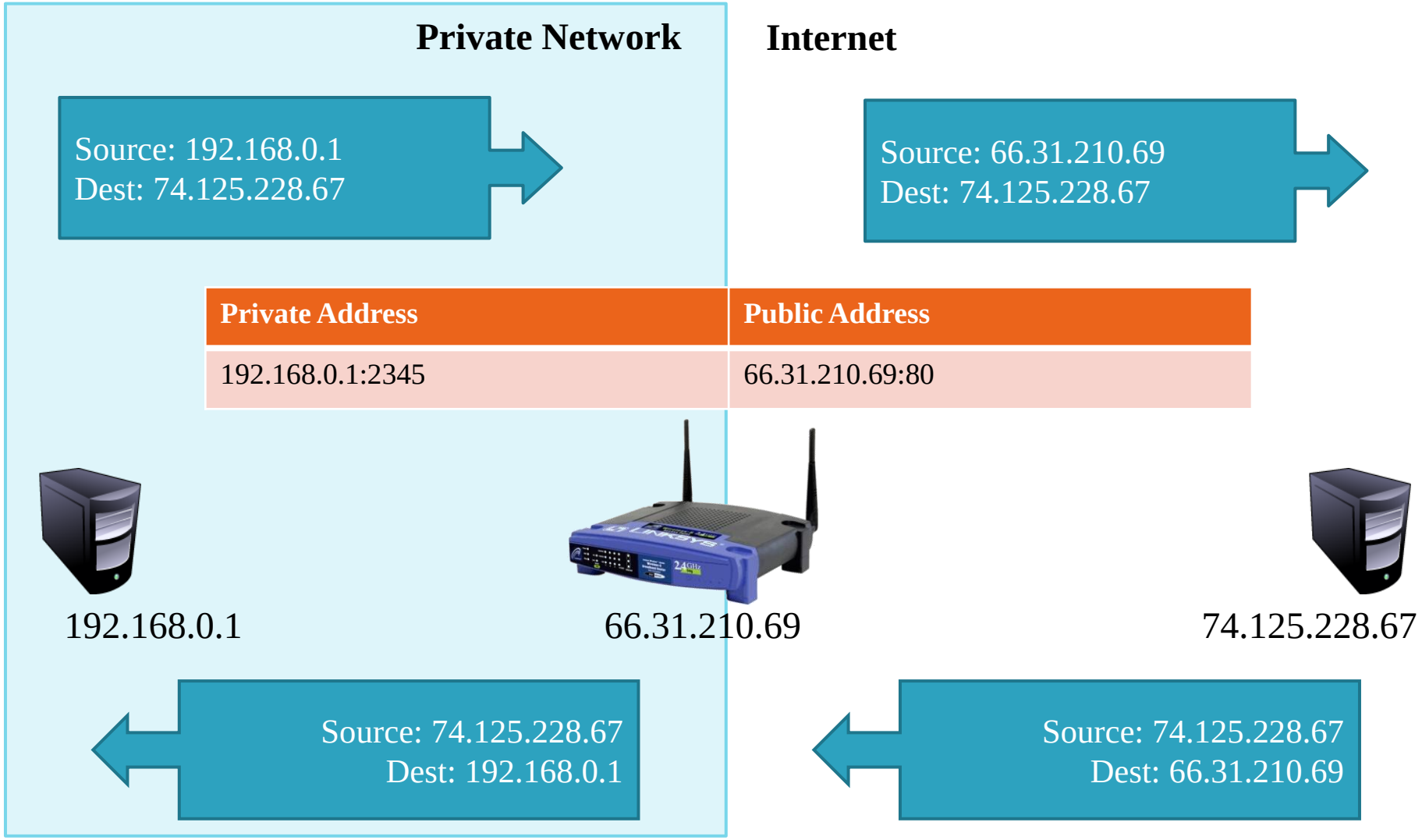
Internal IP:Port	NAT Router Public IP:Port	External Server
192.168.0.2 : 5500	66.31.210.69 : **1024**	74.125.228.67 : 80
192.168.0.3 : 5500	66.31.210.69 : **1025**	74.125.228.67 : 80

> **The Magic:** When the web server replies to `66.31.210.69:1025`; its table, rewrites the destination to , and forwards it internally. `192.168.0.3:5500`

Basic NAT Operation



Basic NAT Operation



Advantages of NAT

Conserves IPv4

Allows dozens of internal devices to share a single, scarce public IP address.

Easy ISP Migration

If you change ISPs, your public IP changes, but your entire internal 192.168.x.x network remains untouched.

Load Balancing

NAT can be configured to forward incoming web traffic to multiple internal servers, balancing the load.

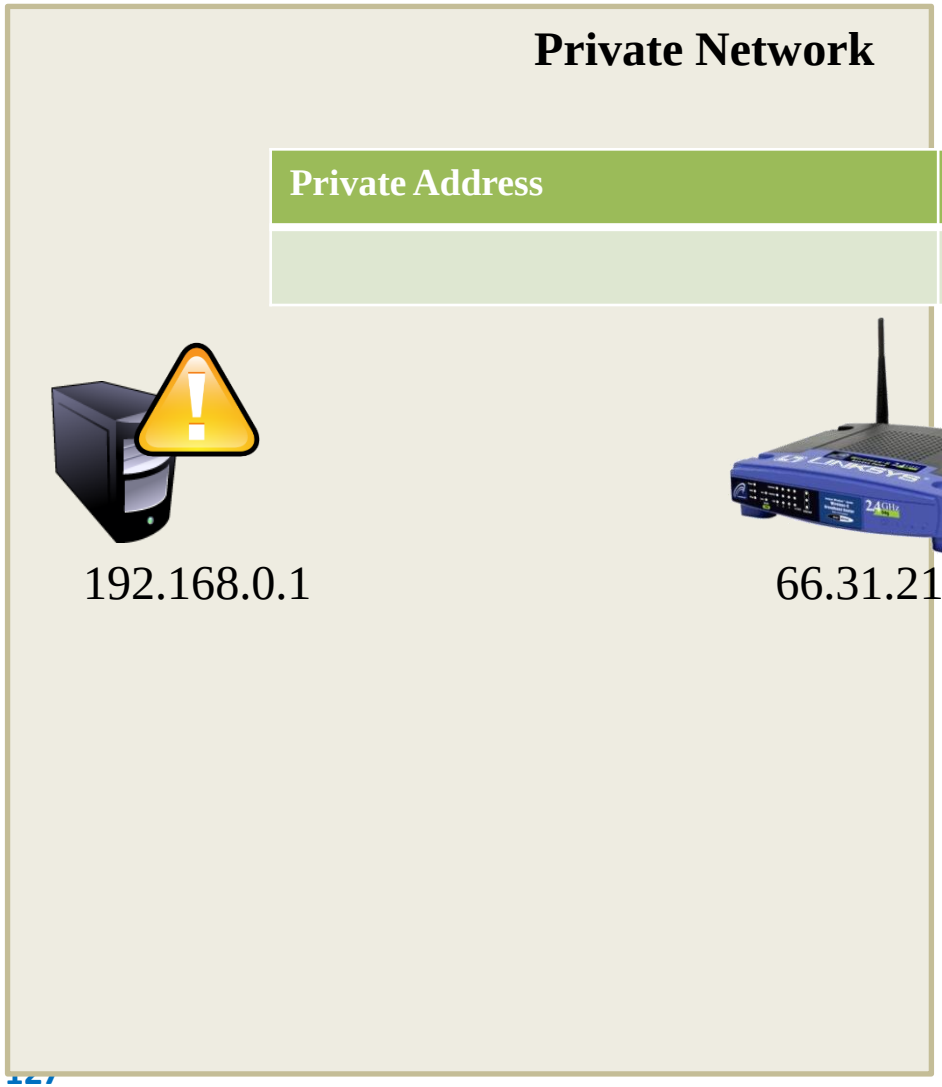
Concerns About NAT

Despite saving IPv4, NAT is heavily criticized by network purists.

- **Breaks Layered Abstraction:** The Network layer (IP) router is digging into the Transport layer (TCP/UDP) to read and modify ports.
- **Performance Hit:** Because IP and Port numbers change, the NAT router must recalculate the TCP and IP checksums for every single packet.
- **Embedded IPs:** Protocols like FTP and SIP embed IP addresses inside the data payload. NAT doesn't translate the payload, breaking these protocols unless specific application-level gateways (ALGs) are used.

NAT as a Natural Firewall

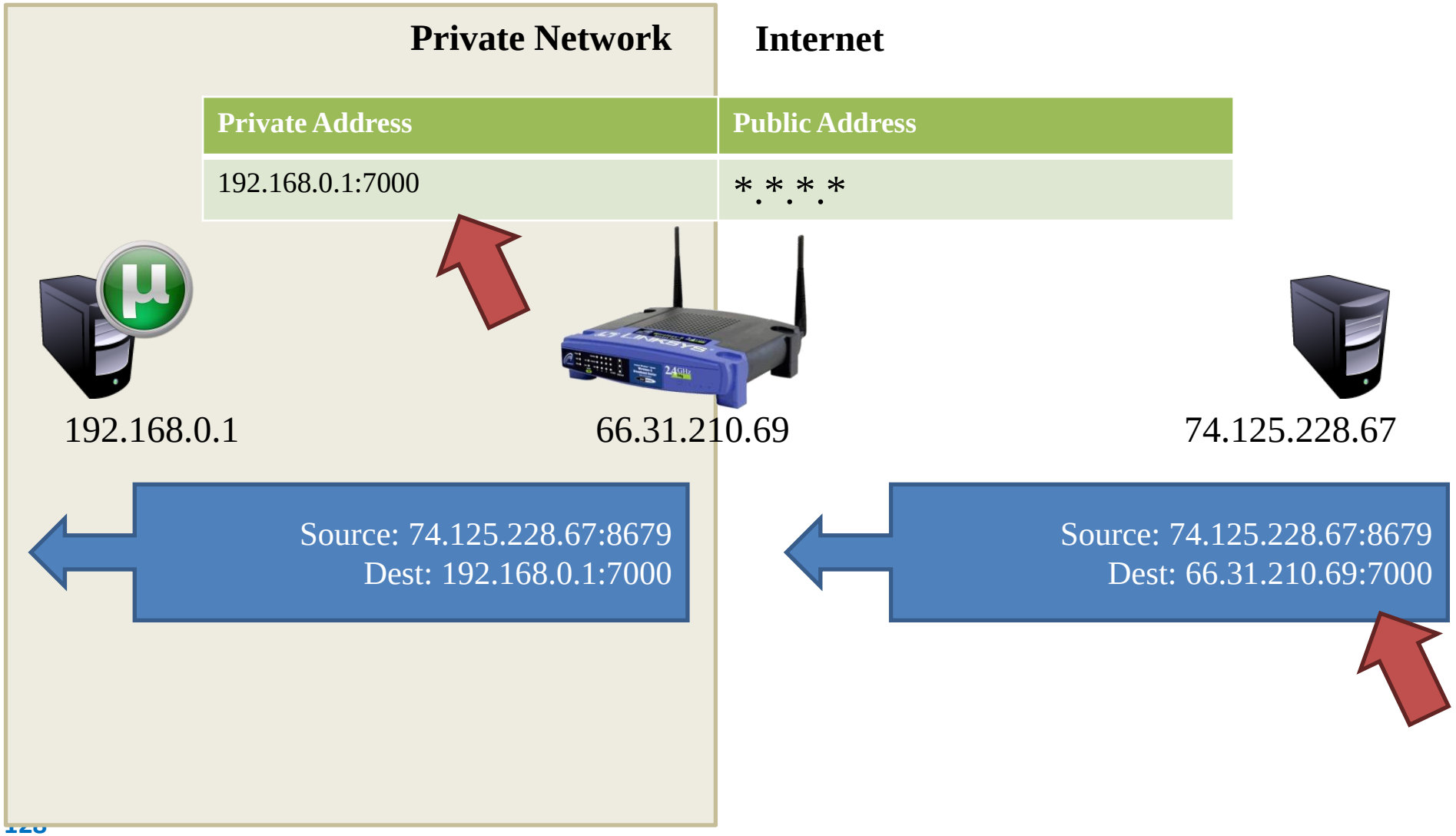
NAT inherently blocks incoming connections. If a hacker scans your public IP, the NAT router has no table entry for the



behind a NAT, you must manually punch a hole.

to Internal IP 192.168.0.5 Port 80".

Port Forwarding



IP Version 6 (The Next Generation)

Solving the IPv4 shortage and streamlining the network layer.

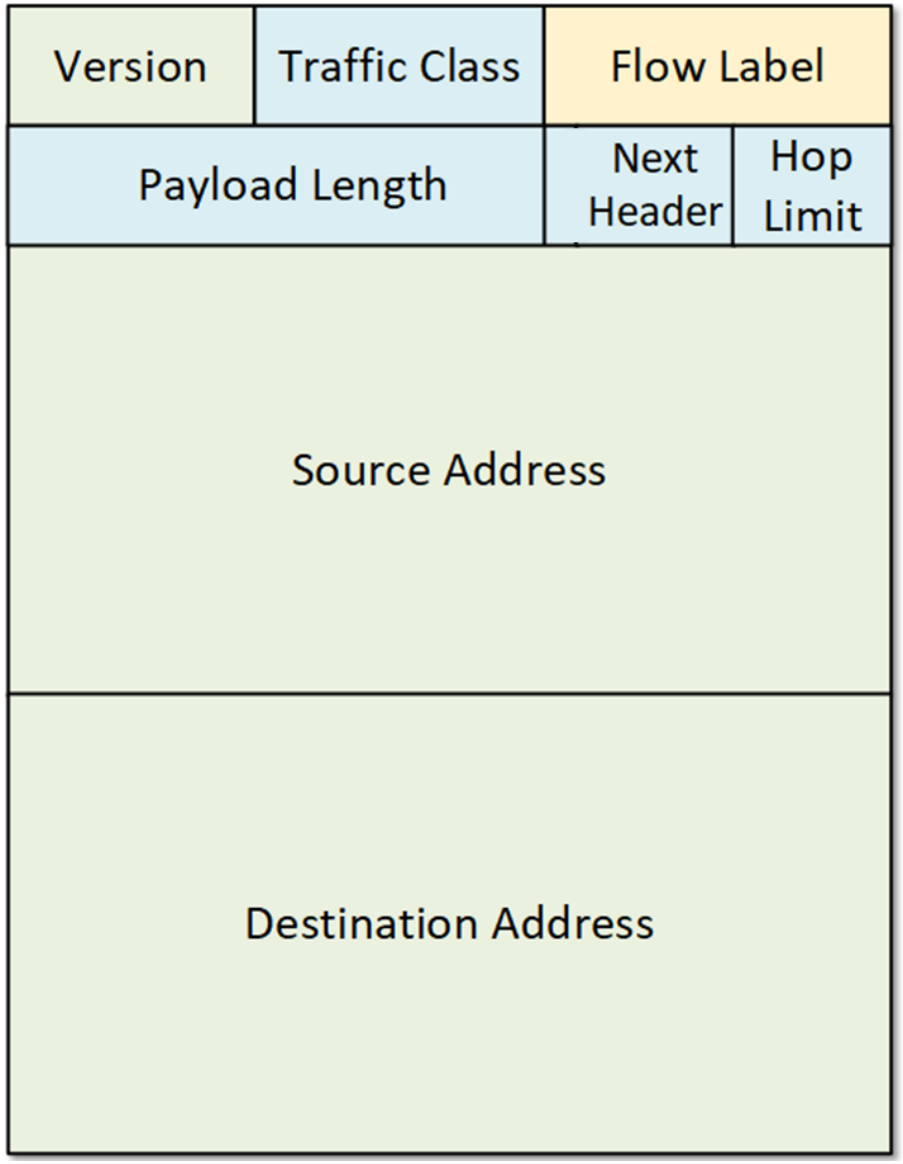
Goals of IPv6

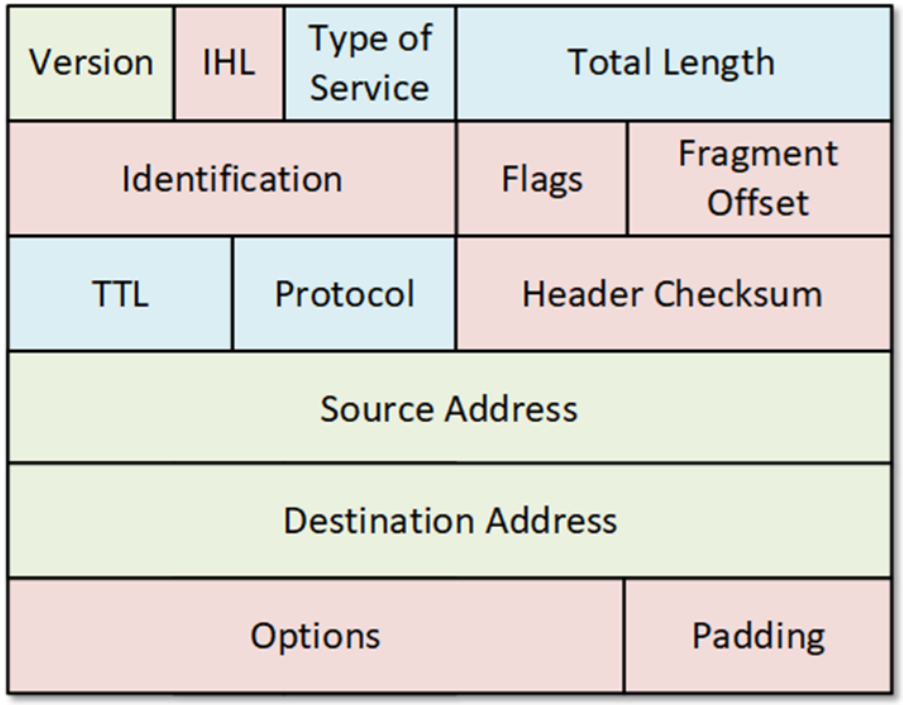
- **Support billions of hosts:** Even with inefficient address space allocation.
- **Reduce routing table size:** Simplifying core internet routers.
- **Simplify the protocol:** Allow routers to process packets faster.
- **Better security:** Make authentication and privacy key features.
- **Type of Service:** Pay more attention to real-time payloads.
- **Aid multicasting:** Allow specifying scopes.
- **Roaming hosts:** Support moving without changing IP addresses.
- **Coexistence:** Permit old and new protocols to run simultaneously for years.

The Scale of IPv6: The IPv6 address space uses 128 bits, providing 2^{128} addresses. This is roughly 3.4×10^{38} addresses, meaning we could assign a unique IP to every grain of sand on Earth multiple times over.

The IPv6 Fixed Header

The IPv6 fixed header is exactly 40 bytes long. It has been dramatically simplified from IPv4.





Legend

- Fields **kept** in IPv6
- Fields **kept** in IPv6, but name and position changed
- Fields **not kept** in IPv6
- Fields that are **new** in IPv6

Streamlining the Header

To achieve high-speed processing, several IPv4 fields were entirely removed in IPv6.

No Checksum

Because data-link layers (Ethernet) and transport layers (TCP/UDP) both have their own checksums, checking it again at the Network layer was deemed redundant and costly.

No Fragmentation

IPv6 routers **do not fragment** packets. If a packet is too big, the router drops it and sends an ICMP error. Senders must use Path MTU Discovery.

Few fields are maintained but renamed

Hop Limit

Hop Limit and TTL: In IPv4, it was called TTL, supposedly measured in seconds. In practice, routers decremented it by 1 per hop. IPv6 renames it to Hop Limit to reflect what it actually does.

Next Header

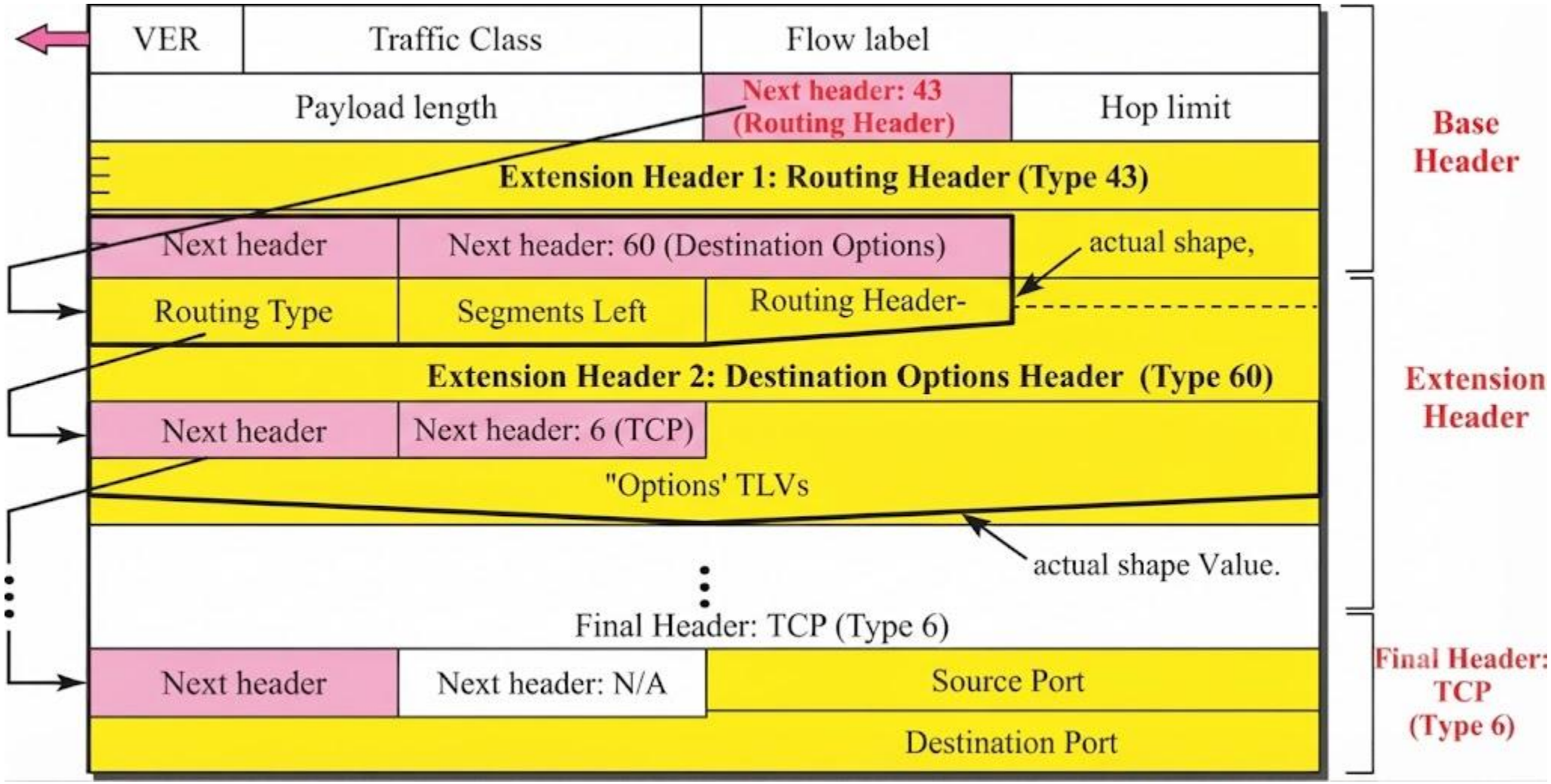
Next Header and Protocol: The Protocol field in IPv4 indicates the protocol carried in the payload. IPv6 renamed this field to Next Header, indicating a more meaningful name.

IPv6 Extension Headers

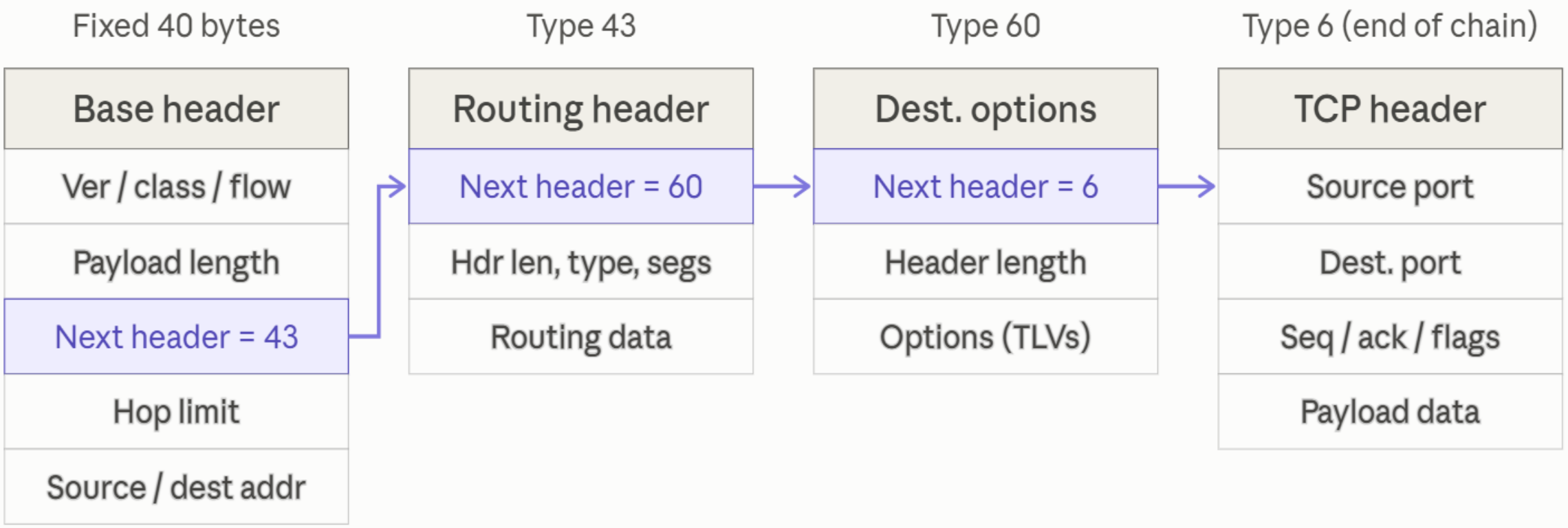
Instead of variable-length "Options" that slow down routers, IPv6 uses optional Extension Headers that are daisy-chained using the **Next Header** field.

Extension Header	Description
Hop-by-hop options	Miscellaneous information for routers on the path.
Destination options	Additional information for the final destination.
Routing	Loose list of routers to visit (similar to source routing).
Fragmentation	Used by the source to fragment packets (routers still don't do it).
Authentication	Verification of the sender's identity.
Encapsulating payload	Information about the encrypted contents.

IPv6 Extension Headers



IPv6 Extension Headers



Each Next header value is the type number of the header that follows it.

TCP (type 6) is the last header, so the chain ends there.

Internet Control Protocols

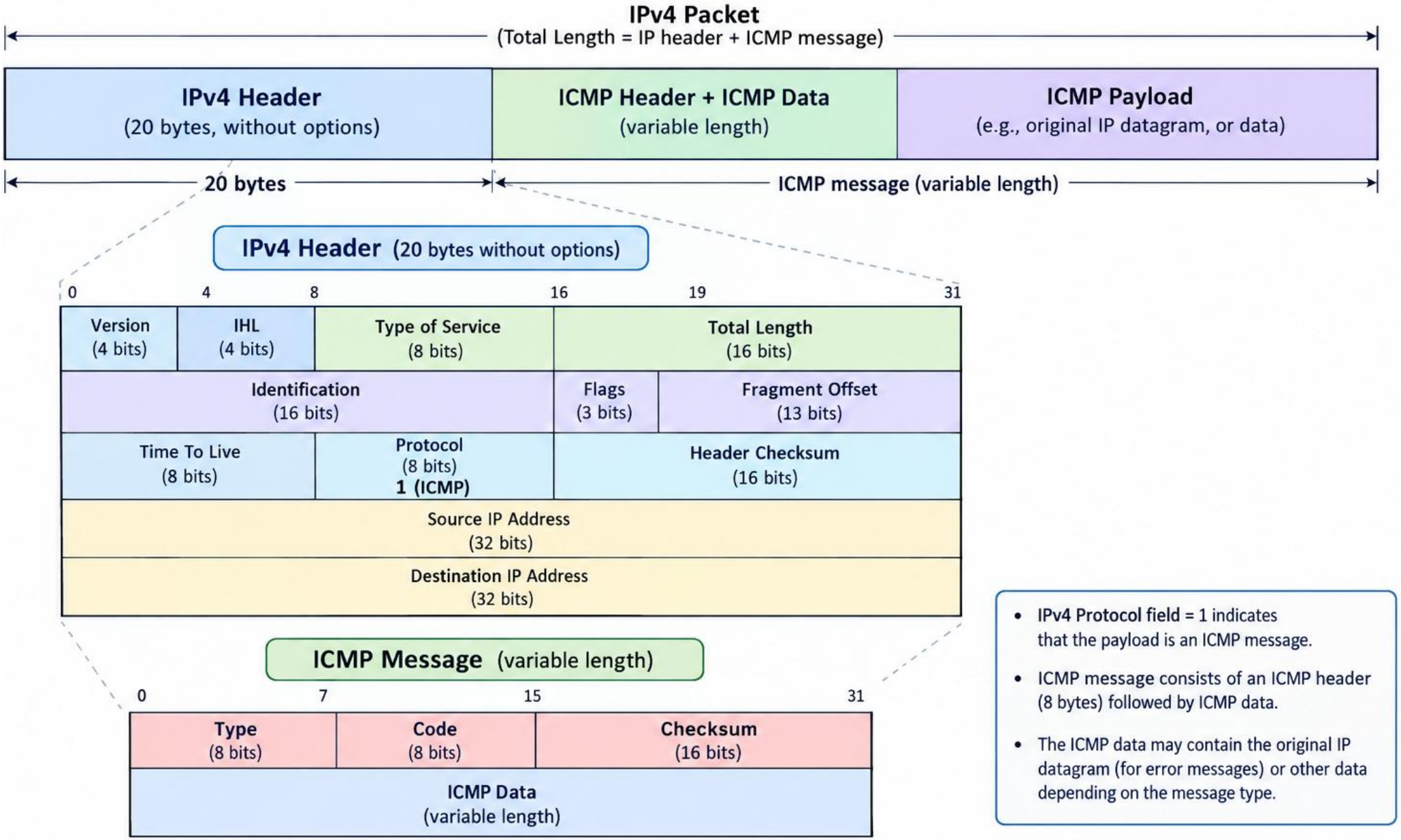
ICMP, ARP, and DHCP: The unsung heroes of network operations.

ICMP Message Types

The **Internet Control Message Protocol (ICMP)** is used by routers to report errors and unexpected events.

Message Type	Description
Destination Unreachable	Packet could not be delivered to the destination.
Time Exceeded	Time to live (TTL) hit 0. Packet was destroyed.
Parameter Problem	Invalid header field found.
Source Quench	Choke packet: tells sender to slow down (deprecated).
Redirect	Tells a host to use a different router for this destination.
Echo Request / Reply	Used by the **ping** utility to test connectivity.

ICMP Encapsulation



- **IPv4 Protocol field = 1** indicates that the payload is an ICMP message.
- ICMP message consists of an ICMP header (8 bytes) followed by ICMP data.
- The ICMP data may contain the original IP datagram (for error messages) or other data depending on the message type.

Address Resolution Protocol (ARP)

IP addresses are logical. Ethernet cards only understand physical MAC addresses. How does a host find the MAC address for a given IP?

1. The Broadcast

The sender broadcasts an ARP Request onto the local LAN: *"Who has IP 192.168.1.5? Tell 192.168.1.10"*

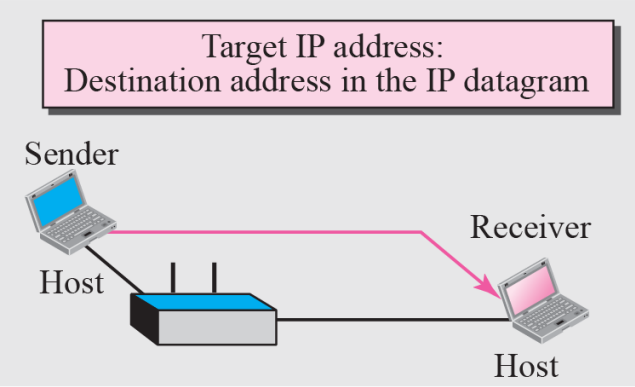
2. The Unicast Reply

The host owning that IP responds directly to the sender: *"I am 192.168.1.5, my MAC is 00:1A:2B:3C:4D:5E"*

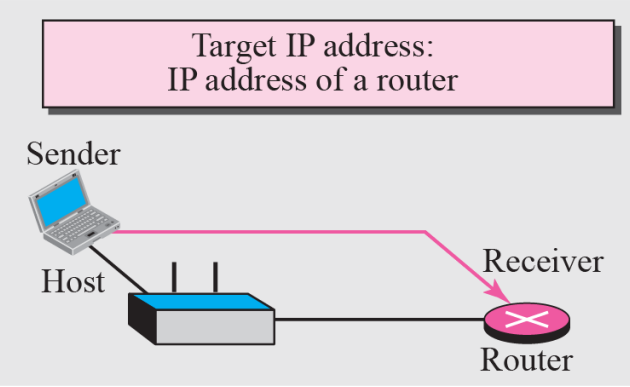
Four Cases Using ARP

ARP behavior changes depending on where the destination host is located.

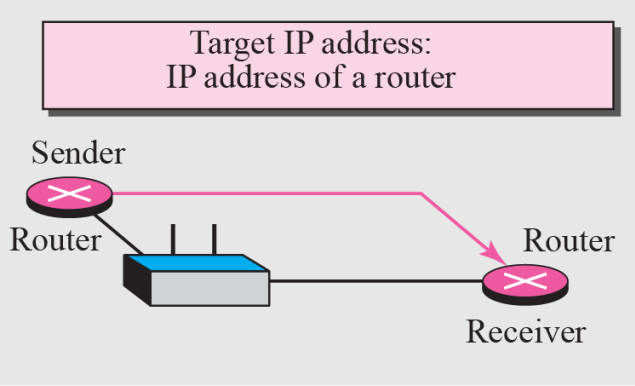
Case 1: A host has a packet to send to a host on the same network.



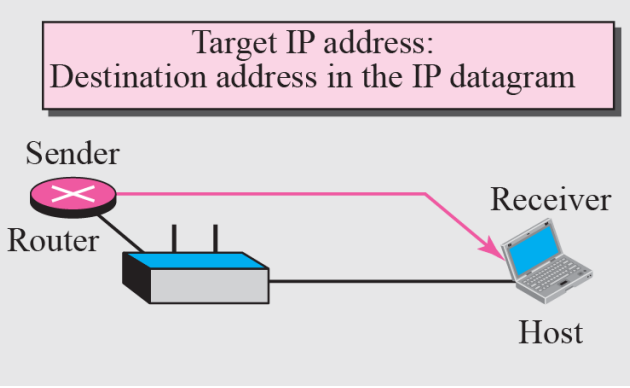
Case 2: A host has a packet to send to a host on another network.



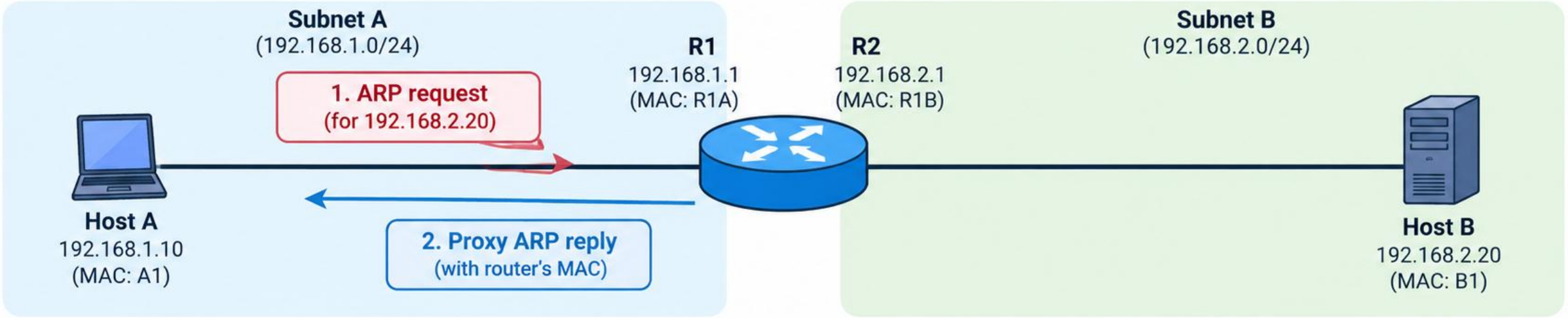
Case 3: A router has a packet to send to a host on another network.



Case 4: A router has a packet to send to a host on the same network.



Proxy ARP



1 Host A broadcasts ARP request

"Who has 192.168.2.20? Tell 192.168.1.10"

The destination is on a different subnet, so Host B does not see it.

2 Router replies (Proxy ARP)

"192.168.2.20 is reachable via me (MAC: R1A)"

The router knows the destination is on the other subnet and responds with its own MAC address.

3 Host A sends the data

Frame to 192.168.2.20 (with router's MAC)

Host A sends the packet to the router (thinking it is the destination).

4 Router forwards the packet

Router removes the L2 header and forwards to Host B

The router uses its routing table to deliver the packet to Host B on the other subnet.

When is it used?

- When a host thinks a destination is on its local subnet, but it is actually on a different subnet (separated by a router).
- Common in small networks or misconfigured subnets.

Why and how it works

- Avoids the host sending the ARP request beyond its subnet.
- Router acts as a proxy by replying with its own MAC address.
- Enables the host to send the packet to the router, which then forwards it.

DHCP Operation (DORA)

The Dynamic Host Configuration Protocol automates IP address assignment.

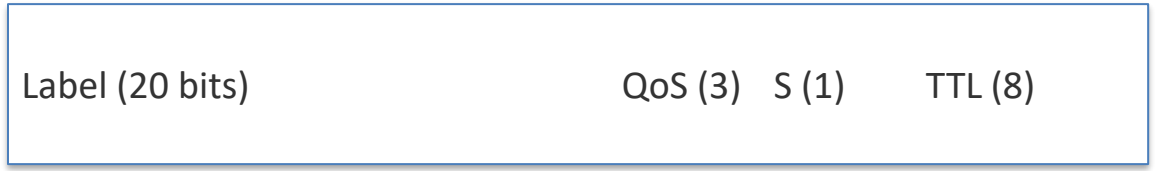
- **Discover:** Client broadcasts to find a server. `DHCPDISCOVER`
- **Offer:** Server responds with `DHCPOFFER` containing an IP address.
- **Request:** Client broadcasts to accept the offered IP. `DHCPREQUEST`
- **Acknowledge:** Server sends to finalize the lease. `DHCPACK`

Label Switching & MPLS

Bridging the gap between IP routing and fast hardware switching.

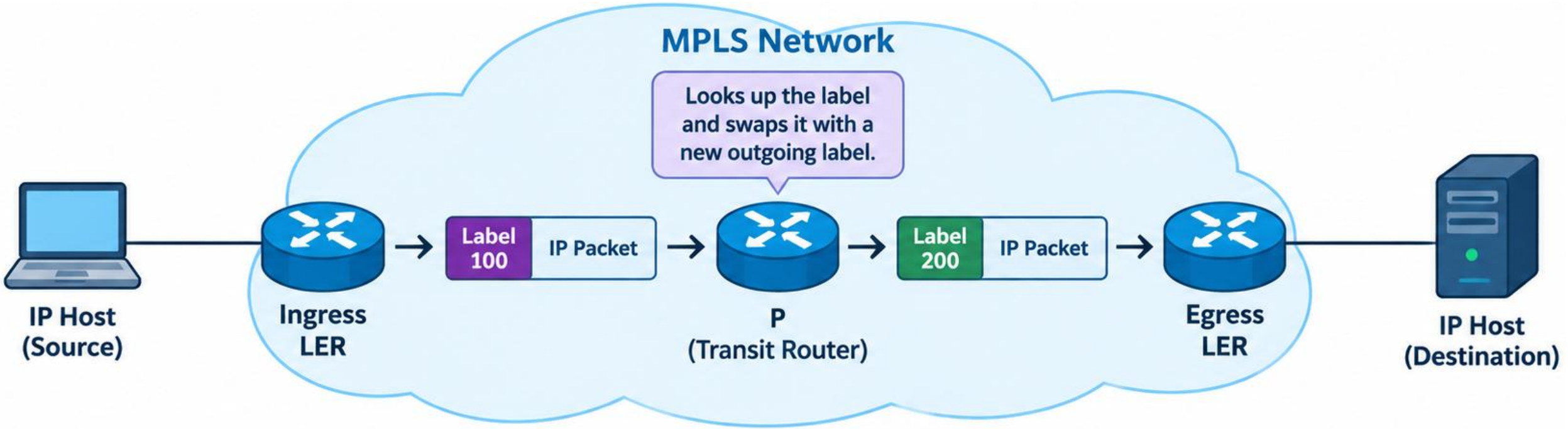
The MPLS Shim Header

Multi-Protocol Label Switching (MPLS) adds a small, 32-bit "Shim Header" directly in front of the IP header.



Why Label Switching? IP routing requires performing a complex Longest Prefix Match on a 32-bit address. MPLS uses a simple 20-bit label as an exact index into an array, making forwarding blazing fast in hardware.

Forwarding in MPLS



1 **IP packet enters an MPLS network.**

The edge router (ingress LER) attaches a label to the incoming IP packet.



2 **Core MPLS routers swap the label.**

Each core router ignores the IP header and only looks at the label.

3 **Label swapping.**

The router looks up the incoming label, replaces it with a new outgoing label, and forwards the packet.



4 **Repeat at each hop.**

The exit edge router strips the label.

5 **IP packet is forwarded.**

The router forwards the raw IP packet to the destination.



Interior vs Exterior Routing

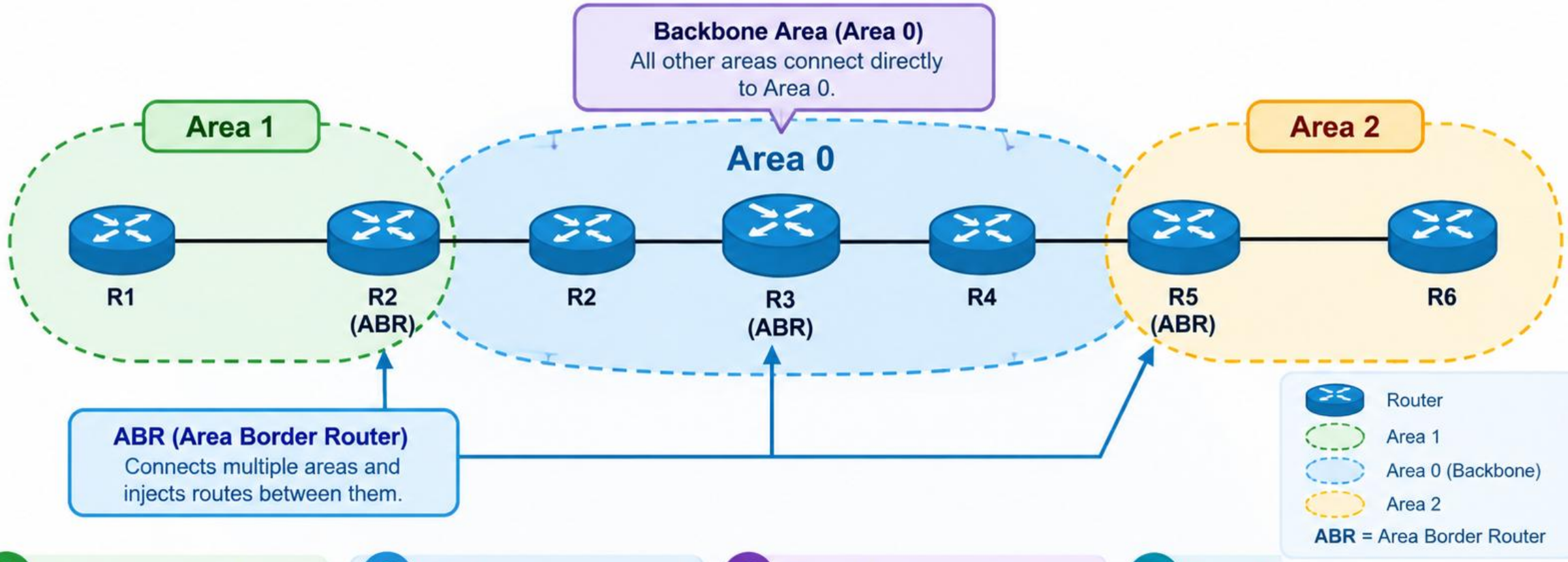
Navigating within Autonomous Systems (OSPF) vs between them (BGP).

OSPF (Interior Gateway)

Open Shortest Path First (OSPF) is an Interior Gateway Protocol used to route traffic *inside* a single Autonomous System (AS).

- It is a **Link State** protocol. Every router has a complete topological map of the AS.
- It uses Dijkstra's algorithm to calculate the shortest paths.
- Supports Equal Cost Multipath (ECMP) to split traffic over multiple equal routes.
- Rapidly detects and converges around link failures.

OSPF Areas and Hierarchy



1 Areas and Backbone

- OSPF divides the network into areas.
- Area 0 is the backbone area.
- All other areas must connect directly to Area 0.

2 ABRs

- Routers on the boundary of two areas are called Area Border Routers (ABRs).
- They summarize routes from their local area and inject them into the backbone.

3 Route Summarization

- ABRs summarize (aggregate) routes from their area before advertising them to other areas.
- This reduces the size of the routing table and keeps the network scalable.

4 Result

- Area 1 and Area 2 are separate areas.
- Both connect to Area 0 through their ABRs (R2 and R5).
- R3 is the backbone ABR (connecting to both areas), ensuring communication across the network.

Five Types of OSPF Messages

OSPF routers communicate using five specific message types to maintain their link state databases:

Message Type	Purpose
Hello	Used to discover who the neighbors are.
Link State Update	Provides the sender's actual costs to its neighbors.
Link State Ack	Acknowledges receipt of an Update packet.
Database Description	Announces which updates the sender has.
Link State Request	Requests specific information from a partner.

BGP (Exterior Gateway)

Border Gateway Protocol (BGP) is the protocol that glues the entire Internet together. It routes traffic *between* different Autonomous Systems.

Path Vector

Unlike OSPF, BGP does not track link costs. It advertises the exact sequence of ASes a packet must traverse (e.g., AS100 -> AS200 -> AS300).

Loop Prevention

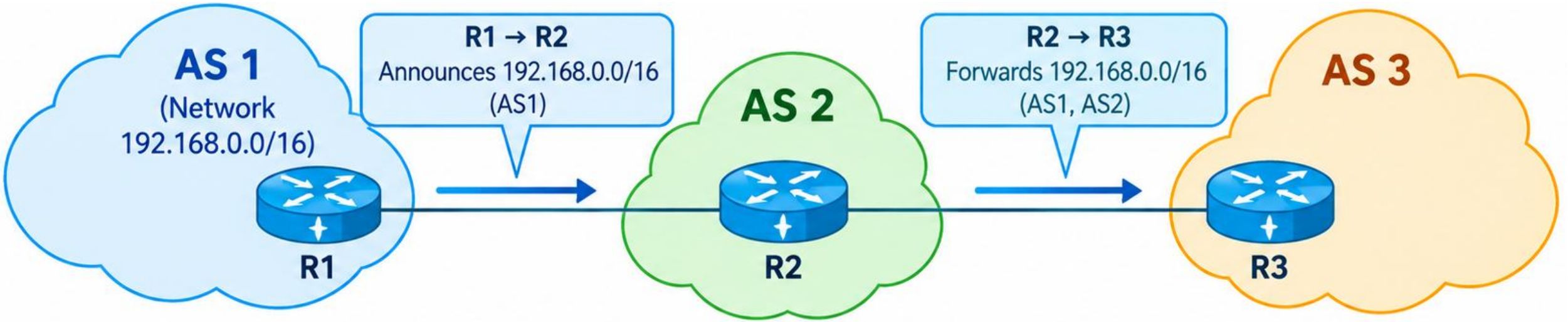
If a router receives a BGP advertisement and sees its own AS number already in the path, it drops the route to prevent a loop.

BGP Routing Policies

BGP is fundamentally about **Policy and Economics**, not technical efficiency.

- **Transit Costs:** An AS might refuse to route traffic through a specific provider because that provider charges too much.
- **Security/Politics:** An AS might be configured to explicitly never route data through a rival nation's infrastructure.
- **Peering Agreements:** Traffic might be routed over a longer physical path simply because a free peering agreement exists on that link.

Propagation of BGP Routes



1 R1 advertises to AS 2

R1 (in AS 1) connects to R2 (in AS 2) and advertises the network 192.168.0.0/16.

Route update:
192.168.0.0/16 (AS1)

2 R2 advertises to AS 3

R2 (in AS 2) receives the route and adds its own AS number, then advertises it to R3 (in AS 3).

Route update:
192.168.0.0/16 (AS1, AS2)

3 R3 learns the route

R3 (in AS 3) receives the route and installs it in its routing table.

Routing table at R3

Destination	AS Path
192.168.0.0/16	AS1, AS2

Result: AS 3 learns the route to 192.168.0.0/16 via AS 1 and AS 2.

Mobile IP

Staying connected while roaming the globe.

Mobile IP Goals

If a laptop moves from New York to Paris, its IP prefix changes, breaking all active TCP connections. Mobile IP solves this.

- Mobile host must be able to use its home IP address anywhere in the world.
- No software changes allowed to fixed hosts (the rest of the internet must remain unaware).
- No changes to core router software or routing tables.
- No overhead for a mobile host when it is sitting at home.

Mobile IP Routing Mechanism

How a packet reaches a roaming laptop:

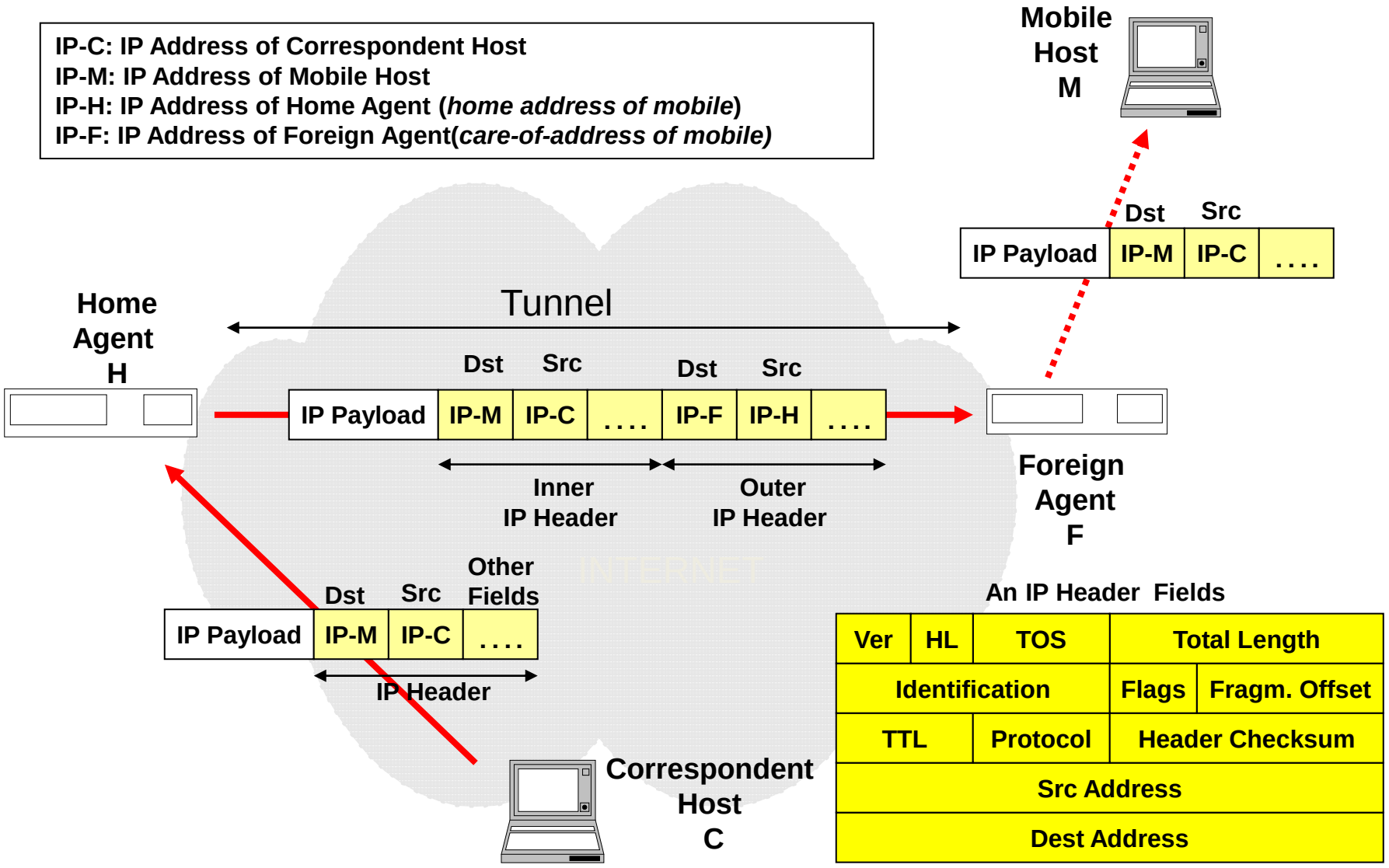
1. Care-of Address

The laptop arrives in Paris and gets a temporary local IP (Care-of Address). It registers this with its Home Agent.

2. Tunneling

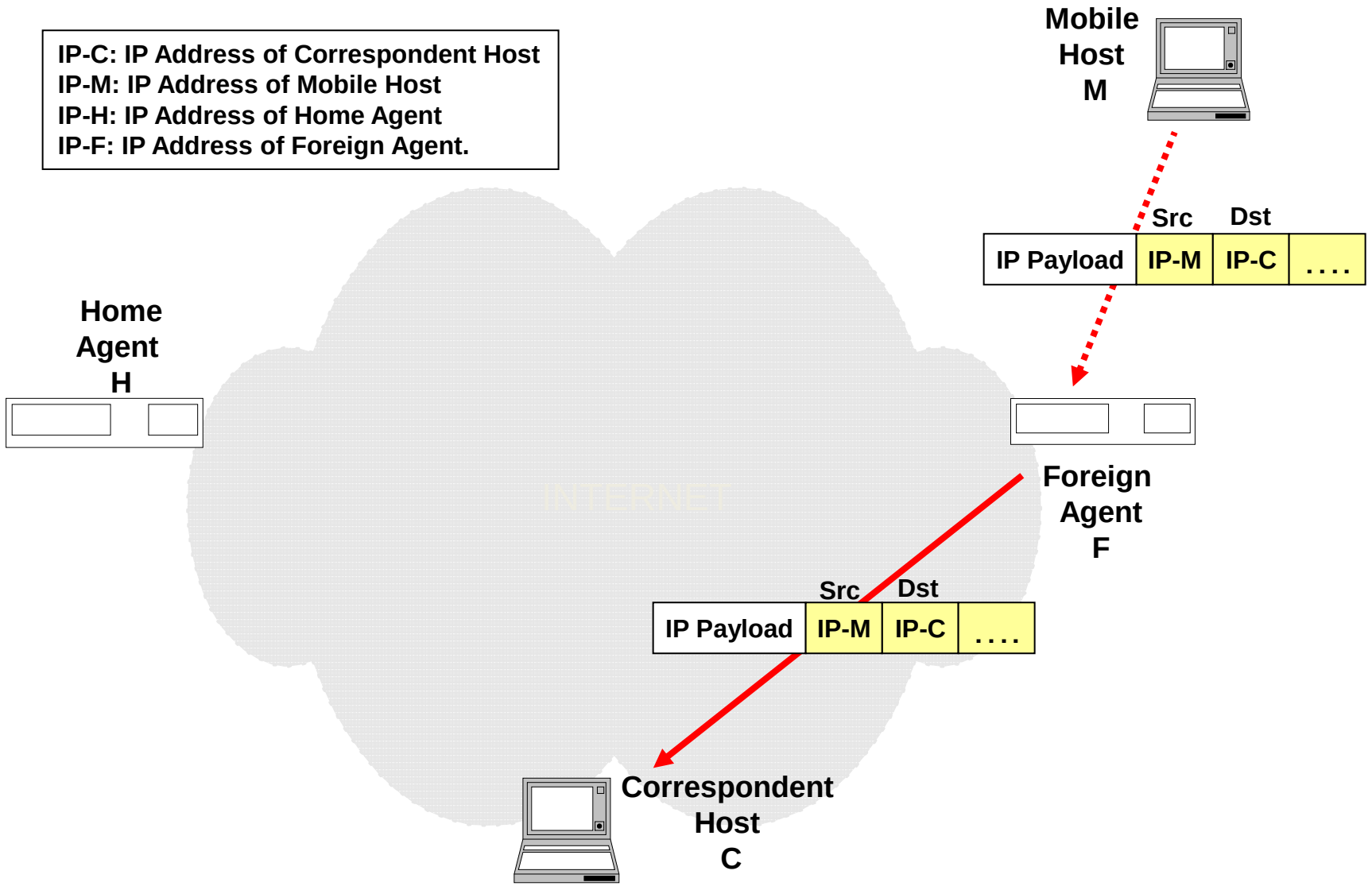
The sender sends packets to the laptop's Home IP. The Home Agent intercepts them, wraps them in a tunnel, and forwards them to Paris.

Mobile IP Tunneling



Packet Transport from a Correspondent Host to a Mobile

Mobile IP Tunneling



Packet Transport from a Mobile to a Correspondent Host

End of Chapter 5

Questions?